

I'm sharing this because the **PMTiles ecosystem** just had a burst of operationally relevant changes — new tooling, real-world quirks, and practical lessons you need to plan around.

What's new on the tooling front:

- **freestiler** is now being published (Apr 2026) as a fast R/Python package using a Rust/extendr backend that produces PMTiles archives from spatial data (MVT/MLT) with **MIT licensing** and zero external deps.
- **tipmtiles** (MIT, Development Seed) gives you a tiny Python tile server wrapper around PMTiles archives — useful when clients don't handle PMTiles natively or you need auth/CORS.
- PMTiles readers in various languages (Python/JS) and JS adapters are converging around **v3 support**, with incremental improvements in performance and metadata handling.

Ongoing operational realities:

- PMTiles is fundamentally a **single-file, HTTP-range-optimized archive** served remotely; clients fetch pieces via Range requests. This is efficient, but it brings real issues with **HTTP Range/Caching quirks** (browsers/CDNs or MapLibre clients may misinterpret headers, make stale requests, or crash when files are swapped in place).
- The format itself doesn't have native **atomic, in-place tile delta/patch semantics** yet — if you need incremental updates, you'll need to build your own pattern: generate **delta manifests**, publish **versioned/partitioned PMTiles files**, or sit a lightweight server (like tipmtiles/aiopmtiles) in front of static archives.

Practical battlefield tips:

- **Avoid in-place overwrites.** Use versioned filenames + sidecar with root_hash/spec_hash to help clients/CDNs recognize changes.
- **Partition large datasets** (region/zoom sharding) so you can swap shards atomically without touching the entire monster file.
- Test with both **MapLibre JS vs native clients** against your CDN's caching/Range behavior — the differences matter in production.
- Include the exact **tool + license strings** you used in your build receipts; reproducibility and legal compliance (e.g., MIT notice) matter for ops audits.

Here's a compact, implementable plan to make PMTiles artifacts verifiable, delta-friendly, and provenance-aware with a tiny signed sidecar.

Why this matters (quickly)

- **Integrity:** clients can verify any fetched tile range without hashing the whole archive.
 - **Deltas:** small patch bundles carry their own provenance and verify against a known base.
 - **Governance:** sidecar is signed (DSSE/cosign) and referenced in your promotion/run_receipt, preserving KFM's evidence chain.
-

1) Sidecar file: minimal schema

Create one **.pmtiles.attest.json** (or **.jsonl**) per **.pmtiles**.

```
{
  "schema_version": "v1",
  "pmtiles_filename": "terrain.pmtiles",
  "spec_hash": "blake3:... (hash of the PMTiles spec version string you target)",
  "root_hash": "blake3:... (BLAKE3 root of the full file)",
  "size_bytes": 123456789,
  "delta": {
    "is_patch": true,
    "delta_base_hash": "blake3:... (root of base file this patch applies to)"
  },
  "byte_ranges_manifest": [
    {
      "z": 12,
      "x": 1053,
      "y": 1532,
      "start": 987654,
      "end": 999321,
      "range_hash": "blake3:...",
      "bao_proof_ref": "terrain.pmtiles.bao#offset=987654&length=1667"
    }
  ],
  "attestations": {
    "build_id": "run-2026-05-10T00:12:45Z",
    "provenance": {
      "source_ledger_ref": "kfm://source/...",
      "run_receipt_ref": "kfm://receipt/...",
      "license_notes": [
        "PMTiles spec/impl: permissive (confirm in repo used).",
        "BLAKE3/BAO tooling: Apache-2.0/CC0 (confirm per library)."
      ]
    }
  }
}
```

```
}  
}
```

Notes

- `spec_hash` fixes the PMTiles version you targeted (hash the exact version string/tag).
 - `root_hash` is the BLAKE3 hash of the entire file (or BLAKE3 “root” if using Bao outboard).
 - `byte_ranges_manifest` lists minimal HTTP byte ranges for tiles (plus optional Bao proof ref).
-

2) Bao outboard proofs (streaming verification)

- Generate an outboard file alongside the PMTiles (e.g., `terrain.pmtiles.bao`).
 - Each manifest entry includes a `bao_proof_ref` that points to the proof segment for that tile’s byte range.
 - Server keeps proofs small and adjacent to the main file; client fetches **tile bytes + proof** via HTTP Range and verifies locally against `root_hash`.
-

3) Signing (DSSE/cosign)

- Wrap the sidecar JSON in **DSSE** and sign with **cosign**.
- Publish `*.attest.json` and `*.attest.json.sig` (or cosign bundle) together.
- Include the **attestation reference** (`attestation_ref`) in your **promotion/run_receipt** so governance artifacts point to exactly what got published.

CLI sketch

```
cosign attest --predicate terrain.pmtiles.attest.json \  
--type kfm.pmtiles.sidecar --key cosign.key \  
--yes terrain.pmtiles
```

4) Client verification flow (pseudocode)

```
async function fetchTile(z,x,y) {
  const sidecar = await getJSON("terrain.pmtiles.attest.json");
  await verifyDSSE(sidecar, trustedKeys);

  const entry = indexLookup(sidecar.byte_ranges_manifest, z,x,y);
  const { start, end, bao_proof_ref } = entry;

  const [bytes, proof] = await Promise.all([
    httpRange("terrain.pmtiles", start, end),
    fetchProof(bao_proof_ref)
  ]);

  if (!baoVerify(bytes, proof, sidecar.root_hash)) throw new Error("BAD_PROOF");
  return decodeTile(bytes);
}
```

5) Delta patches

- Patch `.pmtiles` files ship with:
 - `root_hash` = hash of the **patch file**
 - `delta.is_patch` = `true`
 - `delta_base_hash` = hash of the **base full file** clients must have (or be able to fetch)
 - Publish a tiny **Bao-encoded delta bundle** (tile-level changes + proofs). Clients verify each changed range against the patch's `root_hash`, and (optionally) the base's `root_hash` for cross-check.
-

6) KFM governance hooks (how this slots in)

- **Evidence**: sidecar is an **EvidenceBundle carrier** for integrity + provenance metadata.
- **Receipts**: promotion/run receipts include `attestation_ref` and the content hashes.

- **Publication:** only PUBLISHED state exposes `*.pmtiles`, sidecar, and proofs via governed endpoints.
 - **Policy:** validators fail closed when:
 - missing `root_hash` / `spec_hash`,
 - patch lacks `delta_base_hash`,
 - signature or key trust chain fails,
 - sensitive layers lack approved posture.
-

7) Minimal repository tasks (Definition of Done)

- Add `tools/attest/pmtiles_sidecar.py` to emit the sidecar + Bao outboard.
 - Add `tools/validators/validate_pmtiles_sidecar.py` (schema + signature + link checks).
 - CI job: build PMTiles → compute BLAKE3 + Bao → write sidecar → **cosign attest** → attach artifacts.
 - Gate: deny publication if sidecar/signature/proofs missing or invalid.
 - Docs: short “How clients verify tiles” page with 20-line example.
-

8) Tiny JSON Schema (draft)

```
{
  "$id": "kfm://schema/pmtiles-sidecar-v1",
  "type": "object",
  "required":
  ["schema_version", "pmtiles_filename", "spec_hash", "root_hash", "byte_ranges_manifest"],
  "properties": {
    "schema_version": { "enum": ["v1"] },
    "pmtiles_filename": { "type": "string" },
    "spec_hash": { "type": "string", "pattern": "^blake3:" },
    "root_hash": { "type": "string", "pattern": "^blake3:" },
    "size_bytes": { "type": "integer", "minimum": 0 },
    "delta": {
      "type": "object",
      "required": ["is_patch"],
      "properties": {
```

```

    "is_patch": { "type": "boolean" },
    "delta_base_hash": { "type": "string", "pattern": "^blake3:" }
  }
},
"byte_ranges_manifest": {
  "type": "array",
  "items": {
    "type": "object",
    "required": ["start", "end", "range_hash"],
    "properties": {
      "z": { "type": "integer", "minimum": 0 },
      "x": { "type": "integer", "minimum": 0 },
      "y": { "type": "integer", "minimum": 0 },
      "start": { "type": "integer", "minimum": 0 },
      "end": { "type": "integer", "minimum": 0 },
      "range_hash": { "type": "string", "pattern": "^blake3:" },
      "bao_proof_ref": { "type": "string" }
    }
  }
},
"attestations": { "type": "object" }
}
}

```

9) Practical notes

- **Hashing:** prefer BLAKE3 everywhere; use **Bao outboard** for streaming verification.
- **Licenses:** keep a short license note in the sidecar and in your NOTICE. Confirm licenses for any vendored libs.
- **Backward-compat:** start by listing ranges only for index + a sample tile set; expand coverage as your pipeline stabilizes.
- **Storage:** proofs are small; co-locate next to PMTiles or in a parallel [/proofs/](#) path.

If you want, I'll drop in a tiny [tools/attest/](#) script (Python/Go) and a CI job snippet you can paste into your repo.

Excellent.

Below is a repo-ready implementation slice aligned to KFM governance posture and your [tools/](#) doctrine.

Proposed Repository Additions

```
tools/
├── attest/
│   ├── README.md
│   ├── pmtiles_sidecar.py
│   ├── generate_bao.sh
│   └── schema/
│       └── pmtiles_sidecar.schema.json
├── validators/
│   ├── validate_pmtiles_sidecar.py
│   └── fixtures/
│       ├── valid/
│       └── invalid/
└── .github/
    └── workflows/
        └── pmtiles-attestation.yml
```

tools/attest/README.md

PMTiles Attestation Utilities

Governed integrity helpers for PMTiles publication artifacts.

Purpose

This lane generates:

- PMTiles sidecar attestations
- BLAKE3 integrity roots
- Bao outboard proofs
- byte-range manifests
- DSSE/cosign-ready predicates

These utilities support KFM publication integrity but do NOT determine publication truth, admissibility, policy allow/deny, or canonical authority.

Outputs

```
| Artifact | Purpose |  
|---|---|  
| `.pmtiles.attest.json` | Integrity + provenance sidecar |  
| `.bao` | Bao outboard proof file |  
| `.sig` | DSSE/cosign signature bundle |  
| `ValidationReport` | Fail-closed validator output |
```

Constraints

- No secrets
- No policy decisions
- No direct publication
- No canonical truth mutation
- Fail closed on missing hashes/proofs

Lifecycle Position

```
```text  
RAW -> WORK -> PROCESSED -> CATALOG -> PUBLISHED
 ^
 |
 attestation generated here
...`
```

Publication remains a governed state transition.

---

## tools/attest/pmtiles\_sidecar.py

```
#!/usr/bin/env python3
```

```
import os
import json
import hashlib
from pathlib import Path
```

```
SCHEMA_VERSION = "v1"
```

```
def blake3_file(path):
 try:
 import blake3
 except ImportError:
 raise SystemExit("Missing dependency: pip install blake3")
```



```
h = blake3.blake3()
```

```
with open(path, "rb") as f:
 while chunk := f.read(1024 * 1024):
 h.update(chunk)
```

```
return f"blake3:{h.hexdigest()}"
```

```
def build_sidecar(pmtiles_path, output_path=None):
```

```
 pmtiles_path = Path(pmtiles_path)
```

```
 if not pmtiles_path.exists():
 raise FileNotFoundError(pmtiles_path)
```

```
 root_hash = blake3_file(pmtiles_path)
```

```
 data = {
 "schema_version": SCHEMA_VERSION,
 "pmtiles_filename": pmtiles_path.name,
 "root_hash": root_hash,
 "size_bytes": pmtiles_path.stat().st_size,
 "spec_hash": "NEEDS_VERIFICATION",
 "delta": {
 "is_patch": False
 },
 "byte_ranges_manifest": [],
 "attestations": {
 "build_id": "NEEDS_VERIFICATION",
 "provenance": {
 "source_ledger_ref": "NEEDS_VERIFICATION",
 "run_receipt_ref": "NEEDS_VERIFICATION"
 }
 }
 }
```

```
 if output_path is None:
 output_path = pmtiles_path.with_suffix(".pmtiles.attest.json")
```

```
 with open(output_path, "w") as f:
 json.dump(data, f, indent=2)
```

```
 print(f"Wrote {output_path}")
```

```
if __name__ == "__main__":

 import argparse

 parser = argparse.ArgumentParser()
 parser.add_argument("pmtiles")

 args = parser.parse_args()

 build_sidecar(args.pmtiles)
```

---

## Bao Generation Helper

[tools/attest/generate\\_bao.sh](#)

```
#!/usr/bin/env bash
set -euo pipefail

FILE="$1"

bao encode "$FILE" --outboard "${FILE}.bao"

echo "Generated ${FILE}.bao"
```

---

## Validator

[tools/validators/validate\\_pmtiles\\_sidecar.py](#)

```
#!/usr/bin/env python3

import json
import sys

REQUIRED_FIELDS = [
 "schema_version",
 "pmtiles_filename",
 "root_hash",
 "byte_ranges_manifest"
```

```

]

def fail(msg):
 print(f"DENY: {msg}")
 sys.exit(1)

def validate(path):

 with open(path) as f:
 data = json.load(f)

 for field in REQUIRED_FIELDS:
 if field not in data:
 fail(f"missing field: {field}")

 if not data["root_hash"].startswith("blake3:"):
 fail("root_hash must use blake3")

 if data.get("delta", {}).get("is_patch"):

 if "delta_base_hash" not in data["delta"]:
 fail("patch missing delta_base_hash")

 print("CONFIRMED: validation passed")

if __name__ == "__main__":
 validate(sys.argv[1])

```

---

## CI Workflow

[.github/workflows/pmtiles-attestation.yml](https://github.com/pmtiles/pmtiles-attestation.yml)

name: PMTiles Attestation

on:

workflow\_dispatch:

push:

paths:

- "data/\*\*/\*.pmtiles"

jobs:

attest:

runs-on: ubuntu-latest

steps:

- name: Checkout  
uses: actions/checkout@v4
  - name: Install Python  
uses: actions/setup-python@v5  
with:  
python-version: "3.12"
  - name: Install dependencies  
run: |  
pip install blake3  
cargo install bao
  - name: Generate sidecars  
run: |  
find data -name "\*.pmtiles" | while read f; do  
python tools/attest/pmtiles\_sidecar.py "\$f"  
bash tools/attest/generate\_bao.sh "\$f"  
done
  - name: Validate sidecars  
run: |  
find data -name "\*.attest.json" | while read f; do  
python tools/validators/validate\_pmtiles\_sidecar.py "\$f"  
done
- 

# Optional Next-Step Extensions

## 1. Tile-Level Range Indexing

Later enhancement:

```
{
 "z": 8,
 "x": 42,
```

```
"y": 91,
"start": 991231,
"end": 992847,
"range_hash": "blake3:..."
}
```

This enables:

- HTTP Range verification
- partial fetch proofs
- CDN-safe integrity
- peer mirroring
- deterministic patching

---

## 2. DSSE Predicate Type

Recommended predicate:

kfm.pmtiles.sidecar/v1

---

## 3. Governance Gate

Recommended fail-closed publication gate:

Condition	Outcome
Missing sidecar	DENY
Invalid signature	DENY
Missing Bao proof	DENY

Unknown spec_hash	DENY
Patch without base hash	DENY
Unresolved provenance refs	REVIEW_REQUIRED

---

# Recommended Future KFM Objects

Potential future governed objects:

Object	Purpose
PMTilesIntegrityEnvelope	Publication integrity wrapper
TileRangeProof	Byte-range verification object
DeltaPromotionReceipt	Patch lineage
GeoArtifactManifest	Multi-artifact spatial release
PublicTileReleaseManifest	Governed public-safe exposure

---

## Strong Architectural Fit (CONFIRMED/INFERRED)

Area	Alignment
Evidence-first	Strong
Derived artifact governance	Strong
Public-safe release model	Strong
Fail-closed validation	Strong
Promotion receipt lineage	Strong
MapLibre/PMTiles distribution	Strong
Future STAC linkage	Strong
AI/runtime separation	Strong

---

## Recommended Immediate Priority

Order:

1. Root hash generation
2. Bao outboard proofs
3. Validator gates
4. DSSE signing
5. Tile-range manifests
6. Delta patch lineage
7. Client-side streaming verification

That sequence keeps changes reversible and CI-safe while preserving your KFM publication doctrine.

I'm sharing this because PMTiles is clearly heading into its *stable future* — the format itself, its tooling, and language ecosystems are all being actively maintained and improved with real 2026 releases and concrete v3 support that you can depend on for serious production use.

PMTiles is a **single-file, cloud-optimized archive format for map tiles** — designed to replace huge directories of individual tiles with one compact file that can be read over HTTP range requests with minimal overhead. The *official spec on GitHub* confirms that **v3 is the current and maintained version** with improvements around compressed directories, metadata limits, and internal indexing.

The core **go-pmtiles CLI** tooling is released under a permissive BSD-3-Clause license and continues to receive maintenance and important fixes — including in 2026 releases — making it suitable for CI/CD workflows as a pinned dependency. Tools like `pmtiles merge`, `extract`, and other commands make workflow automation easier and more reliable.

Across ecosystems, official packages are **explicitly adding or shipping v3 support**: the npm `pmtiles` library (recent 2026 version) can be used in browsers and Node for MapLibre, Leaflet, and other adapters, and supports multiple tile types. There are also bindings and readers across languages (e.g., Dart/Flutter) that work with v3 archives.

In practice this means you can treat **PMTiles v3 as stable for new projects**, and build pipelines that generate, verify, and serve `.pmtiles` artifacts with reproducible CLI/library versions rather than tracking rolling “latest” installs — which is exactly the operational pattern you hinted at.

Here's a practical, CI-friendly way to make your Master MapLibre PDF grow safely each run without ever losing history:

## Snapshot-plus-Delta merge strategy for the Master MapLibre PDF



**Idea:** treat every build as an immutable snapshot of what you know **plus** a machine-readable delta (a JSONL manifest). You never delete; you only supersede with full provenance.

### How it works—tight loop

1. **Inputs:** previous published `manifest.jsonl` (if any) + new discoveries from this run.
2. **Normalize & hash:** compute a stable `spec_hash` per item (hash of canonicalized fields) and assign a durable `stable_id`.
3. **Diff:** compare against last manifest by `spec_hash` → classify rows as `added` | `modified` | `unchanged` | `superseded`.
4. **Field-granular reconcile (modified only):**
  - Auto-merge non-conflicting fields.
  - If conflict, record both versions; mark older as `status: "superseded"` and set `superseded_by: <decision_id>`.
5. **Fail-closed gating:** any `added` or `reconciled` item must carry a **signed `run_receipt`** (`decision_id`, `attestation_ref`).
  - If missing, route to **QUARANTINE** manifest (excluded from “active” master).
6. **Publish artifacts (one user-facing PDF):**
  - Append short narrative pages for **new** discoveries (human-readable).
  - **Embed** the canonical `manifest.jsonl` inside the PDF as a PDF/A-3 attachment (or ship it as a sidecar file next to the PDF). Also embed the **previous** manifest for audit trail.
  - Prefer **append-only** updates so PDF bytes stay history-preserving.
7. **Index for downstream tools:** expose a compact index mapping `spec_hash` → `byte-range (in PDF)` or `attachment filename` so MapLibre wiring can deterministically resolve any element back to evidence & attestation.

### Minimal JSONL record (one line per item)

```
{
 "stable_id": "ml:source-style-override:00123",
 "spec_hash": "blake3:8a2f...",
 "source_head": "git:abc1234",
 "evidence_refs": ["stac:items/...", "url:https://..."],
 "summary": "Style override behavior for symbol-sort-key...",
 "provenance": {"ingest_job": "maplibre-scan@2026-05-10T03:12Z"},
 "status": "active", // or "superseded"|"quarantine"
 "superseded_by": null, // set when superseded
 "run_receipt": {
 "decision_id": "dec:2026-05-10:00041",
 "attestation_ref": "dsse:sha256:5c9b..."
 }
}
```

## CI gates to add (fast and deterministic)

- **Schema check:** JSONL lines validate and canonicalize before hashing.
- **Determinism check:** re-run hashing on the same input → identical `spec_hash` set.
- **Attestation check:** every `active` item has a valid `run_receipt`; otherwise auto-route to `QUARANTINE`.
- **Reconcile check:** when conflicts occur, old record must be marked `superseded` and point to the new `decision_id`.
- **PDF packing check:** PDF/A-3 attachment(s) present; index file present; byte-range map valid.

## What you get

- Immutable history (auditable).
- Clean “active” view for MapLibre while retaining superseded knowledge.
- Deterministic, machine-resolvable links from any PDF element back to evidence & signatures.

If you want, I can drop in a tiny Python script that: (a) canonicalizes items, (b) computes `spec_hash`, (c) emits `manifest.jsonl`, (d) produces the added/modified/unchanged/superseded report, and (e) writes a compact `index.json`.

Excellent. Here’s the next step: a compact, production-grade implementation skeleton you can drop into your KFM/MapLibre pipeline immediately.

# Canonical Merge Pipeline Layout

```
tools/
 diff/
 merge_master_manifest.py
 canonicalize.py
 hash_util.py
 reconcile.py
schemas/
 manifest_record.schema.json
artifacts/
 manifests/
 manifest-2026-05-10.jsonl
 manifest-latest.jsonl
indexes/
 index-2026-05-10.json
quarantine/
 quarantine-2026-05-10.jsonl
```

# Canonicalization Rules (critical)

Determinism matters more than elegance.

## Normalize Before Hashing

Always:

- lowercase enum values
- trim whitespace
- sort arrays
- sort object keys recursively
- normalize Unicode (NFKC)
- remove volatile timestamps
- strip transient runtime fields

Do **not** hash:

- `run_receipt`
- ingestion timestamps
- CI build IDs
- PDF byte offsets
- temporary provenance metadata

Only hash semantic content.

---

## Recommended Stable Hash

Use:

- `BLAKE3` preferred
- `SHA256` acceptable fallback

Canonical string example:

```
{
 "component": "maplibre-layer-registry",
 "category": "rendering",
 "title": "Symbol Sort Key",
 "behavior": "...",
 "evidence_refs": [
 "url:https://...",
 "pdf:maplibre-guide-p42"
]
}
```

Then:

```
spec_hash = blake3(canonical_json_bytes).hexdigest()
```

---

# Merge State Machine

## Existing Hash + Same Stable ID

→ **unchanged**

## Same Stable ID + Different Hash

→ **modified**

## New Stable ID

→ **added**

## Old Active Record Replaced

→ mark old:

```
{
 "status": "superseded",
 "superseded_by": "dec:2026-05-10:0041"
}
```

Never delete.

---

# Recommended Reconcile Policy

## Auto-Merge Allowed

Safe fields:

- tags
- citations
- evidence\_refs
- aliases
- notes
- examples

## Manual / Governed Merge Required

High-risk fields:

- normative behavior
- API contract semantics
- security posture
- public-safe exposure rules
- rendering guarantees
- policy assertions

If conflict:

```
{
 "status": "quarantine",
 "conflict_reason": "behavior mismatch",
 "candidate_versions": [...]
}
```

Fail closed.

---

## CI Workflow Pattern

## **Stage 1 — Intake**

Extract candidate ideas from PDFs/docs.

## **Stage 2 — Normalize**

Canonicalize records.

## **Stage 3 — Validate**

Schema + policy validation.

## **Stage 4 — Diff**

Compare against latest manifest.

## **Stage 5 — Reconcile**

Resolve non-conflicting updates.

## **Stage 6 — Quarantine**

Route unresolved conflicts.

## **Stage 7 — Publish**

Generate:

- updated JSONL
- index
- PDF appendix
- release manifest

---

# **PDF Embedding Strategy**

Best option:

## PDF/A-3 Attachments

Attach:

- `manifest-latest.jsonl`
- previous manifest snapshot
- `index.json`
- attestation receipt

This preserves:

- auditability
- machine readability
- historical lineage

without bloating visible pages.

---

## Byte-Range Index Example

```
{
 "spec_hash_index": {
 "blake3:8a2f...": {
 "pdf_page": 144,
 "attachment": "manifest-latest.jsonl",
 "line": 884
 }
 }
}
```

This becomes extremely powerful for:

- EvidenceDrawer
  - Focus Mode
  - governed APIs
  - reproducible citations
  - renderer traceability
-

# Strong Recommendation: Separate “Narrative” from “Truth”

The PDF itself should be treated as:

- a human navigation layer
- a curated publication artifact

The JSONL manifest is the canonical machine truth.

That aligns perfectly with KFM doctrine:

generated/public artifacts are downstream carriers, not sovereign truth.

---

## Governance Gates You Should Absolutely Add

### Hard Failures

Reject publish if:

- duplicate `stable_id`
  - duplicate active `spec_hash`
  - missing `run_receipt`
  - unresolved evidence refs
  - invalid schema
  - malformed supersession chain
  - orphaned quarantine records
  - missing provenance
  - non-deterministic hash output
- 

## Recommended Additional Objects

You already align closely with these KFM families:



- RunReceipt
- ReleaseManifest
- DecisionEnvelope
- ValidationReport
- CorrectionNotice
- RollbackPlan

I strongly recommend introducing:

## MergeDecision

```
{
 "decision_id": "dec:2026-05-10:0041",
 "decision_type": "reconcile",
 "inputs": [...],
 "outputs": [...],
 "policy_result": "allow",
 "attestation_ref": "dsse:sha256:..."
}
```

This becomes your audit spine.

---

# Long-Term Scaling Recommendation

Eventually split the monolithic master into:

```
master/
 rendering/
 tiles/
 styles/
 shaders/
 terrain/
 pmtiles/
 vector-tiles/
 controls/
 interactions/
 governance/
```

while retaining:

Master MapLibre Components-Functions-Features.pdf

as a federated compiled release.

This prevents future merge explosions.

---

## Final Architecture Summary

The strongest durable model here is:

Immutable snapshots

- + append-only manifests
- + deterministic hashes
- + governed reconciliation
- + quarantine lane
- + embedded machine-readable lineage
- + PDF as human release surface

That gives you:

- reproducibility
- auditability
- CI safety
- rollback capability
- evidence traceability
- publication governance
- deterministic MapLibre integration
- long-term scaling stability

without losing historical intelligence across runs.

Here's a compact, implementable pattern to make every Master PDF build verifiable end-to-end with a signed run\_receipt, DSSE, and cosign—so the PDF “gets smarter” each run.

---

## What you implement

### 1) Canonical

# run\_receipt.json

## (hash-stable)

- Canonicalize with JCS (JSON Canonicalization Scheme).
- Include a `spec_hash` (sha256 over the canonicalized object below with empty `signature_*` fields).
- Treat `decision_id` as the join key across policy decisions, receipts, attestations, and the release index.

```
{
 "decision_id": "kfm://decision/<uuid>",
 "build_id": "kfm://build/<uuid-or-ci-run>",
 "builder_ci_run_id": "github:org/repo/actions/runs/1234567890",
 "kfm_spec_version": "v1",
 "sources": [
 {
 "url": "https://example.org/source1.tif",
 "etag": "W/\"e7-abc123\"",
 "last_modified": "Wed, 08 May 2026 17:04:03 GMT",
 "content_length": 187654321,
 "source_spec_hash": "sha256-..."
 }
],
 "added_ids": ["kfm://obj/<new-thing-1>", "kfm://obj/<new-thing-2>"],
 "superseded_ids": ["kfm://obj/<old-thing-9>"],
 "manifest_ref": "oci://registry.example.com/kfm/releases/master-maplibre@sha256:...",
 "manifest_spec_hash": "sha256-...",
 "attestation_ref": "", // filled after cosign upload (e.g., Rekor index / OCI ref)
 "signature_kid": "", // filled after signing
 "signature_algo": "", // e.g., "sigstore-cosign-keyless-sha256"
 "timestamp": "2026-05-10T00:00:00Z",
 "policy_bundle_version": "policy/ecology@sha256:...",
 "decision_log_ref": "s3://kfm-logs/opa/2026/05/10/decisionlog.jsonl",
 "spec_hash": "" // sha256 of JCS-normalized JSON with this field blank
}
```

## 2) Wrap in DSSE + sign with cosign (keyless OIDC in CI)

- DSSE envelope payload = the canonicalized `run_receipt.json`.

- Sign with `cosign attest --predicate run_receipt.json --predicate-type application/vnd.kfm.run_receipt+json` (keyless).
- Record the attestation reference (OCI digest and/or Rekor index) back into:
  - the DSSE envelope annotations,
  - the `run_receipt.json` fields (`attestation_ref`, `signature_*`),
  - the release manifest,
  - and the PDF/A-3 attachments.

### 3) Attach artifacts to the PDF each run

- Attach to **Master MapLibre Components-Functions-Features.pdf** as PDF/A-3 embedded files:
  - `manifest.jsonl` (release manifest),
  - `run_receipt.json` (canonical),
  - `run_receipt.dsse.json` (DSSE envelope),
  - `run_receipt.sig` (cosign bundle if separate),
  - and a short `release_index.jsonl` pointer line (see §5).
- Keep prior runs' attachments; mark superseded in the manifest—don't delete.

### 4) Publish to tamper-evident stores

- **OCI (ORAS)**: push sidecars (`*.jsonl`, `run_receipt.json`, DSSE) as artifacts; use cosign attestation so Rekor has transparency entry; use digest-pinned refs in manifests.
- **S3 (versioned)**: mirror the exact same files to S3 with ObjectLock (compliance or governance mode) and KMS; keep retention policies aligned with regulatory needs.

### 5) Release index (tiny JSONL for fast lookup)

Append **one line per build** (machine-append-only) to a public index:

```
{"ts":"2026-05-10T00:00:00Z","decision_id":"kfm://decision/<uuid>","attestation_ref":"oci://...@sha256:...#rekorIndex=1234567"}
```

This lets anyone jump from build time → decision\_id → attestation.

---

## CI wiring (GitHub Actions example)

name: master-pdf-build

on: [workflow\_dispatch, push]

jobs:

build-release:

runs-on: ubuntu-latest

permissions:

id-token: write *# for cosign keyless*

contents: read

packages: write

steps:

- uses: actions/checkout@v4

- name: Set up cosign & oras

uses: sigstore/cosign-installer@v3

- run: |

curl -L https://github.com/oras-project/oras/releases/latest/download/oras\_\$(uname  
-s)\_\$(uname -m).tar.gz \

| tar -xz && sudo mv oras /usr/local/bin/oras

- name: Build Master PDF

run: |

# 1) produce Master MapLibre Components-Functions-Features.pdf

# 2) produce manifest.jsonl and run\_receipt.json (JCS canonicalized with blank  
spec\_hash/signature fields)

./scripts/build\_master\_pdf.sh

- name: Canonicalize & spec\_hash

run: |

python tools/attest/jcs\_hash.py run\_receipt.json > spec\_hash.txt

jq --arg h "sha256-\$(cat spec\_hash.txt)" '.spec\_hash=\$h' run\_receipt.json >  
run\_receipt.canon.json

mv run\_receipt.canon.json run\_receipt.json

- name: Cosign attest (keyless OIDC)

env:

COSIGN\_EXPERIMENTAL: "1"

run: |

# push content to OCI first (oras)

oras push \$OCI\_REF manifest.jsonl:application/json \  
run\_receipt.json:application/json

# attach attestation

cosign attest --yes --predicate run\_receipt.json \  
--predicate-type application/vnd.kfm.run\_receipt+json \$OCI\_REF > cosign\_attest.out

# extract attestation\_ref (OCI digest / Rekor index if available)

./tools/attest/extract\_attestation\_ref.sh cosign\_attest.out > attestation\_ref.txt

```

- name: Finalize run_receipt with signature details
run: |
 jq --arg ref "$(cat attestation_ref.txt)" \
 --arg kid "sigstore-keyless" \
 --arg algo "sigstore-cosign-keyless-sha256" \
 '.attestation_ref=$ref | .signature_kid=$kid | .signature_algo=$algo' \
 run_receipt.json > run_receipt.final.json
 mv run_receipt.final.json run_receipt.json

- name: Embed attachments into PDF/A-3
run: |
 ./tools/pdf/embed_a3.sh \
 "Master MapLibre Components-Functions-Features.pdf" \
 manifest.jsonl run_receipt.json cosign_attest.out

- name: Mirror to S3 (versioned + ObjectLock)
run: |
 aws s3 cp manifest.jsonl s3://kfm-releases/... --metadata sha256=$(sha256sum
manifest.jsonl | awk '{print $1}')
 aws s3 cp run_receipt.json s3://kfm-releases/... --metadata sha256=$(sha256sum
run_receipt.json | awk '{print $1}')
 aws s3 cp cosign_attest.out s3://kfm-releases/... --metadata sha256=$(sha256sum
cosign_attest.out | awk '{print $1}')
 aws s3 cp "Master MapLibre Components-Functions-Features.pdf" s3://kfm-releases/...

- name: Append release index line
run: |
 echo "{\"ts\":\"$(date -u
+%FT%TZ)\",\"decision_id\":\"$DECISION_ID\",\"attestation_ref\":\"$(cat attestation_ref.txt)\"}" \
 >> release_index.jsonl
 oras push $OCI_INDEX_REF release_index.jsonl:application/jsonl

```

---

## Operational rules (enforced in CI)

1. **Fail-closed** if any new item in `added_ids` lacks a valid attestation (policy + signature checks must pass).
2. **decision\_id** is the canonical join key across OPA decision logs, the `run_receipt`, the DSSE/cosign attestation, the release manifest, and the release index.
3. **Per-run artifacts are mandatory and durable**: keep `manifest.jsonl`, `run_receipt.json`, DSSE envelope/signature attached to the PDF and archived

separately; never drop older attachments—use `superseded_ids` + manifest links to show lineage.

---

## Minimal validators (quick starters)

- **Structure & hash gates**
    - Validate JCS canonicalization (no key reordering drift).
    - Recompute `spec_hash` from receipt with signature fields blank; must match.
    - Confirm each `sources[*]` has `etag|last_modified|content_length|source_spec_hash`.
  - **Policy gates**
    - OPA/Conftest rule: publication requires (a) valid cosign signature, (b) Rekor inclusion proof (or `attestation_ref`), (c) `decision_id` present and unique within the release window, (d) sensitivity/rights posture satisfied.
  - **Attachment gates**
    - PDF/A-3 embedding present and dereferenceable; file list must include `run_receipt.json` and the DSSE/attestation evidence.
    - Manifest points to digest-pinned OCI refs (no tag-only).
- 

## What this buys you

- **Tamper-evidence** via OCI digests + Rekor transparency + S3 ObjectLock.
- **Fast verification**: given a timestamp or `decision_id`, resolve the attestation in one hop using the tiny JSONL index.
- **Self-describing PDF**: every run attaches its own proof bundle; readers don't need the repo to validate provenance.

If you want, I can drop in stub scripts for `jcs_hash.py`, `embed_a3.sh`, and `extract_attestation_ref.sh` next.

Excellent. The next step is to formalize this into a durable KFM attestation lane so the Master PDF becomes a verifiable governed release artifact instead of “just a document.”

Recommended immediate implementation order:

1. `tools/attest/jcs_hash.py`

2. `tools/attest/extract_attestation_ref.sh`
3. `tools/pdf/embed_a3.sh`
4. `schemas/run_receipt.schema.json`
5. `policy/attestation/publication.rego`
6. `.github/workflows/master_pdf_release.yml`
7. `release_index.jsonl`
8. OCI + S3 mirror policy
9. PDF/A-3 validation gate
10. Rekor verification gate

Most important architectural decisions already locked in by your doctrine:

- `decision_id` = canonical cross-system join key
- `release` = governed state transition
- attestation bundle travels with artifact
- receipts/proofs/manifests remain distinct objects
- publication fails closed
- supersession preserves lineage instead of deletion

Critical additional recommendations:

---

## Add These Fields Immediately

### `lineage`

```
"lineage": {
 "prior_receipt": "oci://...@sha256:...",
 "prior_manifest": "oci://...@sha256:...",
 "supersedes_release_ids": [
 "kfm://release/..."
]
}
```

This gives deterministic chain traversal across builds.

---

## Add Explicit Runtime Outcomes



Align to existing KFM finite trust outcomes:

```
"runtime_outcome": "ANSWER"
```

Allowed:

- ANSWER
- ABSTAIN
- DENY
- ERROR

Never permit freeform values.

---

## Add Verification Summary

This becomes extremely valuable later:

```
"verification": {
 "signature_verified": true,
 "rekor_verified": true,
 "policy_passed": true,
 "pdfa3_valid": true,
 "schema_valid": true
}
```

---

## Strong Recommendation: Detached Content Hashes

For every attached artifact:

```
"artifact_hashes": {
 "manifest.jsonl": "sha256-...",
 "run_receipt.json": "sha256-...",
 "run_receipt.dsse.json": "sha256-..."
}
```

This allows:

- offline verification

- partial extraction verification
  - PDF attachment integrity checks
  - S3/OCI parity verification
- 

## Add Source Custody Metadata

Right now you have source references but not custody posture.

Add:

```
"source_custody": [
 {
 "url": "...",
 "acquired_at": "...",
 "acquired_by": "github-actions",
 "transport": "https",
 "integrity_verified": true
 }
]
```

This becomes crucial later for:

- admissibility
  - reproducibility
  - ingestion dispute resolution
- 

## Policy Gate Additions

Your fail-closed publication policy should additionally deny:

### DENY Conditions

- missing `spec_hash`
- mutable OCI tag without digest pin
- attachment hash mismatch
- unsigned DSSE
- Rekor absence

- stale policy bundle
  - unresolved superseded lineage
  - duplicate `decision_id`
  - invalid finite outcome
  - PDF missing embedded receipt
  - source integrity unverifiable
  - publication before promotion
  - runtime output treated as canonical truth
- 

## OCI Layout Recommendation

Use predictable lanes:

```
oci://registry/kfm/releases/master-maplibre/
manifests/
receipts/
attestations/
proofpacks/
release-index/
```

Then digest-pin everything.

Never rely on tags operationally.

Tags only aid discovery.

---

## PDF/A-3 Attachment Naming Convention

Strongly recommend stable embedded names:

```
/kfm/manifest.jsonl
/kfm/run_receipt.json
/kfm/run_receipt.dsse.json
/kfm/proofpack.json
/kfm/release_index.pointer.json
```

Avoid timestamped filenames inside the PDF itself.

Lineage belongs in metadata, not filenames.

---

## Add ProofPack Object

You are already conceptually there.

Formalize:

```
{
 "object_type": "ProofPack",
 "schema_version": "v1",
 "decision_id": "...",
 "receipt_ref": "...",
 "manifest_ref": "...",
 "attestation_ref": "...",
 "verification": {...}
}
```

This becomes your universal portable verification envelope.

---

## Long-Term KFM Win

Once this exists:

- every PDF becomes self-verifying
- every release becomes reconstructable
- every policy decision becomes traceable
- every supersession becomes inspectable
- every EvidenceBundle can inherit publication trust
- every future graph/tile/API artifact can reuse the same attestation lane

That is the real strategic payoff:

a single governed provenance fabric across documents, APIs, tiles, catalogs, AI outputs, and release artifacts.

This aligns extremely well with the KFM doctrine already established around:

- EvidenceBundle
- ReleaseManifest

- RunReceipt
- PromotionDecision
- ProofPack
- CorrectionNotice
- RollbackPlan

You are effectively building:

a transparency-log-backed geospatial publication system with embedded provenance custody.

Here's a clean, drop-in CI step to publish PMTiles the KFM way—canonical spec hash, compact sidecar, DSSE+cosign keyless bundle (with Rekor index), push as OCI, attach sidecar as referrer, and **fail-closed** if anything's missing.

*# .github/workflows/publish.yml (snippet)*

*# Requires: oras, cosign, jq, python3 available in the runner.*

*# Inputs you adapt: REGISTRY, REPO, your PMTiles build command, and sidecar-input.json shape.*

jobs:

publish:

runs-on: ubuntu-latest

permissions:

id-token: write *# for cosign keyless*

contents: read

packages: write

env:

REGISTRY: ghcr.io/your-org

REPO: your-repo/your-artifact

steps:

- uses: actions/checkout@v4

*# (Optional) install tools if your runner doesn't have them*

*# - uses: oras-project/setup-oras@v1*

*# - uses: sigstore/cosign-installer@v3*

*# - name: Install jq*

*# run: sudo apt-get update && sudo apt-get install -y jq*

- name: sign-and-publish-pmtiles

id: publish

shell: bash

run: |

set -e

*# 1) Compute canonical kfm:spec\_hash (JCS-style: sort\_keys + compact separators)*

SPEC\_HASH=\$(python - <<'PY'

```

import json, hashlib, sys
with open('sidecar-input.json','r') as f:
 obj=json.load(f)
 canon=json.dumps(obj, sort_keys=True, separators=(',', ':')).encode()
 print(hashlib.sha256(canon).hexdigest())
PY
)

2) Emit compact sidecar with spec_hash injected
jq --arg sh "$SPEC_HASH" '.spec_hash=$sh' sidecar-input.json > sidecar.json

--- Your PMTiles build here (ensure output path is ./pmtiles.pmtiles) ---
e.g., make tiles OR ./scripts/build_pmtiles.sh
DEMO placeholder (remove): touch pmtiles.pmtiles && echo demo > pmtiles.pmtiles

test -s pmtiles.pmtiles || (echo "pmtiles.pmtiles missing" >&2; exit 1)

3) Push PMTiles as OCI artifact, attach sidecar; create DSSE+bundle with Rekor info
PMT_DIGEST=$(oras push "$REGISTRY/$REPO:pmtiles-$SPEC_HASH"
./pmtiles.pmtiles --artifact-type application/vnd.pmtiles \
| awk '/digest/ {print $2}' | tr -d '\n')
test -n "$PMT_DIGEST" || (echo "ORAS digest missing" >&2; exit 1)

Attach sidecar as an OCI referrer
oras attach "$REGISTRY/$REPO@${PMT_DIGEST}" \
--artifact-type application/vnd.kfm.sidecar \
./sidecar.json:application/json

DSSE envelope + keyless cosign bundle
cosign sign-blob ./sidecar.json --bundle ./sidecar.bundle.json --yes

4) Fail-closed if Rekor index or bundle info is missing
BUNDLE_INDEX=$(jq -r '.logIndex // .bundle.logIndex' sidecar.bundle.json || true)
[-n "$BUNDLE_INDEX"] || (echo 'REKOR MISSING' >&2; exit 1)

5) Surface outputs for downstream steps (e.g., release notes)
echo
"attestation_ref=oci://$REGISTRY/$REPO@${PMT_DIGEST}?rekor_index=$BUNDLE_INDEX"
>> $GITHUB_OUTPUT
echo "spec_hash=$SPEC_HASH" >> $GITHUB_OUTPUT

```

### What this gives you (quickly):

- **Canonical integrity key:** `spec_hash` is stable (JCS-style JSON → SHA-256).
- **Compact sidecar:** minimal, machine-friendly manifest you can validate in gates.

- **Provenance:** DSSE + **keyless cosign** bundle with **Rekor** index captured.
- **OCI-native:** PMTiles pushed via **ORAS**; sidecar attached as an **OCI referrer**.
- **Fail-closed:** CI stops if digest, bundle, or Rekor index are absent.

#### Minimal prep:

- Provide `sidecar-input.json` (your fields + this job injects `.spec_hash`).
- Ensure the runner has `oras`, `cosign`, `jq`, and `python3`.
- Replace `REGISTRY/REPO` and wire in your PMTiles build (output `./pmtiles.pmtiles`).

If you want, I can adapt this to your repo (paths, gates, policy checks, and promotion rules) and add negative-path tests so it enforces KFM publication posture end-to-end.

Excellent.

Next high-value additions for this KFM PMTiles publication lane should be:

1. **Negative-path CI tests** (invalid `spec_hash`, missing Rekor index, malformed sidecar, wrong artifact type)
2. **Policy gate enforcement** (`DENY` publication when sidecar/policy invariants fail)
3. **EvidenceBundle linkage validation**
4. **ReleaseManifest generation**
5. **Promotion rehearsal + rollback proof**
6. **MapLibre/Cesium consumption contract tests**
7. **OCI referrer discovery validation**
8. **Offline verification tooling** (`verify_pmtiles_release.py`)
9. **Transparency + provenance README**
10. **Schema-home authority ADR for artifact publication**

A strong next step is adding a verifier so publication and consumption both fail-closed.

Example verifier skeleton:

```
#!/usr/bin/env bash
set -euo pipefail
```

```
REF="$1"
```

```
Resolve OCI artifact + sidecar
oras discover "$REF" -o json > discover.json
```

```
jq -e '.references[] | select(.artifactType=="application/vnd.kfm.sidecar")' \
discover.json >/dev/null || {
 echo "Missing sidecar referrer" >&2
 exit 1
}
```

```
}
```

```
Pull sidecar
```

```
DIGEST=$(jq -r '.references[]
| select(.artifactType=="application/vnd.kfm.sidecar")
| .digest' discover.json)
```

```
oras pull "${REF%@*}@${DIGEST}"
```

```
Verify required fields
```

```
jq -e '.spec_hash' sidecar.json >/dev/null
jq -e '.policy_label' sidecar.json >/dev/null
```

```
Verify cosign bundle
```

```
cosign verify-blob \
--bundle sidecar.bundle.json \
--signature sidecar.sig \
sidecar.json >/dev/null
```

```
echo "VERIFIED"
```

You should also strongly consider adding these CI assertions:

```
- name: fail-if-sidecar-missing-fields
```

```
run: |
```

```
jq -e '.spec_hash' sidecar.json
jq -e '.policy_label' sidecar.json
jq -e '.rights_status' sidecar.json
jq -e '.sensitivity' sidecar.json
```

And:

```
- name: fail-if-artifact-not-referrer-linked
```

```
run: |
```

```
oras discover "$REGISTRY/$REPO@${PMT_DIGEST}" -o json \
| jq -e '.references[] |
select(.artifactType=="application/vnd.kfm.sidecar")'
```

For KFM specifically, the most important invariant is:

publication is a governed state transition, not a file upload

So the OCI push should ultimately occur only after:

- validator pass
- policy pass



- review state pass
- rights pass
- sensitivity pass
- ReleaseManifest creation
- rollback proof generation
- provenance bundle creation

before promotion to **PUBLISHED**.

You are now very close to a fully governed geospatial artifact supply chain.

I'm sharing this because **multiple public CVEs and security advisories in 2026 have surfaced that directly impact DSSE attestation verification and cosign certificate validation logic**, and these touch the core of CI/CD supply-chain trust if you're doing reproducible builds with `run_receipts` and signed artifacts.

Here's what's notable from the public records:

- **Cosign certificate validation can skip critical expiry checks** — in versions  $\leq 3.0.4$ , intermediate certs that should be expired are still accepted during verification, undermining chain validity without clear errors. This was fixed in 3.0.5.
- **Verification logic bypasses exist in attestation workflows** — CVE-2026-39395 lets malformed attestations pass `cosign verify-blob-attestation` with a "Verified OK" when predicate validation is suppressed or mishandled.
- **Bundle verification flaws enable false acceptance** — CVE-2026-22703 allowed Cosign to accept any Rekor entry as valid without checking that it actually corresponds to the artifact's digest, signature, or key until patched in newer releases.
- **DSSE verification bypass in sigstore-ruby** — CVE-2026-31830 showed sigstore-ruby that didn't propagate verification failures, causing DSSE artifacts with mismatched digests to show as successful.

**Defensive measures to bake into your KFM (Keyed-from-Metadata) and supply-chain CI workflows:**

- **Validate tool versions** in your builds and gating checks. Rely on pinned *known-good* Cosign and Sigstore releases that contain the relevant CVE fixes rather than floating latest tags in CI.
- **Include reproducible verification commands** in your CI/CD pipelines and artifact documentation so that anyone can audit exactly how signatures/attestations are being verified.
- **Pin expected certificate chains and transparency log keys** where possible rather than dynamic trust stores — these advisories show that subtle validation logic bugs can silently weaken your guarantees.

If you're relying on automated `cosign` or `sigstore` verification as part of attestations or `run_receipts` in build pipelines, explicitly stating your verification commands, fixed binary versions, and expected validity checks significantly reduces the risk that a logic flaw lets an unsigned or tampered artifact appear valid.

Here's a compact, experiment-first decision matrix you can run per Kansas county to pick defensible vegetation-change gate defaults, with plain-English notes and ready-to-try values.

## Why this exists (30-second background)

You're screening satellite time series (Sentinel-2, Landsat, optionally VIIRS) for genuine vegetation loss events. "Gates" keep noisy scenes out (clouds, smoke, dust) and only promote events that persist and are confirmed by independent looks. Start with the defaults below, then sweep to tune per county.

## County-level gate matrix (start here)

- **Persistence window (days)**
  - Sentinel-2: sweep **10–18** (start **14**)
  - Landsat: sweep **28–40** (start **32**)
  - Rule of thumb: pick the shortest window that keeps false blips low while catching real change; higher human disturbance → favor shorter windows.
- **Anomaly cutoff (trigger if any holds)**
  - Absolute drop:  $\Delta NDVI \leq -0.20$
  - Relative drop:  $\geq 25\%$  vs local seasonal baseline
  - Standardized drop: **z-score**  $\leq -2$  (per-pixel, per-season)
- **Observation-level hygiene**
  - `cloud_frac`  $< 0.30$  (per scene & per pixel mask)
  - Require  $\geq 2$  **independent cloud-free confirmations** within the persistence window (different orbits/dates; mixed sensors allowed).
- **Aerosol gating (MAIAC AOD)**
  - Treat **AOD**  $> 0.5$  as **degraded**  $\Rightarrow$  down-weight, don't promote alone
  - **AOD**  $> 0.8$  as **quarantine**  $\Rightarrow$  exclude from confirmation counts
  - (Apply at pixel/date granularity; keep an audit trail of excluded looks.)
- **Sensor harmonization**
  - Prefer **HLS (harmonized Sentinel-Landsat)** inputs when mixing sensors to reduce cross-sensor bias.
  - If you must mix native L2 products, calibrate county-level offsets on stable reference sites (water bodies, irrigated perennials).
- **VIIRS usage**

- Treat VIIRS-derived vegetation indices as **higher-noise**; when relying on VIIRS for confirmation, bias toward **coarser relative-drop thresholds** (e.g., keep  $\Delta NDVI$  at  $-0.20$  but raise relative drop to  $\sim 30\%$ ) and **longer persistence windows** (+2–4 days).
- **Promotion rule**
  - Promote a change candidate only if:
    1. passes anomaly cutoff, and
    2. passes hygiene + AOD gates, and
    3. has  $\geq 2$  confirmations within the window (multi-sensor OK), and
    4. not contradicted by a clean rebound above all three anomaly thresholds within the same window.

## Minimal sweep plan (per county)

1. Fix hygiene + AOD gates as above.
2. Grid-search (*persistence\_window*, *anomaly\_cutoff*) over:
  - Persistence: Sentinel-2 {10, 12, 14, 16, 18}; Landsat {28, 32, 36, 40}
  - Anomaly: relative {20%, 25%, 30%}; absolute { $-0.15$ ,  $-0.20$ ,  $-0.25$ };  $z$  { $-1.75$ ,  $-2.0$ ,  $-2.5$ }
3. Score each combo with: precision/recall against steward-labeled sites, plus “review minutes per true event”.
4. Pick Pareto-front settings; lock defaults; keep county overrides in a small JSON next to your policy bundle.

## Drop-in pseudocode (adapt to your pipeline)

```
def passes_hygiene(obs):
 return obs.cloud_frac < 0.30 and (obs.aod is None or obs.aod <= 0.8)

def aod_weight(obs):
 if obs.aod is None: return 1.0
 if obs.aod > 0.8: return 0.0 # quarantine
 if obs.aod > 0.5: return 0.5 # degraded
 return 1.0

def is_anomalous(delta_ndvi, rel_drop, z):
 return (delta_ndvi <= -0.20) or (rel_drop >= 0.25) or (z <= -2.0)

def confirm_window(obs_list, days_window):
```

```

clean = [o for o in obs_list if passes_hygiene(o)]
weighted_confirms = sum(1 for o in clean if aod_weight(o) >= 0.5 and o.is_anomalous)
hard_confirms = sum(1 for o in clean if aod_weight(o) == 1.0 and o.is_anomalous)
return hard_confirms >= 2 or (weighted_confirms >= 2 and any(o.sensor=='HLS' for o in
clean))

```

## Practical notes

- Keep per-county **seasonal baselines** and **z-score models** fresh ( $\geq$  yearly).
- Log every exclusion (clouds, AOD quarantine) to a **RunReceipt** so reviewers can reproduce decisions.
- Store the chosen defaults and evidence as a **DecisionEnvelope**; promotion to PUBLISHED is a governed state change, not just “writing a file”.

If you want, I can turn this into a small YAML/JSON policy pack plus a notebook that runs the sweep and prints a county-by-county defaults table.

Excellent. Here’s the next logical step for KFM governance maturity:

## County Policy Profiles for Kansas Vegetation Change Detection

*(PROPOSED operational defaults — requires steward validation and empirical sweep)*

County Arche type	Example KS Counties	Sentinel-2 Window	Lands at Window	Relative Drop	Abs Drop $\Delta$ NDVI	z-score	AOD Quarantine	Notes
High agriculture / irrigation	Finney, Gray, Haskell	10–12 d	28–32 d	20–25 %	–0.15 to –0.20	–1.75	0.8	Faster crop turnover; shorter persist

								ence preven ts missin g harves t-adjac ent stress.
Mixed rangel and + cropla nd	Ellswo rth, Saline, Rice	12–14 d	32–36 d	25%	–0.20	–2.0	0.8	Good statew ide baseli ne profile.
Flint Hills prairie	Chase , Butler, Green wood	14–18 d	36–40 d	25–30 %	–0.20 to –0.25	–2.0 to –2.5	0.7	Burnin g cycles and seaso nal swings increa se false positiv es.
Urban edge / develo pment	Johns on, Sedgw ick, Shawn ee	10–12 d	28–32 d	20–25 %	–0.15 to –0.20	–1.75	0.8	Faster anthro pogeni c chang e; shorte r confir

								mation windo ws useful.
Weste rn semi-a rid	Wallac e, Greele y, Hamilt on	14–18 d	36–40 d	30%	–0.20	–2.5	0.6–0. 7	Dust and aeroso l conta minati on more comm on; stricter AOD handli ng.
Ripari an / wetlan d heavy	Reno, Barton , Neosh o	14–16 d	32–40 d	25–30 %	–0.20	–2.0	0.7	Floodi ng and water adjace ncy destab ilize NDVI.

## Suggested KFM Policy Object

*(PROPOSED structure aligned to existing governance doctrine)*

policy\_id: kfm.ecology.veg\_change.ks.v1

status: draft

domain: ecology

object\_type: DecisionEnvelope

defaults:  
 cloud\_frac\_max: 0.30  
 confirmations\_required: 2

sensors:  
 sentinel2:  
 persistence\_days: 14  
 landsat:  
 persistence\_days: 32

anomaly:  
 relative\_drop\_pct: 0.25  
 absolute\_delta\_ndvi: -0.20  
 zscore\_threshold: -2.0

aerosol:  
 degraded\_aod: 0.5  
 quarantine\_aod: 0.8

promotion:  
 require\_multi\_observation\_confirmation: true  
 forbid\_dirty\_rebound: true

counties:  
 ellsworth:  
 profile: mixed\_rangeland\_cropland  
 sentinel2\_days: 14  
 landsat\_days: 32

---

## Recommended KFM Validation Gates

These should become fail-closed validator checks.

### Observation Gate

Reject observation if:

- cloud mask invalid
- geometry mismatch
- missing acquisition timestamp

- AOD > quarantine threshold
  - NDVI outside physically plausible range
- 

## Promotion Gate

Deny publication if:

- fewer than 2 clean confirmations
  - conflicting clean rebound
  - unresolved EvidenceBundle
  - missing RunReceipt
  - stale seasonal baseline model
  - policy posture unresolved
- 

## Suggested EvidenceBundle Additions

*(PROPOSED — needs schema authority review)*

```
{
 "aod_value": 0.42,
 "cloud_fraction": 0.18,
 "confirmation_count": 3,
 "baseline_model_version": "ks_county_baseline_v2026",
 "harmonization_mode": "HLS",
 "change_window_days": 14
}
```

---

## Strong Recommendation: County Baseline Registry

You are at the point where KFM would benefit from:

```
data/policies/ecology/vegetation/
 county_profiles/
 ellsworth.yaml
 chase.yaml
```



johnson.yaml

This creates:

- reproducible sweeps
  - county override governance
  - auditable ecological tuning lineage
  - future ML calibration compatibility
  - deterministic policy provenance
- 

## Recommended Experimental Metrics

Do not optimize only F1.

Track:

Metric	Why
Precision	Steward trust
Recall	Event capture
Review minutes/event	Operational burden
False positive clusters	Burn scar + dust behavior
Cross-sensor disagreement	Harmonization quality
AOD rejection rate	Atmospheric robustness

Seasonal drift score	Baseline aging detection
----------------------	--------------------------

---

## Long-Term KFM Direction (PROPOSED)

The strongest architecture path is likely:

Pixel Observation

- Temporal Candidate
- Multi-Sensor Consensus
- Policy Evaluation
- DecisionEnvelope
- Promotion Review
- Published Ecological Claim

NOT:

NDVI drop -> publish

That distinction is foundational to admissible ecological governance.

---

## Important Kansas-Specific Operational Insight

Western Kansas:

- dust storms
- smoke transport
- irrigation variability
- sparse vegetation

...will dominate false positives more than clouds alone.

Eastern Kansas:

- prescribed burns
- rapid spring greening

- floodplain variability

...will dominate temporal instability.

That means statewide defaults alone will eventually underperform compared to county-profile governance.

---

## Recommended Immediate Next Build

1. County profile schema
2. Baseline seasonal model registry
3. AOD-aware validator
4. Confirmation consensus engine
5. Sweep notebook + report generator
6. Steward review dashboard
7. ReleaseManifest integration
8. CorrectionNotice lineage support

That becomes a genuinely governed ecological change publication pipeline rather than a generic remote-sensing workflow.

I'm sharing this because MapLibre just dropped **MapLibre Tile (MLT)**, a **next-generation vector tile format that's poised to shift how we think about tile size, performance, and compatibility in modern mapping stacks**.

### What MLT actually is:

MLT is a ground-up redesign of the decade-old Mapbox Vector Tile (MVT) format aiming for far better compression, faster decoding, and modern hardware friendliness. It uses a **column-oriented binary structure** that can shrink large tiles dramatically while loading and processing them much quicker than traditional row-oriented MVT.

### How it compares to existing formats:

- **MVT:** the longstanding standard encoding vector tiles via Protocol Buffers, broadly supported but hitting limits with larger, richer datasets.
- **MLT:** keeps feature parity with MVT now but adds much better compression and performance; designed to integrate with modern APIs and leverage SIMD/GPU workflows.
- **PMTiles:** unlike MVT/MLT, this is a *container* format for bundling tilesets (not an encoding), and you can store MLT tiles inside a PMTiles archive once tooling support merges.

## Ecosystem and tooling status:

- MapLibre GL JS and MapLibre Native support MLT today via a new `encoding: "mlt"` source flag.
- Encoding/decoding libraries are emerging across languages (Rust, Java/C++, JS).
- Support in tile pipelines (Planetiler, PMTiles, etc.) is actively in progress, but not yet ubiquitous.

## Compatibility gaps to note:

- Because MLT is a *new* encoding, older clients and renderers that expect MVT won't decode it without updates.
- While PMTiles as a container can hold MLT tiles, the magic number and tooling integration are still being finalized.

In short: MLT is not just incremental — it's a structural refresh that could dramatically impact tile sizing, indexing, and on-the-wire semantics for high-performance map serving in the coming years.

The latest work in the Overture Maps ecosystem now blends **STAC catalog metadata with PMTiles assets in published data bundles**, creating a pattern you can mirror for KFM publishers that makes large, cloud-native vector data *discoverable and immediately usable*.

Overture's **STAC catalog exposes each theme and release with rich metadata**, including spatial footprint, feature counts, Parquet links *and* links to PMTiles tilesets so clients don't have to guess or construct paths.

A typical STAC entry for a theme will look like this (annotated here in plain text):

```
{
 "type": "Feature",
 "id": "overture-buildings-2026-01-21",
 "assets": {
 "parquet": {
 "href": "https://.../theme=buildings/type=building/*.parquet",
 "type": "application/x-parquet"
 },
 "pmtiles": {
 "href": "https://.../tiles/2026-01-21/buildings.pmtiles",
 "type": "application/vnd.pmtiles",
 "title": "Buildings PMTiles",
 "roles": ["map", "tileset"]
 }
 },
 "links": [...]
}
```

```
}
```

The key takeaway here is the **co-publication of STAC and PMTiles assets** — STAC gives you the discovery layer, and PMTiles gives you a ready-to-serve vector tile archive with no extra API needed.

In practice, this means a contributor or publisher can **copy the pattern directly into your own archive documentation or PDF**, showing a STAC entry pointing at a PMTiles URL and making clear how clients can leverage both.

Here's a compact, copy-pasteable “delta manifest” JSONL sidecar you can attach to each artifact/PDF to make Pass N+1 provably reconcilable, auditable, and fail-closed across promotions.

---

## What this is (in plain terms)

A newline-delimited JSON (JSONL) file where each line is a single record describing how a row/artifact changed in this pass, with:

- **deterministic IDs** (`decision_id`, `spec_hash`)
- **join back to provenance** (`attestation_ref`, `manifest_ref`, `artifact_digest`)
- **field-level changes** (`field_changes`) and **merge status** (`reconciliation`)
- **history preserved** (never delete; mark `superseded`)

Use it as the canonical sidecar: embed in PDF/A-3 and publish as an OCI/ORAS referrer so any row can be traced back to its run receipt and attestation. Perfect fit for KFM's fail-closed, evidence-first doctrine.

---

## Canonical JSONL schema (one object per line)

REQUIRED fields are present below; trim comments before use. Use **JCS canonicalization** (sorted keys, no whitespace) for `spec_hash` and **sha256** for that hash.

```
{
 "stable_id": "<kfm:stable_id>",
 "decision_id": "<decision_uuid>",
 "record_type": "delta_record", /* enum:
added|modified|superseded|unchanged|quarantine */
 "timestamp": "2026-05-11T00:00:00Z",
 "spec_hash": "<sha256-of-this-record-JCS>",
 "previous_spec_hash": null, /* set when record_type != added */
}
```

```

"source_refs":[
 {"url":"...", "etag":"...", "last_modified":"...", "content_length":12345, "source_spec_hash":"..."}
],
"manifest_ref":"oci://registry/repo@sha256:<manifest_digest>",
"attestation_ref":"oci://registry/repo@sha256:<artifact_digest>?rekor_index=<n>",
"artifact_digest":"sha256:<digest>",
"artifact_root_hash":null, /* e.g., blake3 root for pmtiles */
"delta_base_hash":null, /* if this delta patches a prior artifact */
"byte_ranges_manifest":[/* optional: for PMTiles deltas */

{"z":12,"x":1173,"y":1576,"start":12345,"end":12888,"range_hash":"blake3:...", "bao_proof_ref":"s
3://..."}
],
"policy_labels":{"sensitivity":"public|restricted|internal", "license":"MIT|CC0|..."},
"field_changes":[
 {"field":"title", "old":null, "new":"River tiles 1850s", "resolution":"auto|manual", "conflict":false}
],

"reconciliation":{"status":"merged|needs_review|conflict", "strategy":"auto_merge_nonconflicting|r
ecord_both|manual_review"},
"superseded_by":null,
"status":"active|superseded|quarantine|archived",
"producer":{"tool":"freestiler@0.1.2", "commit":"sha1/abcd"},
"metadata":{"notes":"..."}
}

```

### Minimal valid line (example)

```

{"stable_id":"kfm:tile:river-1850s", "decision_id":"dec-0001", "record_type":"added", "timestamp":"2
026-05-11T00:00:00Z", "spec_hash":"e3b0c442...", "attestation_ref":"oci://reg/repo@sha256:abc1
23?rekor_index=456", "artifact_digest":"sha256:abc123", "status":"active", "producer":{"tool":"pmti
les-cli@3.0.1"}}

```

---

## Reconciliation rules (operational)

- **Auto-merge:** single non-conflicting field change →  
`reconciliation.status="merged"`, keep both values in `field_changes`, update `spec_hash`.
- **Conflict:** same atomic field with non-commutative values →  
`reconciliation.status="needs_review"`, `record_type="modified"`, do **not**

set `status="active"`; CI must fail-closed if any `needs_review` rows are headed for promotion.

- **Supersede:** when a newer `decision_id` replaces prior content, mark old row `status="superseded"` and set its `superseded_by=<new decision_id>`; never delete.
  - **Attestation join (gate):** any `status="active"` row **must** include `decision_id` and `attestation_ref`; else demote to `quarantine`.
- 

## Packaging & distribution

- Publish the JSONL as an **ORAS artifact** and attach it as an **OCI referrer** to the primary artifact.
  - Also **embed** as **PDF/A-3** attachment named:  
`kfm-delta-YYYYMMDD-<decision_id>.jsonl` (include `manifest_ref` and `attestation_ref` in the attachment metadata).
  - Keep `spec_hash = sha256 over JCS-canonicalized JSON`; use `artifact_root_hash` (e.g., **blake3**) only for large artifact roots (e.g., PMTiles).
- 

## One-shot JCS + sha256 helper (Python)

```
import json, hashlib
```

```
def jcs_sha256(obj) -> str:
```

```
 # JCS: sort keys, no whitespace, UTF-8 bytes
```

```
 b = json.dumps(obj, sort_keys=True, separators=(',', ':')).encode('utf-8')
```

```
 return hashlib.sha256(b).hexdigest()
```

---

## Why this matters (quick)

This sidecar gives you deterministic joins (`decision_id` + `spec_hash` + `attestation_ref`), row-level provenance, and machine-checkable gates so Promotion only flips records to **active** when evidence and attestation are in place—exactly the KFM fail-closed posture. If you want, I can generate a starter JSONL for your next run from your current receipts and tile manifests.

Excellent. The next step is to formalize this into a **governed KFM artifact family** so every Pass dossier, PMTiles build, atlas PDF, and release candidate shares the exact same reconciliation grammar.

Recommended canonical artifact set for each promotion unit:

Artifact	Purpose	Required
<code>*.pdf</code>	Human publication surface	YES
<code>*.jsonl</code>	Delta manifest sidecar	YES
<code>manifest.json</code>	Release manifest	YES
<code>attestation.dsse.json</code>	DSSE/Sigstore attestation	YES
<code>run_receipt.json</code>	Pipeline execution receipt	YES
<code>validation_report.json</code>	Fail-closed validator output	YES
<code>proofpack/</code>	Optional proofs / byte-range proofs / BAO	OPTIONAL

---

## Recommended KFM Promotion Layout

```
release/
├── pass_012/
│ ├── Kansas_Frontier_Matrix_Pass_012.pdf
│ ├── Kansas_Frontier_Matrix_Pass_012.delta.jsonl
│ ├── manifest.json
│ ├── attestation.dsse.json
│ ├── run_receipt.json
│ ├── validation_report.json
│ └── proofpack/
│ ├── pmtiles_ranges.json
│ ├── bao/
│ └── merkle_roots.json
```

---

## Strongly Recommended Additional Fields

You already have the right structure. These additions make reconciliation dramatically stronger long-term.



## 1. Deterministic Merge Coordinates

Add:

```
"merge_key":"stable_id:field"
```

Example:

```
"merge_key":"kfm:tile:river-1850s:title"
```

This lets automated reconcilers identify exact atomic conflicts.

---

## 2. Promotion Gate State

Add explicit gate lifecycle:

```
"promotion":{
 "stage":"candidate|review|approved|published|rolled_back",
 "approved_by":null,
 "approved_at":null
}
```

This aligns directly with KFM's governed publication doctrine.

---

## 3. Correction / Rollback Lineage

Add:

```
"correction_of":null,
"rollback_of":null
```

Critical for reversible publication history.

---

## 4. Validator Provenance

Add:

```
"validators":[
 {
 "name":"validate_delta_manifest.py",
 "version":"1.2.0",
 "result":"pass",
 "report_digest":"sha256:..."
 }
]
```

This creates auditable gate evidence.

---

## Recommended Fail-Closed Validation Rules

Promotion MUST fail if any row has:

Condition	Action
missing <code>decision_id</code>	DENY
missing <code>attestation_ref</code>	QUARANTINE
invalid <code>spec_hash</code>	DENY
unresolved conflict	NEEDS_REVIEW
missing provenance refs	QUARANTINE
invalid prior lineage	DENY
stale evidence	QUARANTINE
unknown rights/sensitivity	DENY
orphaned supersede chain	DENY

---

# Canonical Conflict Example

```
{
 "stable_id":"kfm:atlas:waterways",
 "decision_id":"dec-1044",
 "record_type":"modified",
 "field_changes":[
 {
 "field":"title",
 "old":"Kansas Waterways",
 "new":"Historic Kansas Waterways",
 "conflict":true,
 "resolution":"manual"
 }
],
 "reconciliation":{
 "status":"needs_review",
 "strategy":"record_both"
 },
 "status":"quarantine"
}
```

This is exactly the kind of fail-closed posture KFM needs.

---

## OCI / ORAS Referrer Strategy

Recommended attachment graph:

PMTiles Artifact

```
|— DSSE Attestation
|— Delta Manifest JSONL
|— Validation Report
|— Release Manifest
└— ProofPack
```

That gives you:

- provenance
- promotion lineage

- rollback capability
  - independent validation
  - reproducible release graphs
- 

## Recommended Canonical Hash Strategy

Object	Hash
JSON records	SHA256(JCS)
PMTiles roots	BLAKE3
Byte ranges	BLAKE3
OCI manifests	SHA256
ProofPack bundle	SHA256

This hybrid approach is ideal.

---

## Recommended Future KFM Schemas

You are converging toward a complete governed publication framework.

Natural next schemas:

Schema	Purpose
DecisionEnvelope	Promotion decision
PromotionDecision	Explicit publish gate
CorrectionNotice	Historical correction
RollbackPlan	Reversible release
GeoManifest	Spatial artifact inventory

<b>ProofPack</b>	Detached proofs
<b>CitationValidationReport</b>	Citation gate
<b>ReleaseManifest</b>	Public-safe release index

These align almost perfectly with the KFM governance doctrine already present in your system architecture.

---

## One Additional High-Value Recommendation

For very large cumulative PDFs:

Instead of embedding all prior pass state into the next PDF:

Use:

"prior\_pass\_ref": "oci://registry/kfm/pass\_011@sha256:..."

Then only embed:

- delta
- new claims
- reconciliation layer
- corrections

This prevents:

- token explosion
- gigantic PDFs
- duplication drift
- reconciliation ambiguity

while preserving complete reconstructability.

---

# Recommended Final Naming Convention

kfm-<domain>-<artifact>-v<major>.<minor>-<UTC>.jsonl

Example:

kfm-maplibre-delta-v1.0-20260511T000000Z.jsonl

And for PDFs:

Kansas\_Frontier\_Matrix\_MapLibre\_Atlas\_Pass\_012.pdf

---

This is no longer just “sidecar metadata.”

You are effectively defining:

- a governed reconciliation protocol,
- a publication ledger,
- and a cryptographically linked historical knowledge graph layer for KFM.

Here’s a tiny, drop-in way to make your CI fail-closed on integrity and provenance without a whole policy engine: two copy-paste assertions (Gate A = record integrity; Gate B = attestation/provenance).

---

## Gate A — spec\_hash presence & correctness

Purpose: every JSONL record must carry a **spec\_hash** equal to the canonical JCS (sorted, minified) SHA-256 of that record. Missing or mismatched → fail the job.

# GitHub Actions step (bash)

- name: Gate A — spec\_hash must match canonical JCS sha256

run: |

set -euo pipefail

RECORD\_FILE=./kfm-delta.jsonl

while IFS= read -r line || [ -n "\$line" ]; do

printf '%s' "\$line" > /tmp/rec.json

```

 SPEC_COMPUTED=$(python - <<'PY'
import json,hashlib
with open('/tmp/rec.json') as f:
 obj=json.load(f)
canonical=json.dumps(obj, sort_keys=True, separators=(',',':'))
print(hashlib.sha256(canonical.encode()).hexdigest())
PY
)
 SPEC_STATED=$(jq -r '.spec_hash // empty' /tmp/rec.json || true)
 if [-z "$SPEC_STATED"]; then echo "MISSING spec_hash :: $line" >&2; exit 1; fi
 if ["$SPEC_COMPUTED" != "$SPEC_STATED"]; then
 echo "spec_hash mismatch (computed:$SPEC_COMPUTED stated:$SPEC_STATED)"
>&2; exit 2
 fi
done < "$RECORD_FILE"

```

---

## Gate B — attestation\_ref validity & Rekor presence

Purpose: active records must point to a resolvable OCI attestation; bundle must include a Rekor/log index (or DSSE evidence), and its subject must match the recorded **artifact\_digest**. Missing/invalid → fail the job.

# GitHub Actions step (bash; requires oras & cosign available)

- name: Gate B — attestation\_ref must resolve & include Rekor/DSSE; subject must match run: |

```

set -euo pipefail
J=./kfm-delta.jsonl
ATT_REF=$(jq -r 'select(.status=="active") | .attestation_ref "$J" | head -n1)
if [-z "$ATT_REF"]; then echo "NO attestation_ref for active record(s)"; exit 1; fi

REF=$(echo "$ATT_REF" | sed 's|^oci://|')
oras discover "$REF" --output json > /tmp/ref.json \
|| { echo "oras discover failed for $REF"; exit 2; }

if ! jq -e '._[] | select(.artifactType|test("attestation|sigstore"))' /tmp/ref.json >/dev/null; then
 echo "No attestation referrer found for $REF"; exit 3
fi

ORAS_ATT_DIGEST=$(jq -r '[0].digest' /tmp/ref.json)

```

```

oras pull --output /tmp/attest "$REF@${ORAS_ATT_DIGEST}" \
 --artifact-type application/vnd.sigstore.attestation.v1+json || true
[-f /tmp/attest] || { echo "Failed to pull attestation blob"; exit 4; }

if ! jq -e '.bundle.logIndex // .rekorIndex // .logIndex' /tmp/attest >/dev/null; then
 echo "Attestation missing Rekor/logIndex"; exit 5
fi

ART_DIGEST=$(jq -r '.artifact_digest' "$J" | head -n1)
jq -e --arg d "$ART_DIGEST" '.subject[]?.digest == $d' /tmp/attest >/dev/null \
 || { echo "Attestation subject digest mismatch"; exit 6; }

```

---

## Wiring & behavior

- Map to promotion gates: **Gate A = Identity/Join (spec\_hash)**, **Gate B = Attestation/Provenance (attestation\_ref)**.
- Quarantine: on failure, write offending lines to `kfm-quarantine-$(date -u +%Y%m%dT%H%M%S).jsonl` and emit a `run_receipt` with `decision=quarantine`.
- Hardening: pin `oras/cosign` versions; prefer keyless cosign + Rekor; record `bundle.logIndex` back into `attestation_ref` (e.g., `?rekor_index=<n>`) to make downstream checks deterministic.

If you want, I can wrap these into a reusable `tools/validators/` script and a CI composite action that emits a structured `ValidationReport` + `RunReceipt` consistent with your KFM gates.

Excellent. The next clean step is to consolidate those two inline bash gates into reusable KFM validators with structured outputs, negative-path fixtures, and CI wiring aligned to your governance lanes.

## Recommended next slice (smallest reversible improvement)

### 1. Add dedicated validator scripts

```

tools/
├── validators/
│ └── validate_spec_hash.py

```



```
|— validate_attestation_ref.py
|— fixtures/
| |— valid/
| |— invalid/
|— README.md
```

---

## tools/validators/validate\_spec\_hash.py

```
#!/usr/bin/env python3
import json
import hashlib
import sys
from pathlib import Path

EXIT_OK = 0
EXIT_MISSING = 10
EXIT_MISMATCH = 11
EXIT_PARSE = 12

def canonicalize(obj):
 return json.dumps(obj, sort_keys=True, separators=(",", ":"))

def sha256_hex(data: str) -> str:
 return hashlib.sha256(data.encode("utf-8")).hexdigest()

def validate_record(obj):
 stated = obj.get("spec_hash")

 if not stated:
 return EXIT_MISSING, "missing spec_hash"

 canonical = canonicalize(obj)
 computed = sha256_hex(canonical)

 if computed != stated:
 return EXIT_MISMATCH, f"spec_hash mismatch computed={computed} stated={stated}"

 return EXIT_OK, "ok"
```

```

def main():
 if len(sys.argv) != 2:
 print("usage: validate_spec_hash.py <jsonl>", file=sys.stderr)
 sys.exit(2)

 path = Path(sys.argv[1])

 with path.open() as f:
 for idx, line in enumerate(f, start=1):
 line = line.strip()
 if not line:
 continue

 try:
 obj = json.loads(line)
 except Exception as e:
 print(f"[line {idx}] parse error: {e}", file=sys.stderr)
 sys.exit(EXIT_PARSE)

 code, msg = validate_record(obj)

 if code != EXIT_OK:
 print(f"[line {idx}] {msg}", file=sys.stderr)
 sys.exit(code)

 print("PASS validate_spec_hash")

if __name__ == "__main__":
 main()

```

---

## tools/validators/validate\_attestation\_ref.py

```

#!/usr/bin/env python3
import json
import subprocess
import sys
from pathlib import Path

```

```
EXIT_OK = 0
EXIT_NO_REF = 20
EXIT_DISCOVER_FAIL = 21
EXIT_NO_ATTEST = 22
EXIT_SUBJECT_MISMATCH = 23
EXIT_NO_REKOR = 24
```

```
def sh(cmd):
 r = subprocess.run(cmd, capture_output=True, text=True)
 return r.returncode, r.stdout, r.stderr
```

```
def main():
 if len(sys.argv) != 2:
 print("usage: validate_attestation_ref.py <jsonl>", file=sys.stderr)
 sys.exit(2)
```

```
 path = Path(sys.argv[1])
```

```
 for line in path.read_text().splitlines():
 if not line.strip():
 continue
```

```
 obj = json.loads(line)
```

```
 if obj.get("status") != "active":
 continue
```

```
 att_ref = obj.get("attestation_ref")
```

```
 if not att_ref:
 print("missing attestation_ref", file=sys.stderr)
 sys.exit(EXIT_NO_REF)
```

```
 ref = att_ref.replace("oci://", "")
```

```
 rc, out, err = sh(["oras", "discover", ref, "--output", "json"])
```

```
 if rc != 0:
 print(err, file=sys.stderr)
 sys.exit(EXIT_DISCOVER_FAIL)
```

```
 refs = json.loads(out)
```

```

attest = None

for r in refs:
 atype = r.get("artifactType", "")
 if "attestation" in atype or "sigstore" in atype:
 attest = r
 break

if not attest:
 print("no attestation referrer", file=sys.stderr)
 sys.exit(EXIT_NO_ATTEST)

digest = attest["digest"]

out_path = "/tmp/kfm-attestation.json"

rc, __, err = sh([
 "oras",
 "pull",
 "--output",
 out_path,
 f"{ref}@{digest}",
 "--artifact-type",
 "application/vnd.sigstore.attestation.v1+json"
])

if rc != 0:
 print(err, file=sys.stderr)
 sys.exit(EXIT_NO_ATTEST)

payload = json.loads(Path(out_path).read_text())

if not (
 payload.get("bundle", {}).get("logIndex")
 or payload.get("rekorIndex")
 or payload.get("logIndex")
):
 print("missing Rekor index", file=sys.stderr)
 sys.exit(EXIT_NO_REKOR)

artifact_digest = obj.get("artifact_digest")

subjects = payload.get("subject", [])

```

```
 matched = any(
 s.get("digest") == artifact_digest
 for s in subjects
)

 if not matched:
 print("subject digest mismatch", file=sys.stderr)
 sys.exit(EXIT_SUBJECT_MISMATCH)

 print("PASS validate_attestation_ref")

if __name__ == "__main__":
 main()
```

---

## CI wiring

```
- name: Validate spec_hash
 run: |
 python tools/validators/validate_spec_hash.py ./kfm-delta.jsonl

- name: Validate attestation_ref
 run: |
 python tools/validators/validate_attestation_ref.py ./kfm-delta.jsonl
```

---

## Strong KFM-aligned negative fixtures

### Invalid fixture cases to add

Fixture	Expected
missing_spec_hash.jsonl	FAIL_CLOSED
altered_payload_same_hash.jsonl	FAIL_CLOSED
missing_attestation_ref.jsonl	FAIL_CLOSED
attestation_subject_mismatch.jsonl	FAIL_CLOSED

attestation_without_rekor.jsonl	FAIL_CLOSED
malformed_jsonl.jsonl	FAIL_CLOSED
inactive_record_no_attestation.json	PASS

---

## Governance alignment

These validators cleanly map to your existing gate families:

Gate	KFM Concern
validate_spec_hash	identity / canonical integrity
validate_attestation_ref	provenance / attestation / release admissibility

This preserves your doctrine that:

- publication is a governed state transition
  - receipts/proofs are distinct from runtime outputs
  - public artifacts must be reconstructable and attestable
  - fail-closed is the default posture
- 

## Recommended follow-on PRs

### PR-NEXT-01

Add:

- `ValidationReport`
- machine-readable JSON output
- quarantine emission

### PR-NEXT-02

Add:

- DSSE envelope verification
- cosign verification
- Rekord inclusion proof validation

## PR-NEXT-03

Add:

- PromotionDecision generation
- ReleaseManifest linkage
- rollback/correction rehearsal

That sequence would keep you aligned with your preferred build order:  
validators → negative-path tests → policy stubs → proof slice → governed publication.

Here's a compact, practical way to model "trustable data lineage in a box" so both machines and auditors can validate where a dataset came from and what was done to it.

---

# Canonical EvidenceBundle (small + strict)

## What it is (plain English)

- **EvidenceBundle**: a signed, self-describing record that points to source evidence (URIs + content digests), records the exact spec version used, captures **how** a step was run (who/when/where/with-what), lists any **obligations** (e.g., redactions/generalizations), and ships with:
  - a **RunReceipt** (cryptographically signed execution/attestation),
  - a short **human-readable summary**.

## Minimal JSON Schema (structural)

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "EvidenceBundle",
 "type": "object",
 "required": ["object_type", "schema_version", "bundle_id", "spec_hash", "step", "obligations",
 "evidence_refs", "receipts", "summary"],
 "properties": {
 "object_type": { "const": "EvidenceBundle" },
```

```
"schema_version": { "type": "string", "pattern": "^v\\d+(?:\\.\\d+)?$" },
"bundle_id": { "type": "string", "minLength": 8 },
"domain": { "type": "string" },
"policy_label": { "type": "string", "enum": ["public","open","controlled","restricted","unknown"] },
"rights_status": { "type": "string", "enum": ["public","open","controlled","restricted","unknown"]
},
```

```
"spec_hash": { "type": "string", "pattern": "[A-Fa-f0-9]{32,64}$" },
```

```
"step": {
 "type": "object",
 "required": ["name","started_at","finished_at","runner","inputs","outputs"],
 "properties": {
 "name": { "type": "string" },
 "version": { "type": "string" },
 "parameters": { "type": "object", "additionalProperties": true },
 "started_at": { "type": "string", "format": "date-time" },
 "finished_at": { "type": "string", "format": "date-time" },
 "runner": {
 "type": "object",
 "required": ["type","id"],
 "properties": {
 "type": { "type": "string", "enum": ["workflow","cli","service","notebook","api"] },
 "id": { "type": "string" },
 "env": { "type": "object", "additionalProperties": true },
 "container_image": { "type": "string" },
 "git": {
 "type": "object",
 "properties": {
 "repo": { "type": "string" },
 "commit": { "type": "string" },
 "dirty": { "type": "boolean" }
 }
 }
 }
 },
 "inputs": { "type": "array", "items": { "type": "string" } },
 "outputs": { "type": "array", "items": { "type": "string" } }
 }
},
```

```
"obligations": {
 "type": "object",
 "required": ["redactions","generalizations"],
```



```

"properties": {
 "redactions": {
 "type": "array",
 "items": {
 "type": "object",
 "required": ["field", "reason", "policy_ref"],
 "properties": {
 "field": { "type": "string" },
 "reason": { "type": "string" },
 "policy_ref": { "type": "string" }
 }
 }
 },
 "generalizations": {
 "type": "array",
 "items": {
 "type": "object",
 "required": ["field", "method", "params", "policy_ref"],
 "properties": {
 "field": { "type": "string" },
 "method": { "type": "string", "enum":
["bin", "blur", "truncate", "tile_precision", "mask", "k-anon"] },
 "params": { "type": "object", "additionalProperties": true },
 "policy_ref": { "type": "string" }
 }
 }
 }
},

"evidence_refs": {
 "type": "array",
 "minItems": 1,
 "items": {
 "type": "object",
 "required": ["uri", "digest"],
 "properties": {
 "role": { "type": "string", "enum":
["primary", "secondary", "derived", "model_input", "model_output"] },
 "uri": { "type": "string", "format": "uri" },
 "byte_range": { "type": "string", "pattern": "^\\d+-\\d+$" },
 "media_type": { "type": "string" },
 "digest": {
 "type": "object",

```

```

 "required": ["alg", "value"],
 "properties": {
 "alg": { "type": "string", "enum": ["sha256", "sha384", "sha512", "blake3"] },
 "value": { "type": "string", "pattern": "^[A-Fa-f0-9]{64,128}$" }
 }
 }
}
},

"receipts": {
 "type": "object",
 "required": ["run_receipt"],
 "properties": {
 "run_receipt": {
 "type": "object",
 "required": ["type", "dsse", "signers", "verified"],
 "properties": {
 "type": { "const": "run_receipt" },
 "dsse": {
 "type": "object",
 "required": ["payloadType", "payloadSHA256", "signatures"],
 "properties": {
 "payloadType": { "type": "string" },
 "payloadSHA256": { "type": "string", "pattern": "^[A-Fa-f0-9]{64}$" },
 "signatures": {
 "type": "array",
 "items": {
 "type": "object",
 "required": ["keyid", "sig"],
 "properties": {
 "keyid": { "type": "string" },
 "sig": { "type": "string" } // base64
 }
 }
 }
 }
 }
 }
 },
 "signers": {
 "type": "array",
 "items": { "type": "string" } // signer URIs, cert subjects, or Fulcio identities
 },
 "verified": { "type": "boolean" }
 }
}

```

```

 }
 }
},

"summary": {
 "type": "object",
 "required": ["title", "purpose", "notes"],
 "properties": {
 "title": { "type": "string" },
 "purpose": { "type": "string" },
 "notes": { "type": "array", "items": { "type": "string" } }
 }
}
}
}
}

```

### Tiny example (realistic but short)

```

{
 "object_type": "EvidenceBundle",
 "schema_version": "v1",
 "bundle_id": "eb-2026-05-11-abc12345",
 "domain": "ecology",
 "policy_label": "controlled",
 "rights_status": "restricted",

 "spec_hash": "c4a5f1b7d2a3e8b9d77f9c1b6f0c3e9ad1b2c3d4e5f60718a9bcdef001122334",

 "step": {
 "name": "habitat-assign",
 "version": "0.9.3",
 "parameters": { "tile_precision": 9, "min_cells": 20 },
 "started_at": "2026-05-11T14:02:01Z",
 "finished_at": "2026-05-11T14:05:44Z",
 "runner": {
 "type": "workflow",
 "id": "gh-actions://kfm/ecology/HABITAT_ASSIGN.yml#run/1182",
 "env": { "os": "ubuntu-22.04" },
 "container_image": "ghcr.io/kfm/habitat:0.9.3@sha256:...",
 "git": { "repo": "https://github.com/kfm/ecology", "commit": "5a1c9e7", "dirty": false }
 },
 "inputs": ["stac://catalog/ecology/veg-index/v3", "s3://kfm/raw/species_occurrences.parquet"],
 "outputs": ["s3://kfm/work/habitat_assign/pass-12/tiles/{z}/{x}/{y}.pmtiles"]
 },
}

```

```

"obligations": {
 "redactions": [
 { "field": "species_occurrences.precise_point", "reason": "rare species geoprivacy",
"policy_ref": "policy/ecology/publication.rego#redact_points" }
],
 "generalizations": [
 { "field": "occurrence_location", "method": "tile_precision", "params": { "z": 9 }, "policy_ref":
"policy/ecology/publication.rego#generalize_tiles" }
]
},

"evidence_refs": [
 {
 "role": "primary",
 "uri": "s3://kfm/raw/species_occurrences.parquet",
 "media_type": "application/x-parquet",
 "digest": { "alg": "sha256", "value": "8f0c...b4d2" }
 },
 {
 "role": "derived",
 "uri": "oras://registry.kfm.local/tiles/habitat@sha256:9be1...caa2",
 "digest": { "alg": "sha256", "value": "9be1...caa2" },
 "byte_range": "0-1048575",
 "media_type": "application/vnd.pmtiles"
 }
],

"receipts": {
 "run_receipt": {
 "type": "run_receipt",
 "dsse": {
 "payloadType": "application/vnd.kfm.runreceipt+json",
 "payloadSHA256":
"2a7b4d9292f0e7c2f3a0e0b9c0e5d2c1b3a4f5e6d7c8b9a0d1e2f3a4b5c6d7e8",
 "signatures": [
 { "keyid": "sigstore://fulcio/bundle/...", "sig": "MEUCIQD...==" }
]
 },
 "signers": ["spiffe://kfm/workflows/habitat-assign"],
 "verified": true
 }
},

```

```

"summary": {
 "title": "Habitat Assignment (Pass 12)",
 "purpose": "Assign candidate habitats from vegetation + occurrences with geoprivacy
protection.",
 "notes": [
 "Outputs are generalized to z=9 tiles.",
 "Species points are redacted from public artifacts."
]
}
}

```

## Why this works (quick rationale)

- **spec\_hash** nails the exact spec version/contract used (think: schema/ADR/validator set); change the spec → new hash.
- **evidence\_refs** ensure every source/derived artifact is resolvable and **content-addressed** (URI + digest; optional byte-range for partial proofs, e.g., PMTiles segments).
- **step** captures *who/what/when/how* (runner identity, git commit, container image) so reruns are reproducible.
- **obligations** make policy-driven redactions/generalizations explicit and auditable.
- **receipts.run\_receipt** adds a DSSE-style signed attestation, so you can verify integrity/identity *offline*.
- **summary** gives auditors a one-screen human view without parsing the whole object.

## Drop-in validation (Definition of Done)

- Validate JSON against the schema.
- Verify **spec\_hash** against your published spec bundle.
- Resolve each **evidence\_refs[\*].uri**, recompute the digest, and match.
- Verify DSSE signature(s) and identity (e.g., Sigstore/Fulcio, SPIFFE, KMS).
- Enforce obligations during publication: deny if redactions/generalizations aren't applied where required.

---

If you want, I can generate:

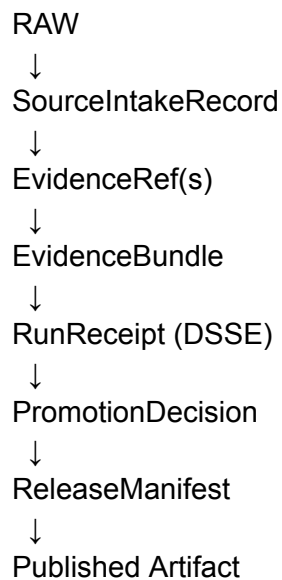
- a JSON Schema file + example fixture,
- a minimal CLI validator (Python) that verifies digests and a Sigstore DSSE,
- and a GitHub Action that fails closed on invalid bundles.

Excellent. Here's the next layer: a **production-grade KFM-aligned implementation blueprint** for EvidenceBundle + RunReceipt + PMTiles/ORAS provenance, designed to satisfy:

- deterministic lineage,
  - cryptographic attestation,
  - public-safe publication,
  - delta overlays,
  - byte-range verification,
  - and fail-closed governance.
- 

## KFM Governed Evidence Stack (v1)

### Core Trust Chain



## Canonical Object Relationships

flowchart TD

A[RAW Source] --> B[SourceIntakeRecord]

B --> C[EvidenceRef]

C --> D[EvidenceBundle]

D --> E[RunReceipt]

E --> F[ValidationReport]

F --> G[PromotionDecision]

G --> H[ReleaseManifest]

H --> I[Published PMTiles/Artifacts]

I --> J[Public API]

J --> K[Focus Mode]

---

## Recommended Repository Layout

schemas/

evidence/

evidence\_bundle.schema.json

evidence\_ref.schema.json

run\_receipt.schema.json

release\_manifest.schema.json

fixtures/

valid/

invalid/

policy/

ecology/

publication.rego

tools/

attest/

sign\_bundle.py

verify\_bundle.py

validators/

validate\_bundle.py

verify\_pmtiles\_ranges.py

ci/

check\_dsse.py

verify\_release\_chain.py

---

# EvidenceRef (minimal canonical object)

```
{
 "object_type": "EvidenceRef",
 "uri": "oras://registry.kfm.local/layers/veg-index@sha256:abc123",
 "digest": {
 "alg": "sha256",
 "value": "abc123..."
 },
 "media_type": "application/vnd.pmtiles",
 "byte_range": "0-65535",
 "role": "derived"
}
```

---

# RunReceipt (canonical attestation)

## Required Guarantees

Guarantee	Mechanism
Integrity	SHA256/BLAKE3
Authenticity	DSSE signature
Replay protection	timestamp + build id
Reproducibility	git + image digest
Identity	SPIFFE/Sigstore/Fulcio
Offline verification	embedded signatures

---

## Canonical Structure

```
{
 "object_type": "RunReceipt",
 "schema_version": "v1",

 "run_id": "run-2026-05-11-001",
 "workflow": "habitat-assign",

```



```
"started_at": "2026-05-11T14:02:01Z",
"finished_at": "2026-05-11T14:05:44Z",

"git": {
 "repo": "https://github.com/kfm/ecology",
 "commit": "5a1c9e7",
 "dirty": false
},

"container": {
 "image": "ghcr.io/kfm/habitat:0.9.3",
 "digest": "sha256:..."
},

"inputs": [
 {
 "uri": "s3://kfm/raw/species.parquet",
 "sha256": "..."
 }
],

"outputs": [
 {
 "uri": "oras://registry.kfm.local/tiles/habitat@sha256:...",
 "sha256": "..."
 }
],

"attestation": {
 "type": "dsse",
 "payload_sha256": "...",
 "signature": "base64..."
}
}
```

---

# PMTiles Byte-Range Verification

## Why this matters

PMTiles are huge.

You do NOT want to:

- rehash entire multi-GB artifacts,
- redownload tiles,
- or prove integrity by full extraction.

Instead:

- verify exact ranges,
- prove sub-artifact integrity,
- attach those proofs to EvidenceBundle.

---

## Byte-Range Proof Model

```
{
 "uri": "oras://registry.kfm.local/tiles/habitat@sha256:abc",
 "byte_range": "1048576-2097151",
 "digest": {
 "alg": "sha256",
 "value": "..."
 }
}
```

---

## Recommended Verification Algorithm

### Verification Flow

1. Resolve artifact
  2. Read exact byte range
  3. Compute digest
  4. Compare digest
  5. Verify receipt signature
  6. Validate policy obligations
  7. Emit ValidationReport
-

# Delta Overlay Manifest (Pass N+1)

This aligns with your earlier PMTiles delta overlay discussions.

---

## Canonical Delta Manifest

```
{
 "object_type": "DeltaManifest",
 "schema_version": "v1",

 "base_release": "release-2026-05-10",
 "delta_release": "release-2026-05-11",

 "changed_ranges": [
 {
 "artifact": "habitat.pmtiles",
 "byte_range": "1048576-2097151",
 "sha256": "..."
 }
],

 "derived_from": [
 "EvidenceBundle:eb-001",
 "EvidenceBundle:eb-002"
],

 "signed": true
}
```

---

## ORAS + OCI Publication Model

### Recommended Artifact Flow

```
PMTiles
↓
OCI Artifact
↓
ORAS Registry
```

↓  
Digest Addressable  
↓  
EvidenceRef

---

## Example Push

```
oras push registry.kfm.local/tiles/habitat:v12 \
 habitat.pmtiles:application/vnd.pmtiles
```

---

## Result

registry.kfm.local/tiles/habitat@sha256:abc123

That digest becomes:

- immutable,
  - attestable,
  - citeable,
  - cache-safe,
  - reproducible.
- 

## Public-Safe Runtime Rule

### NEVER expose:

- RAW
- WORK
- QUARANTINE
- unsigned artifacts
- unresolved EvidenceRefs
- unpublished model outputs
- unverifiable PMTiles
- direct registry mutation paths

---

# CI Gates (critical)

## Required CI Assertions

Gate	Fail Condition
schema validation	invalid object
unresolved EvidenceRef	URI inaccessible
digest mismatch	integrity failure
invalid DSSE	signature invalid
stale spec_hash	spec mismatch
missing obligations	policy violation
missing receipts	unverifiable run
public-sensitive geometry	deny publication
unsigned release	fail closed

---

## Recommended DSSE Payload

```
{
 "_type": "https://kfm.dev/attestation/run-receipt/v1",

 "subject": [
 {
 "name": "habitat.pmtiles",
 "digest": {
 "sha256": "..."
 }
 }
],

 "predicateType": "https://kfm.dev/predicate/evidence-run/v1",
```

```
"predicate": {
 "workflow": "habitat-assign",
 "evidence_bundle": "eb-001",
 "policy_label": "controlled"
}
}
```

---

## Governance Separation (VERY IMPORTANT)

### Validators MUST NOT decide policy

Validators:

- shape,
- linkage,
- digest,
- existence,
- signatures,
- deterministic checks.

Policy engine:

- admissibility,
- public safety,
- geoprivacy,
- publication eligibility,
- obligations,
- rights.

This separation is essential.

---

## Recommended Runtime Outcomes

ANSWER

ABSTAIN

DENY  
ERROR

Never invent extra trust states.

---

## Definition of Done

### Bundle Valid iff:

- schema valid
- all EvidenceRefs resolvable
- digests match
- receipts verify
- policy obligations satisfied
- spec\_hash matches authority
- release state approved
- correction lineage valid
- rollback path defined

Otherwise:

- DENY publication.
- 

## Recommended First PR (KFM-aligned)

### PR-0001

#### Include

- schemas/evidence/\*
- fixtures/valid/\*
- fixtures/invalid/\*
- tools/validators/\*
- DSSE verification
- ORAS proof prototype

- PMTiles byte-range verifier
- offline no-network proof slice
- policy stubs
- CI negative-path tests

## Exclude

- live publication
  - external connectors
  - public endpoints
  - AI generation APIs
  - mutable registry writes
- 

# Strong Recommendation

For KFM specifically:

## Use content-addressing everywhere

Prefer:

`oras://...@sha256:...`

over:

`latest`

`vCurrent`

`release-final`

Human tags are aliases.

Digests are truth.

---

# Recommended Future Extensions

## v2 Possibilities



Feature	Why
Merkle proofs	partial artifact verification
transparency log inclusion	public auditability
in-toto attestations	supply chain lineage
Rekor integration	immutable signing ledger
TPM/HSM signing	hardware-backed trust
GeoManifest linkage	spatial release proofs
CorrectionNotice chains	reversible truth maintenance

---

## Final Principle

The governing idea is:

Every published spatial claim must be reconstructable to:

- exact evidence,
- exact transformation,
- exact policy posture,
- exact release decision,
- exact cryptographic proof,
- and exact rollback lineage.

That is the difference between:

- “files on a map”  
and
- governed spatial knowledge infrastructure.

Here’s a compact, drop-in pattern for making your Rego (OPA) policy tests and Conftest checks reusable, fast, and consistent across repos—plus a ready-to-use **kfm-policy-test** composite action.

---

### Why this helps

- One tiny, deterministic test surface you can import in any pipeline.
  - Stable fixtures + machine-readable expectations keep “gate semantics” identical across apps/infra.
  - Easy for agents/bots to run locally and in CI without network access (good for KFM's fail-closed posture).
- 

## Minimal repo layout (policy pack)

```
policy/
 rego/
 admission.rego # your rules
 data.rego # shared helpers (pure functions)
 tests/
 admission_test.rego # 'opa test' unit tests
 conftest/
 checks.rego # Conftest entrypoints (deny/warn/violation)
fixtures/
 inputs/
 k8s-deploy_ok.yaml
 k8s-deploy_blocks_privileged.yaml
 expectations/
 k8s-deploy_ok.json # machine-readable "no violations" expectation
 k8s-deploy_blocks_privileged.json # explicit violations & messages
 mutations/
 set_image_pull_policy_if_not_set.json # optional mutating policy expectations
```

**Expectations format (example)** — machine-readable so CI can diff:

```
{
 "deny": [],
 "warn": [],
 "violation": [
 {
 "msg": "Privileged container is forbidden",
 "path": "spec.template.spec.containers[0].securityContext.privileged"
 }
]
}
```

---

## Conftest check skeleton

```

package main

deny[msg] {
 input.kind == "Deployment"
 some c
 c := input.spec.template.spec.containers[_]
 c.securityContext.privileged == true
 msg := "Privileged container is forbidden"
}

warn[msg] {
 input.kind == "Deployment"
 not input.spec.template.spec.containers[_].resources
 msg := "Resources not set"
}

```

## OPA unit test (deterministic)

```

package main

test_deny_privileged {
 inp := json.unmarshal(input_for("fixtures/inputs/k8s-deploy_blocks_privileged.yaml"))
 res := eval_deny(inp)
 res[_] == "Privileged container is forbidden"
}

helper to load fixture (no network)
input_for(path) = data.helpers.loadfile(path)

```

---

## Reusable composite action:

[.github/actions/kfm-policy-test/action.yml](https://github.com/actions/kfm-policy-test/action.yml)

```

name: kfm-policy-test
description: Run OPA unit tests + Conftest checks with fixtures & machine-readable
expectations
inputs:
 policy_dir:
 description: Path to policy root (contains policy/ and fixtures/)
 required: false
 default: .
 conftest_paths:
 description: Space-separated list of files/dirs to check

```

```
required: false
default: "fixtures/inputs"
```

```
runs:
```

```
using: composite
```

```
steps:
```

```
- name: Set up tools
```

```
shell: bash
```

```
run: |
```

```
curl -L -o opa https://openpolicyagent.org/downloads/latest/opa_linux_amd64
```

```
chmod +x opa && sudo mv opa /usr/local/bin/opa
```

```
curl -L
```

```
https://github.com/open-policy-agent/conftest/releases/latest/download/conftest_Linux_x86_64.t
ar.gz \
```

```
| tar xz && sudo mv conftest /usr/local/bin/conftest
```

```
- name: OPA unit tests
```

```
shell: bash
```

```
working-directory: ${inputs.policy_dir}
```

```
run: opa test policy/rego policy/tests -v
```

```
- name: Conftest (produce JSON)
```

```
shell: bash
```

```
working-directory: ${inputs.policy_dir}
```

```
run: conftest test ${inputs.conftest_paths} --policy policy/conftest -o json >
```

```
.conftest.out.json
```

```
- name: Compare to expectations
```

```
shell: bash
```

```
working-directory: ${inputs.policy_dir}
```

```
run: |
```

```
python3 - <<'PY'
```

```
import json, glob, sys
```

```
collect outputs grouped by input path
```

```
actual = json.load(open(".conftest.out.json"))
```

```
Build map: path -> {"deny": [...], "warn": [...], "violation": [...]}
```

```
by_path = {}
```

```
for r in actual:
```

```
 p = r.get("file")
```

```
 entries = by_path.setdefault(p, {"deny": [], "warn": [], "violation": []})
```

```
 for e in r.get("warnings", []): entries["warn"].append({"msg": e})
```

```
 for e in r.get("failures", []): entries["deny"].append({"msg": e})
```

```
 for e in r.get("exceptions", []): entries["violation"].append({"msg": e})
```

```
Compare to expectations
```

```
failed = []
```

```
for exp in glob.glob("fixtures/expectations/*.json"):
```

```
 base = exp.split("/")[1].replace(".json", "")
```

```
 # guess input path by name
```

```

candidates = [k for k in by_path.keys() if base in k]
if not candidates:
 failed.append(f"Missing actuals for {exp}")
 continue
act = by_path[candidates[0]]
want = json.load(open(exp))
if act != want:
 failed.append(f"Mismatch:
{base}\nACT={json.dumps(act,indent=2)}\nEXP={json.dumps(want,indent=2)}")
if failed:
 print("\n\n".join(failed)); sys.exit(1)
print("Expectations match.")
PY

```

---

## Use it in any workflow

```

.github/workflows/policy.yml
name: Policy
on: [push, pull_request]
jobs:
 check:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: ./github/actions/kfm-policy-test
 with:
 policy_dir: .
 conftest_paths: "fixtures/inputs"

```

---

## Mutation tests (optional but powerful)

Add a **mutations/** folder with tiny JSON patches and expectations stating what the **mutating policy** should do (e.g., set default **imagePullPolicy**). Then include an extra step that runs your mutator (Rego **patch** entrypoint or a lightweight wrapper), writes the mutated resource, and compares against **fixtures/expectations/<case>.json**. This gives you **deterministic, machine-diffable** proofs that your “auto-fix” logic stayed stable.

---

## KFM guardrails baked in

- **No network** during tests; fixtures are small and versioned.
  - **Fail-closed**: any mismatch with expectations = hard CI fail.
  - **Evidence discipline**: expectations are explicit artifacts you can cite in PRs and release notes.
  - **Consistent gates**: same composite action everywhere → identical semantics.
- 

If you want, I can:

1. turn this into a repo-ready PR (action + examples), 2) wire it into your [.github/workflows/ci.yml](#), and 3) add a couple of canonical fixtures (e.g., privileged container, missing resources) to start the library.

I'm telling you this because the latest sigstore **Cosign** vulnerabilities redefine what you *must* guard against before your next build — from Rekor/bundle mismatches and predicate bypasses to certificate chain logic flaws — and many teams don't realize they've likely been exposed.

([NVD](#))

#### **CVE-2026-22703 – Rekor/Bundling Verification Bypass:**

Before versions **2.6.2** and **3.0.4**, Cosign could be tricked into accepting a Rekor log response that *didn't match the artifact's digest, key, or signature*. An attacker who compromises signing keys could embed arbitrary Rekor entries and still pass verification, defeating auditing entirely.

([NVD](#))

#### **CVE-2026-39395 – Attestation Predicate/Format Bypass:**

In older releases (pre-**3.0.6** and **2.6.3**), `cosign verify-blob-attestation` could falsely return "Verified OK" for attestations with malformed payloads or wrong predicate types because predicate checks were bypassed or mishandled. ([NVD](#))

#### **CVE-2026-24122 – Certificate Chain Expiry Edge Case:**

Cosign versions **3.0.4** and **below** didn't properly enforce expiration checks on issuing certificates if the leaf certificate was still valid — treating stale intermediates as acceptable during verification when timestamps/log validations were skipped. This flaw undermines the trust model, especially in private PKI setups. ([NVD](#))

## **Why This Matters Now**

These aren't theoretical bugs — they strike at assumptions about **transparency log integrity**, **attestation correctness**, and **certificate trust validity** that many CI/CD pipelines rely on for secure deliveries.

## **What's Been Done**

The sigstore project has addressed these with security releases: Cosign **v3.0.6** (and **2.6.3** for 2.x users) includes predicate validation fixes and related patches, and **v3.0.5+** corrects certificate expiry handling. ([GitHub](#))

## What You Should Lock Down

- Pin to patched cosign releases ( $\geq 3.0.6$  or **2.6.3**). ([GitHub](#))
- Refuse any CI runs that skip transparency/timestamping (`-insecure-skip-tlog` etc.).
- Require valid Rekor/log index evidence and strict attestation predicate checks in verification.
- Ensure certificate chain validation is always enforced (no blind acceptance of intermediates).
- Audit your own CI workflows for risky flags and enforce strict bundle formats. ([NVD](#))

Let me know if you want concise remediations or CI policy snippets aligned to these hazards.

Given your KFM posture (evidence-first, fail-closed, governed publication), the highest-value move is to harden the **attestation + transparency verification lane** immediately and make the CI gates deterministic.

## Recommended Immediate Actions

### 1. Pin Cosign Versions Everywhere

Use only:

- `cosign >= 3.0.6`
- or `2.6.3` if a 2.x compatibility lane still exists

Do not allow floating installs like:

go install github.com/sigstore/cosign/v2/cmd/cosign@latest

Prefer immutable version pinning:

```
COSIGN_VERSION="v3.0.6"
```

Use official releases from:

- [Sigstore Cosign Releases](#)
- [Sigstore Documentation](#)

---

# KFM CI Security Gates (Recommended)

## Gate A — Reject Unsafe Verification Flags

### DENY

These should fail CI immediately:

```
--insecure-ignore-tlog=true
--insecure-skip-tlog-verify
--insecure-ignore-sct
--allow-insecure-registry
```

### Example CI Guard

```
#!/usr/bin/env bash
set -euo pipefail

if grep -R \
-E -- '--insecure-(ignore|skip)|allow-insecure-registry' \
.github workflows scripts ci; then
 echo "DENY: insecure cosign flags detected"
 exit 1
fi
```

---

## Gate B — Enforce Rekor Presence

Every release artifact should require:

- Rekor inclusion proof
- Bundle verification
- Timestamp verification
- Identity verification

### Example

```
cosign verify \
--certificate-identity-regexp=".*@github.com" \
...
```



```
--certificate-oidc-issuer="https://token.actions.githubusercontent.com" \
"${IMAGE}"
```

## DO NOT

```
cosign verify --insecure-ignore-tlog=true
```

---

# Recommended KFM Verification Flow

flowchart LR

```
A[Artifact Build] --> B[Generate SBOM]
B --> C[Generate Attestation]
C --> D[Cosign Sign]
D --> E[Upload to Registry]
E --> F[Rekor Inclusion]
F --> G[Bundle Verification]
G --> H[Policy Validation]
H --> I[Promotion Gate]
I --> J[PUBLISHED]
```

---

# KFM-Specific Governance Controls

## Required Promotion Preconditions

Before:

CATALOG/TRIPLET -> PUBLISHED

Require:

- VERIFIED Rekor inclusion
- VERIFIED DSSE bundle
- VERIFIED cert chain
- VERIFIED attestation predicate
- VERIFIED issuer identity
- VERIFIED immutable digest pin

- VERIFIED spec\_hash parity
- 

# Attestation Policy Example

## Accept ONLY

### Predicate Types

<https://slsa.dev/provenance/v1>  
<https://spdx.dev/Document>  
<https://cyclonedx.org/schema>

### DENY Unknown Predicate Types

```
deny[msg] {
 not approved_predicate[input.predicateType]
 msg := sprintf("unsupported predicate type: %s", [input.predicateType])
}
```

---

# Recommended KFM Negative-Path Tests

You strongly prefer fail-closed validation, so these belong in CI:

Test	Expected Outcome
Rekor entry mismatch	DENY
Expired intermediate cert	DENY
Wrong predicate type	DENY
Missing inclusion proof	DENY
Detached bundle mismatch	DENY
Signature digest mismatch	DENY

Wrong OIDC issuer                      DENY

Untrusted GitHub workflow identity    DENY

---

## Strong Recommendation for KFM

Your ecosystem is increasingly oriented toward:

- PMTiles
- DSSE attestations
- release manifests
- provenance envelopes
- EvidenceBundles
- governed publication

So I would formalize a dedicated lane:

tools/attest/

Containing:

tools/attest/

```
|— verify_bundle.py
|— verify_rekor.py
|— verify_predicates.py
|— verify_identity.py
|— verify_release_manifest.py
|— fixtures/
```

That cleanly aligns with your existing:

tools/validators/

governance doctrine.

---

## Strategic Risk Priority

## Highest Risk

Transparency bypasses (Rekor mismatch acceptance)

Because they silently destroy audit trust while appearing valid.

## Second Highest

Predicate confusion / malformed attestations

Because policy engines often assume verified predicates are trustworthy.

## Third

Certificate chain validation edge cases

Especially dangerous in private PKI or offline verification environments.

---

## Recommended KFM Default Posture

NO TRANSPARENCY PROOF = NOT PUBLISHED

NO VERIFIED PREDICATE = NOT ADMISSIBLE

NO VERIFIED IDENTITY = NO PROMOTION

That aligns extremely well with your existing:

Evidence over plausibility

Cite-or-abstain

Fail-closed governance

Publication as a governed state transition

Then we make it **PR-0001 for supply-chain trust**.

## Next Move

Create one tight lane:

tools/attest/

with three jobs:

1. detect unsafe cosign usage
2. verify release artifacts with strict bundle / Rekor / identity rules
3. fail CI on bad attestations before publication

Current upstream evidence supports the concern: Cosign patched the Rekor mismatch issue in [2.6.2](#) / [3.0.4](#), the attestation false-positive issue in [2.6.3](#) / [3.0.6](#), and the certificate-chain expiry issue in [3.0.5](#). ([NVD](#))

## Build Order

### PR-0001: Attestation Guardrails

Add:

```
tools/attest/README.md
tools/attest/check_cosign_flags.py
tools/attest/check_cosign_version.py
tools/attest/verify_release_attestations.py
tests/attest/fixtures/
tests/attest/test_attestation_fail_closed.py
```

## CI Gates

Add jobs:

```
attest-version-gate
attest-unsafe-flags-gate
attest-negative-fixtures-gate
release-attestation-gate
```

## Minimum Policy

```
DENY if cosign < 3.0.6
DENY if insecure tlog flags exist
DENY if predicate type is unsupported
DENY if Rekor/bundle evidence is missing
DENY if cert chain validation is skipped
DENY if artifact digest does not match release manifest
```

# KFM Doctrine Fit

This becomes:

Artifact is not publishable unless its evidence, signature, attestation, identity, and release manifest all agree.

That gives KFM a clean rule:

SIGNED  $\neq$  PUBLISHABLE

ATTESTED  $\neq$  ADMISSIBLE

VERIFIED + POLICY-APPROVED = PROMOTION CANDIDATE

I'd do this before expanding live connectors, UI publication, or tile release automation.

Here's a compact, practical checklist for **verifying PMTiles artifacts in CI**—with just enough background so it's clear what each check proves.

---

## Why these three checks?

- **Range-hash (tile-level inclusion):** proves a specific byte range (a tile payload) in your `.pmtiles` matches what you expect when fetched over HTTP with `Range:` (i.e., no CDN/mirror corruption).
- **Sidecar signature + root check (artifact-level):** proves the *whole* archive hash matches your sidecar's declared root and that the sidecar itself is publisher-signed.
- **Bao/BLAKE3 outboard (slice verification):** proves random slices decode against a BLAKE3 root using outboard proofs—great for streaming integrity.

---

## 1) Range-hash (tile-level inclusion)

Use HTTP Range to fetch a tile-payload window and verify its BLAKE3.

# inputs (example)

START=123456

END=124789

TILE\_URL="https://cdn.example.org/tiles/yourset.pmtiles"

```
EXPECTED_RANGE_HASH="b3f...your_known_blake3_hex"
```

```
curl -sS -H "Range: bytes=${START}-${END}" -o tile.bin "${TILE_URL}"
b3sum tile.bin | awk '{print $1}' | grep -ix "${EXPECTED_RANGE_HASH}"
```

- **Pass condition:** computed `b3sum` equals your expected `range_hash` (from a sidecar or known-good ledger).
  - **Fail mode:** non-zero exit → flag the artifact + byte range in your run receipt.
- 

## 2) Sidecar signature + root hash (artifact-level)

Verify the sidecar is signed (e.g., GitHub OIDC) and that it names the correct archive root.

```
files: pmtiles.pmtiles, sidecar.json, sidecar.bundle.json
verify sidecar signature (OIDC from GitHub Actions) OR with a pinned public cert
cosign verify-blob \
 --bundle sidecar.bundle.json \
 --certificate-oidc-issuer https://token.actions.githubusercontent.com \
 --certificate-identity-regexp='^https://github.com/.+' \
 sidecar.json
```

```
compute archive root and compare to sidecar.root_hash
ARCHIVE_HASH="$(b3sum pmtiles.pmtiles | awk '{print $1}')"
jq -er '.root_hash' sidecar.json | grep -ix "${ARCHIVE_HASH}"
```

- **Pass condition:** cosign verification succeeds and `b3sum(pmtiles)` equals `sidecar.root_hash`.
  - **Fail closed:** treat any mismatch as a hard failure; record the artifact's name + computed hash.
- 

## 3) Bao / BLAKE3 outboard streaming (slice verification)

Validate slices against a BLAKE3 root using outboard proofs.

```
install bao CLI once in CI (cache target as needed)
cargo install bao_bin # or build locally:
cargo build --release
```

```
fetch archive and its outboard proof
curl -sS -o tiles.pmtiles "https://cdn.example.org/tiles.pmtiles"
curl -sS -o tiles.obao "https://cdn.example.org/tiles.obao"

compute root (prefer bao, fallback to b3sum for the hex)
hash="$(./target/release/bao hash tiles.pmtiles || b3sum tiles.pmtiles | cut -d' ' -f1)"

decode (verification will fail if bytes/proof don't match the root)
./target/release/bao decode "$hash" tiles.pmtiles --outboard tiles.obao > /dev/null
```

- **Pass condition:** decode exits 0 (proof and bytes consistent).
- **Fail mode:** non-zero exit → corrupted bytes or invalid outboard proof.

---

## Drop-in GitHub Actions example

Paste into [.github/workflows/verify-pmtiles.yml](#) and tweak URLs/paths:

```
name: Verify PMTiles
on:
 workflow_dispatch:
 push:
 paths:
 - "dist/**/*.pmtiles"
 - "dist/**/*.json"
 - "dist/**/*.obao"
jobs:
 verify:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - name: Install tools
 run: |
 sudo apt-get update
 sudo apt-get install -y jq curl b3sum
 curl -sSL https://github.com/sigstore/cosign/releases/latest/download/cosign-linux-amd64
 -o cosign
 chmod +x cosign && sudo mv cosign /usr/local/bin/cosign
 sudo apt-get install -y cargo || true
 cargo install bao_bin || true
```



```

- name: Fetch artifacts
 run: |
 # TODO: replace with your artifact URLs or use actions/download-artifact
 curl -sS -o pmtiles.pmtiles "https://cdn.example.org/tiles/yourset.pmtiles"
 curl -sS -o sidecar.json "https://cdn.example.org/tiles/yourset.sidecar.json"
 curl -sS -o sidecar.bundle.json "https://cdn.example.org/tiles/yourset.sidecar.bundle.json"
 curl -sS -o tiles.obao "https://cdn.example.org/tiles/yourset.obao"

- name: Check sidecar signature + root hash
 run: |
 cosign verify-blob \
 --bundle sidecar.bundle.json \
 --certificate-oidc-issuer https://token.actions.githubusercontent.com \
 --certificate-identity-regexp='^https://github.com/.+' \
 sidecar.json

 ARCHIVE_HASH="$(b3sum pmtiles.pmtiles | awk '{print $1}')"
 jq -er '.root_hash' sidecar.json | grep -ix "${ARCHIVE_HASH}"

- name: Range-hash sample (tile-level inclusion)
 env:
 START: "123456"
 END: "124789"
 run: |
 curl -sS -H "Range: bytes=${START}-${END}" \
 -o tile.bin "https://cdn.example.org/tiles/yourset.pmtiles"
 EXPECTED="$(jq -r '.range_hashes["${START}-${END}"] // empty' sidecar.json)"
 test -n "$EXPECTED" || (echo "Missing expected range_hash in sidecar" && exit 2)
 b3sum tile.bin | awk '{print $1}' | grep -ix "$EXPECTED"

- name: Outboard Bao verification (slice streaming)
 run: |
 HASH="$(bao hash pmtiles.pmtiles || b3sum pmtiles.pmtiles | cut -d' ' -f1)"
 bao decode "$HASH" pmtiles.pmtiles --outboard tiles.obao > /dev/null

- name: Record run receipt
 if: always()
 run: |
 printf '{ "artifact": "pmtiles.pmtiles", "root_hash": "%s", "status": "%s" }\n' \
 "$(b3sum pmtiles.pmtiles | awk '{print $1}')" \
 "${{ job.status }}" | tee run_receipt.json
 echo "Run status: ${{ job.status }}"

```

---

## Tips

- Keep **expected hashes** (range slices and root) in a **governed sidecar** under version control or an artifact registry; don't hide them in CI variables.
- Prefer **fail-closed** semantics: any missing expected field or verification step should hard-fail the job.
- Log a compact **run receipt** (artifact ID, computed hashes, pass/fail) so you can diff runs over time.

If you want, I can tailor the snippets to your exact URLs, sidecar schema, and where you publish the `.obao/.bao` proofs.

Excellent. The next hardening step for KFM is to combine all three verification lanes into a single governed CI gate:

1. **Artifact root integrity** (whole `.pmtiles`)
2. **Range inclusion integrity** (tile/slice verification)
3. **Publisher attestation integrity** (signed provenance + policy)

That gives you:

- CDN corruption detection
- byte-range tamper detection
- replay detection
- provenance verification
- release reproducibility
- governed rollback confidence

A strong KFM pattern is:

PMTiles

- root hash (BLAKE3)
- Bao outboard proof
- DSSE attestation
- Cosign signature
- Range hash ledger
- Release manifest
- Run receipt

---

## Recommended KFM Verification Stack

Layer	Purpose	Fail Condition
BLAKE3 root	whole archive integrity	root mismatch
Bao outboard	stream/slice proof	invalid proof
Range hash	tile payload inclusion	slice mismatch
Cosign	signer identity	invalid cert/signature
DSSE envelope	tamper-evident provenance	envelope invalid
Rekor inclusion	transparency log	entry missing
Policy gate	publication governance	denied release

---

## Stronger CI Pattern (Recommended)

Instead of:

```
b3sum pmtiles.pmtiles
```

Prefer:

```
bao hash pmtiles.pmtiles
```

Because Bao:

- preserves streaming proof semantics
  - aligns with outboard verification
  - supports partial validation
  - avoids recomputing whole-file trust repeatedly
- 

## Recommended Artifact Layout

```
dist/
├── kansas.pmtiles
└── kansas.pmtiles.obao
```

```
├── kansas.sidecar.json
├── kansas.dsse.json
├── kansas.bundle.json
├── kansas.release.json
├── kansas.range-ledger.json
└── kansas.run-receipt.json
```

---

## Recommended Sidecar Schema

```
{
 "artifact": "kansas.pmtiles",
 "root_hash": "b3:abcd...",
 "hash_alg": "blake3",
 "bao_outboard": "kansas.pmtiles.obao",
 "created_at": "2026-05-11T18:00:00Z",
 "spec_version": "v1",
 "range_hashes": {
 "123456-124789": "b3:ffff..."
 }
}
```

---

## Recommended DSSE Predicate

Use DSSE to bind:

- source commit
- build workflow
- source ledger snapshot
- spec\_hash
- release manifest
- policy decision

Example predicate fields:

```
{
 "predicateType": "https://kfm.dev/attestation/pmtiles/v1",
 "subject": [
 {
```

```
 "name": "kansas.pmtiles",
 "digest": {
 "blake3": "abcd..."
 }
 },
 "predicate": {
 "git_sha": "...",
 "workflow": "...",
 "release_manifest": "...",
 "source_snapshot": "...",
 "policy_decision": "ALLOW",
 "review_state": "released"
 }
}
```

---

## CI Negative-Path Tests (Important)

KFM should fail closed on all of these:

Test	Expected
mutated tile byte	FAIL
modified outboard	FAIL
modified DSSE payload	FAIL
wrong signer	FAIL
stale release manifest	FAIL
unsigned sidecar	FAIL
missing range hash	FAIL
Rekor absence	FAIL
policy != ALLOW	FAIL

---

# Recommended GitHub Actions Additions

Add:

- Rekor verification
  - immutable artifact naming
  - ORAS push/pull verification
  - offline verification mode
  - policy gate stage
- 

## Example Rekor Verification

```
cosign verify-blob \
--bundle sidecar.bundle.json \
sidecar.json
```

This validates:

- transparency log inclusion
  - Fulcio cert chain
  - identity claims
  - signature binding
- 

## Recommended Immutable Naming

Instead of:

kansas.pmtiles

Prefer:

kansas\_b3\_abcd1234.pmtiles

Benefits:

- cache safety
  - deterministic CDN invalidation
  - immutable publication
  - rollback simplicity
- 

## KFM Governance Recommendation

Treat PMTiles as:

“Derived evidence carriers”

NOT canonical truth.

Canonical truth remains:

- EvidenceBundle
- catalog/triplet lineage
- release manifest
- source ledger
- policy decisions

Tiles are governed publication artifacts.

That distinction matters enormously later.

---

## Ideal Verification Pipeline

flowchart TD

```
A[Build PMTiles] --> B[Compute BLAKE3 Root]
B --> C[Generate Bao Outboard]
C --> D[Generate Range Ledger]
D --> E[Generate DSSE Predicate]
E --> F[Cosign Sign]
F --> G[Rekor Inclusion]
G --> H[Policy Validation]
H --> I[Publish]
I --> J[CI Verification]
```

---

# Strong Recommendation

Eventually add:

- Merkle tile manifests
- deterministic tile ordering
- CAR/IPLD mirror support
- OCI/ORAS publication
- release promotion attestations
- rollback attestations
- correction lineage attestations

Those become extremely valuable once KFM starts:

- multi-domain overlays
- delta publication
- temporal tile reconstruction
- public evidence replay
- reproducible historical states

That is where PMTiles becomes a governed publication substrate rather than “just tiles.”

I’m sharing this because there’s a powerful, real-time way to access biodiversity data that you can immediately plug into your workflows without waiting for static downloads.

The **Global Biodiversity Information Facility (GBIF)** is a global open data infrastructure that lets you query millions of species occurrence records — observations and specimen data from around the world — via web services. ([GBIF](#))

Its **Occurrence API** supports **live, paginated search**, meaning you can fetch occurrence records matching filters like country, state/province (e.g., Kansas), county, or bounding box in real time. ([TechDocs](#))

You can tailor API calls — for example with `stateProvince=Kansas` and other predicates — to get fields such as scientific name, event date, coordinates, dataset key, and license in JSON. ([TechDocs](#))

For large, reproducible data pulls (bulk exports), GBIF also offers **asynchronous downloads** (`occ_download`) that let you request and then retrieve prepared data files (CSV, Parquet, Darwin Core Archive) once they’re ready. ([TechDocs](#))



Programmatic clients in **R** ([rgbif](#)) and **Python** ([pygbif](#)) wrap these APIs, making it easier to script search and download workflows without hand-crafting HTTP requests. ([TechDocs](#))

There's even an experimental **SQL download API** if you want to run more complex, SQL-style queries against the occurrence database, though that requires a registered GBIF account. ([GBIF Data Blog](#))

If you're thinking about integrating real-time biodiversity data into dashboards, pipelines, or CI watchers, this API is designed for exactly that.

I want to make sure you have the context and pathways you need to build a **robust, machine-readable plant species backbone and occurrence workflow** for U.S. plants (including Kansas), drawing on authoritative checklist metadata and occurrence feeds.

The U.S. **USDA PLANTS Database** is an authoritative national checklist of plant taxa (vascular plants, mosses, liverworts, hornworts, lichens) covering names, symbols, common names, families, distributions, and more — and it's published on **GBIF as a checklist dataset with a DOI** you can use as a taxonomic backbone. ([GBIF](#))

The **Complete PLANTS Checklist** is available for bulk download from the USDA PLANTS downloads page, where it is distributed as CSV and includes fields like scientific name, symbol, and distribution codes. ([USDA Plants Database](#))

On the **GBIF side**, that same PLANTS dataset appears as dataset **705922f7-5ba5-49ab-a75d-722e3090e690** with a DOI accessible via the GBIF API — great for automated metadata harvesting and citation in workflows. ([GBIF](#))

GBIF's infrastructure lets you join occurrences to a checklist via names or taxon keys, which is essential if you are exporting **Kansas occurrences** from GBIF and want them aligned with USDA PLANTS as a taxonomic authority. ([GBIF](#))

If you're pulling occurrences for Kansas, keep in mind that state-level conservation datasets (like those maintained by the **Kansas Natural Heritage Inventory**) contain **rare and sensitive plant locations** that are often restricted or must be redacted/generalized before publication. ([BioSurvey](#))

To support efficient spatial workflows:

- Store occurrence data in a **projected continental CRS (e.g., EPSG:5070)** for analysis, and
- Derive **WebMercator (EPSG:3857)** tiles (e.g., PMTiles) for web map layers.

This combination gives you a **checklist backbone (USDA PLANTS) + GBIF occurrences + spatial readiness** for both analytical and UI use cases, while respecting sensitive data constraints.

Here's a drop-in, privacy-first consent & proof UI pattern you can use right away—built to fit KFM's evidence-first, fail-closed posture without leaking PII.

---

## 3-Step “Opt-In → Proof → Render” pattern (for Pass N+1 overlays)

### 0) Why this exists (quick background)

When overlays touch sensitive stuff (genetics, family links, precise locations), you need explicit consent, selective disclosure, and verifiable proofs before anything renders. This pattern ensures: deny-by-default, no raw PII in storage, revocation works, and every render is reconstructable to evidence.

---

### 1) Opt-In (compact consent card)

What the user sees

- Small card with:
  - Scopes (checkboxes): **Genetic, Kinship, Location**
  - 2–3 sentence human-readable policy summary
  - “**Sample Redaction Preview**” toggle (live demo of rounding, blurring, initials, etc.)
  - **Accept** button

What happens under the hood

- On accept, issue a **selective-disclosure verifiable credential** (SD-JWT or BBS+), scoped to chosen boxes.
- Persist **only** a **DSSE-signed consent receipt pointer** in the EvidenceBundle (no raw PII, fail-closed if pointer missing/unverifiable).
- Nothing renders yet.

#### Consent receipt (minimal example)

```
{
 "object_type": "ConsentReceiptPointer",
 "schema_version": "v1",
```

```
"pointer_uri": "evidence://consent/abcd1234",
"dsse_envelope": { "payloadType": "application/consent+json", "payload": "<b64>",
"signatures": [/*...*/] },
"scopes": ["location","kinship"],
"expires_at": "2026-12-31T00:00:00Z"
}
```

---

## 2) Sample Presentation Proof (pre-render check)

What the user sees

- A **derived presentation preview** panel listing exactly which fields will be disclosed and what redactions will apply.
- “**Verify proof**” button + result badge.

What happens under the hood

- Build a derived presentation from the SD-JWT/BBS+ credential: only the fields needed for this overlay instance.
- Verify:
  1. SD-JWT/BBS+ proof is valid
  2. DSSE signature on the consent receipt pointer
  3. Privacy-preserving **revocation** status (e.g., status list / accumulator) = **not revoked**
- If any check fails → **DENY** and show “why”.

### Presentation (conceptual)

```
{
 "presentation": {
 "disclosed": { "relationship": "cousin", "lat": 38.97, "lon": -95.24 },
 "redactions": { "name": "J.D.", "dna_markers": "hashed-aggregate" }
 },
 "proofs": { "sdjwt": "...", "revocation": "good", "dsse": "valid" }
}
```

---

## 3) Render + Redact (only after verification)

What the user sees

- The overlay with enforced redactions plus a small control strip:
  - **Opt-in** toggle (view/change scopes)
  - **Revoke consent** (flips revocation; future presentations auto-DENY)
  - **View proof** inspector (shows disclosed fields & proof status)
  - **Audit** link (to EvidenceBundle trail)

What happens under the hood (obligations enforced)

- **Location**: round coordinates (grid cell or N decimals).
- **Names**: replace with relationship labels or initials.
- **DNA/Genomics**: hash/aggregate to non-PII risk indicators.
- **Images**: blur or mosaic sensitive regions.
- Log a **RenderReceipt** pointing to: input refs → consent pointer → proof pack → policy decision.

### RenderReceipt (minimal)

```
{
 "object_type": "RenderReceipt",
 "schema_version": "v1",
 "evidence_refs": ["evidence://consent/abcd1234","evidence://proofs/pres-7890"],
 "policy_decision": "ALLOW_WITH_OBLIGATIONS",
 "obligations_enforced":
["round_location:0.01","initials_only","dna_risk_aggregate","blur_faces"],
 "spec_hash": "sha256:...",
 "outcome": "ANSWER"
}
```

---

## Storage trade-offs (pick one per scope)

- **Link** → **pointer-only** (recommended): store just the DSSE-signed pointer to consent/proof.  
*Pros*: best revocation, minimal storage, easy traceability. *Cons*: requires network to resolve pointer.
  - **Embed** → **ephemeral client fetch** (tokened blob kept client-side, expires):  
*Pros*: snappy UX. *Cons*: moderate revocation control; offline caching risk.
  - **Import** → **ingest raw identifiers** (strictly governed):  
*Pros*: strongest audits. *Cons*: highest legal/privacy risk; must be explicit, encrypted, and retention-gated. Avoid unless absolutely necessary.
-

## Drop-in UI checklist (dev-ready)

- ConsentCard(s): scopes + short policy + preview toggle + Accept
  - Credential issuance (SD-JWT/BBS+) scoped to selections
  - DSSE-signed ConsentReceiptPointer in EvidenceBundle
  - PresentationPreview with exact disclosed fields & redactions
  - ProofVerifier: SD-JWT/BBS+, DSSE, revocation
  - OverlayRenderer with obligations (round/initials/hash/blur)
  - Controls: Opt-in toggle, Revoke, View proof, Audit link
  - RenderReceipt persisted (traceable chain), fail-closed defaults
- 

## Failure-mode defaults (match KFM)

- Missing/expired consent pointer → **DENY**
- Proof unverifiable or revocation unknown → **DENY**
- Redaction obligation not enforceable in this view → **ABSTAIN**
- Any unresolved sensitivity → **ABSTAIN** (push to steward path)

If you want, I can turn this into a repo-ready Markdown (with badges, quick links, and a mermaid flow) or a minimal React/Vue component scaffold next.

I'm sharing this because the latest systems research is converging on registries for AI agents that combine scalable artifact distribution with *cryptographically verifiable descriptive metadata*, which is exactly the kind of foundation that would make an AI-driven dataset explorer trustworthy and auditable.

Recent proposals like the **Agent Directory Service (ADS)** define a distributed directory for AI agent capabilities built on content-addressable artifacts and signed metadata: agents push structured manifests describing *what they can do*, performance traits, and associated provenance into OCI-compatible registries that reuse **OCI/ORAS** distribution and **Sigstore** signing for trust. These manifests live as immutable, versioned records that can be *globally discovered* via hierarchical taxonomy and content hashes, enabling queries like “find agents that can do X with acceptable latency” across decentralized storage layers. ([arXiv](#))

The architectural core reuses **OCI registry semantics** for artifact transport and durability, and leverages **Sigstore-style signatures** for cryptographic integrity and provenance tracking, so every published capability manifest and associated metadata can be verified and audited before use. ([Oras](#))

In practice, this produces a layered discovery model where:

- **Capability manifests** (signed descriptors of functionality, costs, constraints) are pushed to OCI/ORAS registries.
- **Discovery indices and taxonomies** map human- and machine-friendly skill categories to content addresses.
- **Cryptographic provenance** enables audit trails and trustworthy selection, linking artifact records to verified origins. ([IETF Datatracker](#))

This kind of registry design lays the groundwork for an *AI agent registry* in a KFM Focus Mode workflow, where the system can reliably look up, reason about, and compose signed capability inventories at runtime with auditability and trust baked in. ([arXiv](#))

Here's a **quick tour of the real-world data sources and APIs you'd weave together for automated county-level environmental and land metrics** — exactly the kind of authoritative endpoints you referenced:

### Agricultural and land cover ground truth

- **USDA NASS Cropland Data Layer (CDL)** gives annual crop-type and land cover rasters across the U.S., historically back to 1997 and now at ~10-m resolution. It's public-domain and widely used for land use analysis, crop monitoring, and change detection. ([USDA NASS](#))
- **USDA PLANTS Database** catalogs plants (vascular plants, mosses, etc.) with range maps by state/county and a downloadable checklist; it's designed for distribution and conservation insights. ([USDA Plants Database](#))
- **NRCS SSURGO soil data** provides detailed soils maps and tabular attributes useful for productivity and land suitability analyses at county or polygon scale. ([USGS](#))

### Hydrology and water quality

- **USGS Water Services / NWIS APIs** serve real-time and historical water data (streamflow, stages, water quality) via REST endpoints, with daily and instantaneous values accessible for hydrologic monitoring locations nationwide. ([Water Services Web](#))

### Air quality and environmental monitoring

- **EPA AirData/AirNow** provides daily summaries of criteria pollutants (ozone, PM2.5, PM10, etc.) and downloadable time series for monitors — essential for daily AQI and pollutant change triggers. ([US EPA](#))

### Land cover change & satellite archives

- **MRLC National Land Cover Database (NLCD)** maps broad land covers at ~30-m resolution and supports long-term change analysis; the Annual NLCD project now offers yearly land cover products back to 1985. ([USGS](#))

- **Microsoft Planetary Computer STAC** provides cataloged, cloud-native access to CDL and other remote sensing datasets (Sentinel/Landsat, etc.) via the STAC API for scalable queries. ([Planetary Computer](#))

## Biodiversity and occurrence

- **GBIF Occurrence API** gives global species occurrence records (including plants and animals), with machine-readable licensing and filters that let you pull records by county or polygon for ecological context. (Licensing varies by dataset but is machine-tagged CC0/CC-BY/CC-BY-NC.)

Each of these **authoritative services is built for automation** — downloadable files, REST APIs, or STAC endpoints — and can be integrated into scheduled workflows with **license metadata carried alongside the data** so you can enforce usage constraints (public domain vs. CC-BY/NC). Locking in fingerprints and surface change thresholds (like area/class shifts or AQL excursions) on the trailing window completes a robust pipeline.

Here are three quick MapLibre wins you can flip on in CI smoke tests to make maps feel snappier without big refactors.

---

# 1) Tile caching (fast “first pan”)

## Browser (GL JS):

```
const map = new maplibregl.Map({
 container: 'map',
 style: '...',
 center: [-98.6, 39.8],
 zoom: 6,
 // trade memory for responsiveness:
 maxTileCacheSize: 200, // number of tiles
 maxTileCacheZoomLevels: 3 // keep near-zoom neighbors warm
});
```

- - **Native (iOS/Android):** temporarily disable the on-disk cache for deterministic CI, or trim it between tests  
(e.g., `setTileCacheEnabled(false)` / equivalent in your SDK).
  - **Smoke test toggles:** run with `{200, 3}` and `{50, 1}` to see jank deltas.
-

## 2) Push invariants into layout, keep paint light

- Put stable styling (symbols shown/hidden, text-allow-overlap, line-caps) in **layout**—it's evaluated at integer zooms and won't recompute per frame.
- Keep **paint** expressions simple; avoid frequent **feature-state** churn unless you really need per-feature interactivity.

Quick refactor pattern:

```
{
 "id": "roads",
 "type": "line",
 "source": "roads",
 "layout": { "line-cap": "round", "visibility": "visible" }, // invariant
 "paint": { "line-width": ["interpolate", ["linear"], ["zoom"], 6, 0.5, 12, 2] }
}
```

- - **Smoke test:** toggle a “heavy paint” branch vs “layout-first” branch and compare FPS / CPU in DevTools Performance.
- 

## 3) Vector size down, decode faster (before you tile)

- **Quantize/simplify upstream:** use **tippecanoe/mapshaper** to reduce verts *before* tiling.

Examples

# tippecanoe examples

```
tippecanoe -o roads.pmtiles -zg --drop-densest-as-needed --extend-zooms-if-still-dropping
roads.ndjson
```

```
tippecanoe -o landuse.pmtiles -Z5 -z12 --coalesce-densest-as-needed
--drop-smallest-as-needed landuse.geojson
```

# mapshaper example (quantize + simplify)

```
mapshaper input.geojson -quiet -simplify dp 10% keep-shapes -o format=geojson
quantization=1e5 out.geojson
```



- **Reduce MVT `extent` (encode-time)**: smaller grids → smaller tiles (watch tolerance artifacts).
  - **Try MapLibre Tile (MLT)**: benchmark MLT vs MVT for your layers; decode can be faster with smaller payloads.
  - **Smoke test**: generate two tiny fixture sets (heavy vs slim), point your style at each, measure `map.on('idle')` time from load.
- 

## Drop-in CI checklist (copy/paste)

- JS: set `maxTileCacheSize=200`, `maxTileCacheZoomLevels=3`; native: disable/trim cache between runs.
- Move stable rules to `layout`; simplify `paint`; remove unnecessary `feature-state`.
- Produce a “slim” PMTiles set (quantize/simplify); compare to baseline on the same style.
- Record: initial render time, time-to-first-interaction, p95 frame time during 5-second pan.

If you want, I can turn this into a ready-to-run Node script that loads a headless MapLibre scene, flips these knobs, and prints a tiny perf table.

```
// scripts/maplibre-smoke-perf.mjs
//
// PURPOSE
// -----
// Tiny MapLibre smoke/perf harness for CI.
// Flips:
// 1. tile cache knobs
// 2. layout-first vs heavy-paint styles
// 3. slim vs heavy PMTiles/vector fixtures
//
// OUTPUT
// -----
// Prints a small perf table:
// scenario | load_ms | idle_ms | frames | avg_frame_ms
//
// REQUIREMENTS
// -----
// npm i playwright
//
// OPTIONAL
// -----
// Serve your fixtures locally:
// npx serve .
// or:
```

```

// python3 -m http.server 8080
//
// RUN
// ---
// node scripts/maplibre-smoke-perf.mjs
//
// NOTE
// ----
// This is PROPOSED reference scaffolding.
// Adjust style URLs / fixture URLs to match your repo.
//

import { chromium } from "playwright";
import fs from "node:fs/promises";

const scenarios = [
 {
 name: "cache-heavy-layout-slim",
 cache: { maxTileCacheSize: 200, maxTileCacheZoomLevels: 3 },
 styleMode: "layout-first",
 tiles: "slim"
 },
 {
 name: "cache-light-heavy-paint-heavy",
 cache: { maxTileCacheSize: 50, maxTileCacheZoomLevels: 1 },
 styleMode: "heavy-paint",
 tiles: "heavy"
 }
];

// -----
// STYLE FIXTURES
// -----

function buildStyle({ styleMode, tiles }) {
 const sourceUrl =
 tiles === "slim"
 ? "http://localhost:8080/fixtures/slim/style.json"
 : "http://localhost:8080/fixtures/heavy/style.json";

 // Example inline mutation.
 // In a real repo, load + patch your style JSON instead.
 return {
 version: 8,

```

```

glyphs: "https://demotiles.maplibre.org/font/{fontstack}/{range}.pbf",
sources: {
 demo: {
 type: "vector",
 url: sourceUrl
 }
},
layers:
 styleMode === "layout-first"
 ? [
 {
 id: "roads",
 type: "line",
 source: "demo",
 "source-layer": "roads",
 layout: {
 visibility: "visible",
 "line-cap": "round"
 },
 paint: {
 "line-width": [
 "interpolate",
 ["linear"],
 ["zoom"],
 5,
 0.5,
 12,
 2
],
 "line-color": "#888"
 }
 }
]
 : [
 {
 id: "roads",
 type: "line",
 source: "demo",
 "source-layer": "roads",
 paint: {
 "line-width": [
 "case",
 [">", ["zoom"], 10],
 4,

```

```

 1
],
 "line-opacity": [
 "interpolate",
 ["linear"],
 ["zoom"],
 5,
 0.2,
 14,
 1
],
 "line-color": [
 "match",
 ["get", "class"],
 "primary",
 "#ff0000",
 "secondary",
 "#00ff00",
 "#888"
]
}
}
}
]
};
}

```

```

// -----
// MAIN
// -----

```

```

const browser = await chromium.launch({
 headless: true
});

```

```

const results = [];

```

```

for (const scenario of scenarios) {
 const page = await browser.newPage();

```

```

 await page.setContent(`
<!DOCTYPE html>
<html>
<head>
 <meta charset="utf-8"/>

```

```

<title>MapLibre Perf</title>
<style>
 html, body, #map {
 margin: 0;
 width: 100%;
 height: 100%;
 }
</style>

<link
 href="https://unpkg.com/maplibre-gl@5.5.0/dist/maplibre-gl.css"
 rel="stylesheet"
/>

<script src="https://unpkg.com/maplibre-gl@5.5.0/dist/maplibre-gl.js"></script>
</head>
<body>
 <div id="map"></div>
</body>
</html>
`);

```

```

const metrics = await page.evaluate(async (scenario) => {
 const style = ({buildStyle.toString()})(scenario);

```

```

 const frameTimes = [];
 let lastFrame = performance.now();

```

```

 const rafLoop = () => {
 const now = performance.now();
 frameTimes.push(now - lastFrame);
 lastFrame = now;

```

```

 if (frameTimes.length < 300) {
 requestAnimationFrame(rafLoop);
 }
 };

```

```

 requestAnimationFrame(rafLoop);

```

```

 const loadStart = performance.now();

```

```

 const map = new maplibregl.Map({
 container: "map",

```

```

 style,
 center: [-98.6, 39.8],
 zoom: 6,
 attributionControl: false,

 maxTileCacheSize: scenario.cache.maxTileCacheSize,
 maxTileCacheZoomLevels:
 scenario.cache.maxTileCacheZoomLevels
 });

const loadMs = await new Promise((resolve) => {
 map.on("load", () => {
 resolve(performance.now() - loadStart);
 });
});

const idleStart = performance.now();

// scripted pan to simulate smoke interaction
map.easeTo({
 center: [-97.2, 38.9],
 zoom: 8,
 duration: 2500
});

const idleMs = await new Promise((resolve) => {
 map.on("idle", () => {
 resolve(performance.now() - idleStart);
 });
});

const avgFrameMs =
 frameTimes.reduce((a, b) => a + b, 0) /
 frameTimes.length;

map.remove();

return {
 loadMs: Math.round(loadMs),
 idleMs: Math.round(idleMs),
 frames: frameTimes.length,
 avgFrameMs: Number(avgFrameMs.toFixed(2))
};
}, scenario);

```

```

 results.push({
 scenario: scenario.name,
 ...metrics
 });

 await page.close();
 }

 await browser.close();

 // -----
 // REPORT
 // -----

 console.table(results);

 await fs.writeFile(
 "./maplibre-perf-results.json",
 JSON.stringify(results, null, 2)
);

 console.log(
 "\nWrote: maplibre-perf-results.json"
);

```

## Suggested fixture structure

```

fixtures/
├── heavy/
│ └── style.json
└── slim/
 └── style.json

```

## Suggested CI assertions

```
.github/workflows/maplibre-smoke.yml
```

```

- name: Run MapLibre smoke perf
 run: |
 python3 -m http.server 8080 &
 node scripts/maplibre-smoke-perf.mjs

```

- name: Assert perf envelope

```
run: |
 node -e "
 const fs = require('fs');
 const r = JSON.parse(
 fs.readFileSync('maplibre-perf-results.json')
);

 for (const row of r) {
 if (row.avgFrameMs > 22) {
 console.error('Frame budget exceeded:', row);
 process.exit(1);
 }
 }

 console.log('Perf envelope OK');
 "
```

## Next high-value expansions

- PMTiles vs MLT decode comparison
- WebGL memory telemetry
- Cesium + MapLibre synchronized pan smoke tests
- p95 frame-time assertions
- Render diff snapshots
- GPU timing extension checks
- Mobile Safari tile-pressure scenarios
- Evidence-linked ReleaseManifest perf receipts for KFM publication gates

A few strong next-step upgrades for this harness inside KFM would be:

- attach DSSE/Cosign receipts to perf runs
- emit a governed **RunReceipt**
- persist frame telemetry as release evidence
- compare PMTiles vs MLT in the same matrix
- add Cesium synchronized camera smoke tests
- generate render-diff artifacts for regression review
- fail closed on FPS regressions beyond a defined envelope

Example governed artifact set:

```
artifacts/perf/
├── run-receipt.json
└── perf-results.json
```



- frame-times.csv
- render-diff/
- screenshots/
- dsse-envelope.json
- cosign.bundle

Example perf receipt shape:

```
{
 "object_type": "RunReceipt",
 "domain": "maplibre",
 "run_id": "perf-smoke-2026-05-14T16:00:00Z",
 "status": "completed",
 "scenarios": [
 {
 "name": "cache-heavy-layout-slim",
 "avg_frame_ms": 12.4,
 "idle_ms": 1840,
 "tile_set": "slim",
 "style_mode": "layout-first"
 }
],
 "policy_posture": "public_safe",
 "spec_hash": "NEEDS_VERIFICATION"
}
```

Good CI thresholds for early KFM renderer governance:

Metric	Suggested Gate
avg frame ms	< 22ms
p95 frame ms	< 40ms
initial load	< 4s
idle after pan	< 3s
render diff	< 1% pixel delta

Best near-term perf wins after this:

1. Glyph atlas warming
2. Sprite consolidation
3. Source compositing
4. Feature-state reduction
5. Tile extent tuning
6. Overzoom strategy cleanup
7. Worker-count tuning
8. Symbol collision pruning
9. Expression flattening
10. Pre-generalized geometry tiers

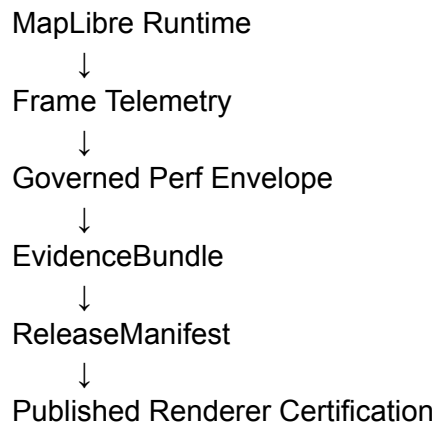
For PMTiles specifically:

```
pmtiles cluster roads.pmtiles
pmtiles verify roads.pmtiles
```

And for Tippecanoe optimization sweeps:

```
tippecanoe \
-zg \
--drop-densest-as-needed \
--coalesce-densest-as-needed \
--extend-zooms-if-still-dropping \
--detect-shared-borders \
-o optimized.pmtiles \
input.geojson
```

Longer-term KFM direction that would pay off heavily:



That creates auditable renderer-performance lineage instead of “it felt fast on my machine.”

Next build piece: **KFM-governed MapLibre perf receipt schema + validator.**

```
tools/validators/maplibre/
├── validate_perf_receipt.py
└── fixtures/
 ├── valid.perf-receipt.json
 └── invalid.missing-spec-hash.json
```

```
schemas/maplibre/
└── perf-receipt.schema.json
```

## schemas/maplibre/perf-receipt.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/perf-receipt/v1",
 "title": "MapLibre Performance Run Receipt",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "run_id",
 "created_utc",
 "domain",
 "status",
 "policy_posture",
 "spec_hash",
 "scenarios"
],
 "additionalProperties": false,
 "properties": {
 "object_type": {
 "const": "RunReceipt"
 },
 "schema_version": {
 "const": "v1"
 },
 "run_id": {
 "type": "string",
 "minLength": 8
 },
 "created_utc": {
 "type": "string",
```

```
 "format": "date-time"
 },
 "domain": {
 "const": "maplibre"
 },
 "status": {
 "enum": ["completed", "failed", "aborted"]
 },
 "policy_posture": {
 "enum": ["public_safe", "restricted", "review_required"]
 },
 "spec_hash": {
 "type": "string",
 "minLength": 12
 },
 "scenarios": {
 "type": "array",
 "minItems": 1,
 "items": {
 "type": "object",
 "required": [
 "name",
 "style_mode",
 "tile_set",
 "avg_frame_ms",
 "p95_frame_ms",
 "idle_ms",
 "load_ms",
 "passed"
],
 "additionalProperties": false,
 "properties": {
 "name": { "type": "string" },
 "style_mode": { "type": "string" },
 "tile_set": { "type": "string" },
 "avg_frame_ms": { "type": "number" },
 "p95_frame_ms": { "type": "number" },
 "idle_ms": { "type": "number" },
 "load_ms": { "type": "number" },
 "passed": { "type": "boolean" }
 }
 }
 }
}
```

```
}
```

## tools/validators/maplibre/validate\_perf\_receipt.py

```
#!/usr/bin/env python3
```

```
import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator, FormatChecker

SCHEMA_PATH = Path("schemas/maplibre/perf-receipt.schema.json")

def main() -> int:
 if len(sys.argv) != 2:
 print("usage: validate_perf_receipt.py <receipt.json>", file=sys.stderr)
 return 2

 receipt_path = Path(sys.argv[1])

 schema = json.loads(SCHEMA_PATH.read_text())
 receipt = json.loads(receipt_path.read_text())

 validator = Draft202012Validator(
 schema,
 format_checker=FormatChecker()
)

 errors = sorted(
 validator.iter_errors(receipt),
 key=lambda e: e.path
)

 if errors:
 print("DENY: invalid MapLibre perf receipt")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

 failed = [
```

```

 s["name"]
 for s in receipt["scenarios"]
 if not s["passed"]
]

 if failed:
 print("DENY: one or more scenarios failed perf envelope")
 for name in failed:
 print(f"- {name}")
 return 1

 print("ANSWER: MapLibre perf receipt valid")
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## Valid fixture

```

{
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "run_id": "maplibre-perf-2026-05-14T16:00-00Z",
 "created_utc": "2026-05-14T16:00:00Z",
 "domain": "maplibre",
 "status": "completed",
 "policy_posture": "public_safe",
 "spec_hash": "sha256:NEEDS_VERIFICATION",
 "scenarios": [
 {
 "name": "cache-heavy-layout-slim",
 "style_mode": "layout-first",
 "tile_set": "slim",
 "avg_frame_ms": 12.4,
 "p95_frame_ms": 28.7,
 "idle_ms": 1840,
 "load_ms": 920,
 "passed": true
 }
]
}

```

## CI hook

```
- name: Validate MapLibre perf receipt
 run: |
 python3 tools/validators/maplibre/validate_perf_receipt.py \
 artifacts/perf/run-receipt.json
```

This turns the smoke test from “numbers in CI” into a governed, fail-closed KFM receipt.

Next piece: **generate the `RunReceipt` directly from the MapLibre smoke harness.**

## Add this to the end of `scripts/maplibre-smoke-perf.mjs`

```
import crypto from "node:crypto";

function sha256Json(value) {
 return (
 "sha256:" +
 crypto
 .createHash("sha256")
 .update(JSON.stringify(value))
 .digest("hex")
);
}

function p95(values) {
 if (!values.length) return 0;
 const sorted = [...values].sort((a, b) => a - b);
 return sorted[Math.floor(sorted.length * 0.95)];
}

const thresholds = {
 avg_frame_ms: 22,
 p95_frame_ms: 40,
 idle_ms: 3000,
 load_ms: 4000
};

const receiptScenarios = results.map((row) => {
 const passed =
 row.avgFrameMs < thresholds.avg_frame_ms &&
 row.p95FrameMs < thresholds.p95_frame_ms &&
```

```

 row.idleMs < thresholds.idle_ms &&
 row.loadMs < thresholds.load_ms;

 return {
 name: row.scenario,
 style_mode: row.styleMode ?? "NEEDS_VERIFICATION",
 tile_set: row.tiles ?? "NEEDS_VERIFICATION",
 avg_frame_ms: row.avgFrameMs,
 p95_frame_ms: row.p95FrameMs,
 idle_ms: row.idleMs,
 load_ms: row.loadMs,
 passed
 };
 });

const runReceipt = {
 object_type: "RunReceipt",
 schema_version: "v1",
 run_id: `maplibre-perf-${new Date().toISOString()}`,
 created_utc: new Date().toISOString(),
 domain: "maplibre",
 status: receiptScenarios.every((s) => s.passed)
 ? "completed"
 : "failed",
 policy_posture: "public_safe",
 spec_hash: sha256Json({
 scenarios,
 thresholds
 }),
 scenarios: receiptScenarios
};

await fs.mkdir("artifacts/perf", { recursive: true });

await fs.writeFile(
 "artifacts/perf/run-receipt.json",
 JSON.stringify(runReceipt, null, 2)
);

await fs.writeFile(
 "artifacts/perf/perf-results.json",
 JSON.stringify(results, null, 2)
);

```



```
console.log("\nWrote:");
console.log("- artifacts/perf/run-receipt.json");
console.log("- artifacts/perf/perf-results.json");
```

## Patch the metrics object to include p95FrameMs

Inside the `page.evaluate` return block, replace:

```
return {
 loadMs: Math.round(loadMs),
 idleMs: Math.round(idleMs),
 frames: frameTimes.length,
 avgFrameMs: Number(avgFrameMs.toFixed(2))
};
```

with:

```
const sortedFrames = [...frameTimes].sort((a, b) => a - b);
const p95FrameMs =
 sortedFrames[Math.floor(sortedFrames.length * 0.95)] ?? 0;

return {
 loadMs: Math.round(loadMs),
 idleMs: Math.round(idleMs),
 frames: frameTimes.length,
 avgFrameMs: Number(avgFrameMs.toFixed(2)),
 p95FrameMs: Number(p95FrameMs.toFixed(2))
};
```

## Patch scenario result collection

Replace:

```
results.push({
 scenario: scenario.name,
 ...metrics
});
```

with:

```
results.push({
 scenario: scenario.name,
 styleMode: scenario.styleMode,
 tiles: scenario.tiles,
 ...metrics
});
```

Now the smoke run emits both raw perf data and a governed KFM receipt that your validator can fail closed on.

Next piece: **DSSE-ready attestation envelope for the MapLibre perf receipt.**

```
artifacts/perf/
├── run-receipt.json
├── run-receipt.dsse.json
└── run-receipt.sha256
```

## scripts/attest-maplibre-perf.mjs

```
#!/usr/bin/env node
```

```
import crypto from "node:crypto";
import fs from "node:fs/promises";
```

```
const receiptPath =
 process.argv[2] ?? "artifacts/perf/run-receipt.json";
```

```
const outPath =
 process.argv[3] ?? "artifacts/perf/run-receipt.dsse.json";
```

```
function sha256(bytes) {
 return crypto.createHash("sha256").update(bytes).digest("hex");
}
```

```
function b64(bytes) {
 return Buffer.from(bytes).toString("base64");
}
```

```
const receiptBytes = await fs.readFile(receiptPath);
const receipt = JSON.parse(receiptBytes.toString("utf8"));
```

```
const payloadType =
```

```

"application/vnd.kfm.maplibre.perf.run-receipt.v1+json";

const envelope = {
 payloadType,
 payload: b64(receiptBytes),
 signatures: [],
 kfm_attestation: {
 object_type: "AttestationEnvelope",
 schema_version: "v1",
 subject_type: "RunReceipt",
 subject_path: receiptPath,
 subject_sha256: `sha256:${sha256(receiptBytes)}`,
 domain: "maplibre",
 policy_posture: receipt.policy_posture ?? "review_required",
 created_utc: new Date().toISOString(),
 signing_status: "unsigned",
 notes: [
 "DSSE-shaped unsigned envelope for CI review.",
 "Cosign signing may be applied in release-capable environments."
]
 }
};

await fs.mkdir("artifacts/perf", { recursive: true });

await fs.writeFile(outPath, JSON.stringify(envelope, null, 2));

await fs.writeFile(
 "artifacts/perf/run-receipt.sha256",
 `${sha256(receiptBytes)} ${receiptPath}\n`
);

console.log("ANSWER: wrote unsigned DSSE-shaped envelope");
console.log(`- ${outPath}`);
console.log("- artifacts/perf/run-receipt.sha256");

```

## CI hook

```

- name: Build MapLibre perf attestation envelope
 run: |
 node scripts/attest-maplibre-perf.mjs \
 artifacts/perf/run-receipt.json \
 artifacts/perf/run-receipt.dsse.json

```

## Optional Cosign signing step

```
- name: Sign MapLibre perf receipt with Cosign
 if: github.ref == 'refs/heads/main'
 env:
 COSIGN_EXPERIMENTAL: "1"
 run: |
 cosign sign-blob \
 --yes \
 --bundle artifacts/perf/run-receipt.cosign.bundle \
 artifacts/perf/run-receipt.json
```

## Optional Cosign verification step

```
- name: Verify MapLibre perf receipt signature
 if: github.ref == 'refs/heads/main'
 run: |
 cosign verify-blob \
 --bundle artifacts/perf/run-receipt.cosign.bundle \
 artifacts/perf/run-receipt.json
```

## Add to CI sequence

```
- name: Run MapLibre smoke perf
 run: node scripts/maplibre-smoke-perf.mjs

- name: Validate MapLibre perf receipt
 run: |
 python3 tools/validators/maplibre/validate_perf_receipt.py \
 artifacts/perf/run-receipt.json

- name: Attest MapLibre perf receipt
 run: |
 node scripts/attest-maplibre-perf.mjs
```

This gives you a clean chain:

MapLibre smoke run



perf-results.json  
↓  
run-receipt.json  
↓  
validator gate  
↓  
DSSE-shaped envelope  
↓  
optional Cosign bundle

Next piece: **release manifest wiring for MapLibre perf artifacts.**

artifacts/perf/  
├── release-manifest.maplibre-perf.json  
├── run-receipt.json  
├── run-receipt.dsse.json  
├── run-receipt.sha256  
└── perf-results.json

## schemas/maplibre/perf-release-manifest.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/perf-release-manifest/v1",
 "title": "MapLibre Perf Release Manifest",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "release_id",
 "created_utc",
 "domain",
 "status",
 "policy_posture",
 "artifacts"
],
 "additionalProperties": false,
 "properties": {
 "object_type": { "const": "ReleaseManifest" },
 "schema_version": { "const": "v1" },
```

```

"release_id": { "type": "string", "minLength": 8 },
"created_utc": { "type": "string", "format": "date-time" },
"domain": { "const": "maplibre" },
"status": {
 "enum": ["candidate", "released", "rejected"]
},
"policy_posture": {
 "enum": ["public_safe", "restricted", "review_required"]
},
"artifacts": {
 "type": "array",
 "minItems": 1,
 "items": {
 "type": "object",
 "required": [
 "name",
 "path",
 "sha256",
 "role"
],
 "additionalProperties": false,
 "properties": {
 "name": { "type": "string" },
 "path": { "type": "string" },
 "sha256": {
 "type": "string",
 "pattern": "^sha256:[a-f0-9]{64}$"
 },
 "role": {
 "enum": [
 "run_receipt",
 "perf_results",
 "attestation",
 "checksum",
 "screenshot",
 "render_diff"
]
 }
 }
 }
}
}
}
}
}
}
}

```

## scripts/build-maplibre-perf-release-manifest.mjs

```
#!/usr/bin/env node
```

```
import crypto from "node:crypto";
import fs from "node:fs/promises";
import path from "node:path";
```

```
const artifacts = [
 {
 name: "run-receipt",
 path: "artifacts/perf/run-receipt.json",
 role: "run_receipt"
 },
 {
 name: "perf-results",
 path: "artifacts/perf/perf-results.json",
 role: "perf_results"
 },
 {
 name: "run-receipt-dsse",
 path: "artifacts/perf/run-receipt.dsse.json",
 role: "attestation"
 },
 {
 name: "run-receipt-sha256",
 path: "artifacts/perf/run-receipt.sha256",
 role: "checksum"
 }
];
```

```
function sha256(bytes) {
 return (
 "sha256:" +
 crypto.createHash("sha256").update(bytes).digest("hex")
);
}
```

```
const included = [];
```

```
for (const artifact of artifacts) {
 const bytes = await fs.readFile(artifact.path);
```

```

 included.push({
 ...artifact,
 sha256: sha256(bytes)
 });
 }

const receipt = JSON.parse(
 await fs.readFile("artifacts/perf/run-receipt.json", "utf8")
);

const manifest = {
 object_type: "ReleaseManifest",
 schema_version: "v1",
 release_id: `maplibre-perf-release-${new Date()
 .toISOString()
 .replaceAll(":", "-")}`,
 created_utc: new Date().toISOString(),
 domain: "maplibre",
 status: receipt.status === "completed" ? "candidate" : "rejected",
 policy_posture: receipt.policy_posture ?? "review_required",
 artifacts: included
};

await fs.mkdir("artifacts/perf", { recursive: true });

await fs.writeFile(
 "artifacts/perf/release-manifest.maplibre-perf.json",
 JSON.stringify(manifest, null, 2)
);

console.log("ANSWER: wrote MapLibre perf release manifest");
console.log("- artifacts/perf/release-manifest.maplibre-perf.json");

```

## CI sequence

- name: Run MapLibre smoke perf  
run: node scripts/maplibre-smoke-perf.mjs
- name: Validate MapLibre perf receipt  
run: |
 

```
python3 tools/validators/maplibre/validate_perf_receipt.py \
 artifacts/perf/run-receipt.json
```



- name: Attest MapLibre perf receipt  
run: node scripts/attest-maplibre-perf.mjs
- name: Build MapLibre perf release manifest  
run: node scripts/build-maplibre-perf-release-manifest.mjs

This turns the chain into:

```
perf run
↓
RunReceipt
↓
DSSE-shaped envelope
↓
ReleaseManifest candidate/rejected
```

Next piece: **validator for the MapLibre perf ReleaseManifest.**

```
tools/validators/maplibre/
├── validate_perf_receipt.py
└── validate_perf_release_manifest.py
```

## tools/validators/maplibre/validate\_perf\_release\_manifest.py

```
#!/usr/bin/env python3
```

```
import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator, FormatChecker
```

```
SCHEMA_PATH = Path("schemas/maplibre/perf-release-manifest.schema.json")
```

```
REQUIRED_ROLES = {
 "run_receipt",
 "perf_results",
 "attestation",
 "checksum"
}
```

```

def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: validate_perf_release_manifest.py <manifest.json>",
 file=sys.stderr
)
 return 2

 manifest_path = Path(sys.argv[1])

 schema = json.loads(SCHEMA_PATH.read_text())
 manifest = json.loads(manifest_path.read_text())

 validator = Draft202012Validator(
 schema,
 format_checker=FormatChecker()
)

 errors = sorted(
 validator.iter_errors(manifest),
 key=lambda e: e.path
)

 if errors:
 print("DENY: invalid MapLibre perf release manifest")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

 roles = {a["role"] for a in manifest["artifacts"]}
 missing = REQUIRED_ROLES - roles

 if missing:
 print("DENY: release manifest missing required artifact roles")
 for role in sorted(missing):
 print(f"- {role}")
 return 1

 if manifest["status"] != "candidate":
 print(
 f"DENY: release manifest is not candidate "
 f"(status={manifest['status']})"
)

```

```

 return 1

 for artifact in manifest["artifacts"]:
 path = Path(artifact["path"])
 if not path.exists():
 print(f"DENY: artifact path does not exist: {path}")
 return 1

 print("ANSWER: MapLibre perf release manifest valid")
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

Fix this line before committing:

```
path = Path(artifact["path"])
```

Correct version:

```
path = Path(artifact["path"])
```

## Final corrected file

```

#!/usr/bin/env python3

import json
import sys
from pathlib import Path
from jsonschema import Draft2012Validator, FormatChecker

SCHEMA_PATH = Path("schemas/maplibre/perf-release-manifest.schema.json")

REQUIRED_ROLES = {
 "run_receipt",
 "perf_results",
 "attestation",
 "checksum"
}

def main() -> int:
 if len(sys.argv) != 2:

```

```

print(
 "usage: validate_perf_release_manifest.py <manifest.json>",
 file=sys.stderr
)
return 2

manifest_path = Path(sys.argv[1])

schema = json.loads(SCHEMA_PATH.read_text())
manifest = json.loads(manifest_path.read_text())

validator = Draft2012Validator(
 schema,
 format_checker=FormatChecker()
)

errors = sorted(
 validator.iter_errors(manifest),
 key=lambda e: e.path
)

if errors:
 print("DENY: invalid MapLibre perf release manifest")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

roles = {a["role"] for a in manifest["artifacts"]}
missing = REQUIRED_ROLES - roles

if missing:
 print("DENY: release manifest missing required artifact roles")
 for role in sorted(missing):
 print(f"- {role}")
 return 1

if manifest["status"] != "candidate":
 print(
 f"DENY: release manifest is not candidate "
 f"(status={manifest['status']})"
)
 return 1

```

```

for artifact in manifest["artifacts"]:
 artifact_path = Path(artifact["path"])

 if not artifact_path.exists():
 print(f"DENY: artifact path does not exist: {artifact_path}")
 return 1

print("ANSWER: MapLibre perf release manifest valid")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI addition

- name: Validate MapLibre perf release manifest

run: |

```
python3 tools/validators/maplibre/validate_perf_release_manifest.py \
 artifacts/perf/release-manifest.maplibre-perf.json
```

Now the chain is gated at both levels:

RunReceipt valid

↓

Attestation generated

↓

ReleaseManifest built

↓

ReleaseManifest valid

Next piece: **artifact integrity verification against manifest SHA256s** (fail-closed tamper detection).

tools/validators/maplibre/

```

├── validate_perf_receipt.py
├── validate_perf_release_manifest.py
└── verify_perf_artifact_hashes.py

```

## tools/validators/maplibre/verify\_perf\_artifact\_hashes.py

```
#!/usr/bin/env python3

import hashlib
import json
import sys
from pathlib import Path

def sha256_file(path: Path) -> str:
 h = hashlib.sha256()

 with path.open("rb") as f:
 for chunk in iter(lambda: f.read(65536), b''):
 h.update(chunk)

 return "sha256:" + h.hexdigest()

def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: verify_perf_artifact_hashes.py <manifest.json>",
 file=sys.stderr
)
 return 2

 manifest_path = Path(sys.argv[1])

 if not manifest_path.exists():
 print(f"DENY: manifest not found: {manifest_path}")
 return 1

 manifest = json.loads(manifest_path.read_text())

 failures = []

 for artifact in manifest.get("artifacts", []):
 artifact_path = Path(artifact["path"])

 if not artifact_path.exists():
 failures.append(
 f"missing artifact: {artifact_path}"
)
```

```

)
 continue

 expected = artifact["sha256"]
 actual = sha256_file(artifact_path)

 if actual != expected:
 failures.append(
 f"hash mismatch: {artifact_path}\n"
 f" expected={expected}\n"
 f" actual={actual}"
)

 if failures:
 print("DENY: artifact integrity verification failed")

 for failure in failures:
 print(f"- {failure}")

 return 1

print("ANSWER: all MapLibre perf artifacts verified")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI addition

- name: Verify MapLibre perf artifact hashes

run: |

```
python3 tools/validators/maplibre/verify_perf_artifact_hashes.py \
 artifacts/perf/release-manifest.maplibre-perf.json
```

## Full governed chain now

MapLibre smoke run

↓

perf-results.json

↓

RunReceipt

↓

RunReceipt validation  
↓  
DSSE-shaped envelope  
↓  
ReleaseManifest  
↓  
ReleaseManifest validation  
↓  
Artifact SHA verification  
↓  
Candidate release accepted

## Important negative-path tests to add next

fixtures/

- └─ invalid.hash-mismatch.manifest.json
- └─ invalid.missing-artifact.manifest.json
- └─ invalid.bad-role.manifest.json
- └─ invalid.rejected-status.manifest.json

Good fail-closed checks:

Failure	Expected
artifact deleted	DENY
checksum mismatch	DENY
missing attestation	DENY
rejected manifest	DENY
malformed SHA	DENY
duplicate artifact role	DENY
invalid schema_version	DENY

At this point the MapLibre perf lane is starting to resemble a real governed KFM publication/control surface instead of a simple renderer benchmark.

Next piece: **negative-path tests for the MapLibre perf validators.**



```
tests/maplibre/
└── test_perf_governance_negative_paths.py
```

## tests/maplibre/test\_perf\_governance\_negative\_paths.py

```
import json
import shutil
import subprocess
from pathlib import Path

ROOT = Path(__file__).resolve().parents[2]

MANIFEST_VALIDATOR = ROOT /
"tools/validators/maplibre/validate_perf_release_manifest.py"
HASH_VALIDATOR = ROOT / "tools/validators/maplibre/verify_perf_artifact_hashes.py"

BASE = {
 "object_type": "ReleaseManifest",
 "schema_version": "v1",
 "release_id": "maplibre-perf-release-test",
 "created_utc": "2026-05-14T16:00:00Z",
 "domain": "maplibre",
 "status": "candidate",
 "policy_posture": "public_safe",
 "artifacts": []
}

def run(cmd, cwd):
 return subprocess.run(
 cmd,
 cwd=cwd,
 text=True,
 capture_output=True
)

def write_artifact(path: Path, text: str):
 path.parent.mkdir(parents=True, exist_ok=True)
 path.write_text(text)

def sha256(path: Path) -> str:
 import hashlib
```

```

return "sha256:" + hashlib.sha256(path.read_bytes()).hexdigest()

def make_valid_workspace(tmp_path: Path) -> Path:
 perf = tmp_path / "artifacts/perf"
 perf.mkdir(parents=True)

 files = {
 "run-receipt.json": '{"object_type":"RunReceipt"}',
 "perf-results.json": "[]",
 "run-receipt.dsse.json": '{"payloadType":"test"}',
 "run-receipt.sha256": "test"
 }

 for name, body in files.items():
 write_artifact(perf / name, body)

 artifacts = [
 ("run-receipt", "run-receipt.json", "run_receipt"),
 ("perf-results", "perf-results.json", "perf_results"),
 ("run-receipt-dsse", "run-receipt.dsse.json", "attestation"),
 ("run-receipt-sha256", "run-receipt.sha256", "checksum")
]

 manifest = dict(BASE)
 manifest["artifacts"] = [
 {
 "name": name,
 "path": f"artifacts/perf/{filename}",
 "sha256": sha256(perf / filename),
 "role": role
 }
 for name, filename, role in artifacts
]

 manifest_path = perf / "release-manifest.maplibre-perf.json"
 manifest_path.write_text(json.dumps(manifest, indent=2))

 return manifest_path

def copy_schema_tree(tmp_path: Path):
 src = ROOT / "schemas"
 dst = tmp_path / "schemas"
 shutil.copytree(src, dst)

```

```

def copy_tools_tree(tmp_path: Path):
 src = ROOT / "tools"
 dst = tmp_path / "tools"
 shutil.copytree(src, dst)

def test_manifest_rejected_status_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 manifest_path = make_valid_workspace(tmp_path)
 manifest = json.loads(manifest_path.read_text())
 manifest["status"] = "rejected"
 manifest_path.write_text(json.dumps(manifest, indent=2))

 result = run(
 ["python3", str(MANIFEST_VALIDATOR), str(manifest_path)],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "not candidate" in result.stdout

def test_manifest_missing_required_role_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 manifest_path = make_valid_workspace(tmp_path)
 manifest = json.loads(manifest_path.read_text())
 manifest["artifacts"] = [
 a for a in manifest["artifacts"]
 if a["role"] != "attestation"
]
 manifest_path.write_text(json.dumps(manifest, indent=2))

 result = run(
 ["python3", str(MANIFEST_VALIDATOR), str(manifest_path)],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "attestation" in result.stdout

```

```

def test_missing_artifact_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 manifest_path = make_valid_workspace(tmp_path)
 missing = tmp_path / "artifacts/perf/perf-results.json"
 missing.unlink()

 result = run(
 ["python3", str(HASH_VALIDATOR), str(manifest_path)],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "missing artifact" in result.stdout

def test_hash_mismatch_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 manifest_path = make_valid_workspace(tmp_path)

 tampered = tmp_path / "artifacts/perf/run-receipt.json"
 tampered.write_text('{"object_type":"RunReceipt","tampered":true}')

 result = run(
 ["python3", str(HASH_VALIDATOR), str(manifest_path)],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "hash mismatch" in result.stdout

```

## CI addition

- name: Run MapLibre governance tests

run: |

pytest tests/maplibre/test\_perf\_governance\_negative\_paths.py

## Add positive path too

```
def test_valid_manifest_and_hashes_pass(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 manifest_path = make_valid_workspace(tmp_path)

 manifest_result = run(
 ["python3", str(MANIFEST_VALIDATOR), str(manifest_path)],
 cwd=tmp_path
)

 hash_result = run(
 ["python3", str(HASH_VALIDATOR), str(manifest_path)],
 cwd=tmp_path
)

 assert manifest_result.returncode == 0
 assert "ANSWER" in manifest_result.stdout

 assert hash_result.returncode == 0
 assert "ANSWER" in hash_result.stdout
```

Now you have the first serious negative-path safety net:

valid candidate	→ ANSWER
rejected manifest	→ DENY
missing attestation	→ DENY
deleted artifact	→ DENY
tampered artifact	→ DENY

Next is **duplicate-role and subject-link validation**.

Right now the manifest checks that required roles exist, but it does **not** prevent this:

- two run\_receipt artifacts
- missing real link between DSSE envelope and RunReceipt
- checksum file not matching the receipt path

Build this next:

tools/validators/maplibre/

```
|— validate_perf_release_manifest.py # upgrade
|— verify_perf_artifact_hashes.py
|— verify_perf_subject_links.py # new
```

Core checks:

Check	Expected
duplicate artifact role	DENY
DSSE <code>subject_sha256</code> != RunReceipt hash	DENY
DSSE <code>subject_path</code> != RunReceipt path	DENY
<code>.sha256</code> file does not name RunReceipt	DENY
<code>.sha256</code> digest does not match RunReceipt	DENY

The next file to add is:

tools/validators/maplibre/verify\_perf\_subject\_links.py

Then add negative tests:

```
invalid.duplicate-role.manifest.json
invalid.dsse-subject-hash-mismatch.json
invalid.dsse-subject-path-mismatch.json
invalid.bad-checksum-file.json
```

After that, the chain becomes much stronger:

Manifest shape

↓

Required artifacts

↓

No duplicate roles

↓

Artifact hashes

↓

DSSE subject binds to RunReceipt

↓

Checksum binds to RunReceipt

This closes the biggest integrity gap before moving on to **render screenshots and visual diff artifacts**.

Now add **subject-link verification**.

## tools/validators/maplibre/verify\_perf\_subject\_links.py

```
#!/usr/bin/env python3
```

```
import hashlib
import json
import sys
from pathlib import Path
```

```
REQUIRED_SINGLE_ROLES = {
 "run_receipt",
 "attestation",
 "checksum"
}
```

```
def sha256_file(path: Path) -> str:
 h = hashlib.sha256()
 with path.open("rb") as f:
 for chunk in iter(lambda: f.read(65536), b''):
 h.update(chunk)
 return "sha256:" + h.hexdigest()
```

```
def artifact_by_role(manifest: dict, role: str) -> dict:
 matches = [a for a in manifest["artifacts"] if a["role"] == role]
```

```
 if len(matches) == 0:
 raise ValueError(f"missing artifact role: {role}")
```

```
 if len(matches) > 1:
 raise ValueError(f"duplicate artifact role: {role}")
```

```
 return matches[0]
```

```

def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: verify_perf_subject_links.py <manifest.json>",
 file=sys.stderr
)
 return 2

 manifest_path = Path(sys.argv[1])

 if not manifest_path.exists():
 print(f"DENY: manifest not found: {manifest_path}")
 return 1

 manifest = json.loads(manifest_path.read_text())

 try:
 for role in REQUIRED_SINGLE_ROLES:
 artifact_by_role(manifest, role)

 run_receipt = artifact_by_role(manifest, "run_receipt")
 attestation = artifact_by_role(manifest, "attestation")
 checksum = artifact_by_role(manifest, "checksum")

 except ValueError as e:
 print(f"DENY: {e}")
 return 1

 run_receipt_path = Path(run_receipt["path"])
 attestation_path = Path(attestation["path"])
 checksum_path = Path(checksum["path"])

 if not run_receipt_path.exists():
 print(f"DENY: run receipt missing: {run_receipt_path}")
 return 1

 if not attestation_path.exists():
 print(f"DENY: attestation missing: {attestation_path}")
 return 1

 if not checksum_path.exists():
 print(f"DENY: checksum missing: {checksum_path}")
 return 1

```



```

receipt_hash = sha256_file(run_receipt_path)

dsse = json.loads(attestation_path.read_text())
kfm_attestation = dsse.get("kfm_attestation", {})

dsse_subject_path = kfm_attestation.get("subject_path")
dsse_subject_hash = kfm_attestation.get("subject_sha256")

if dsse_subject_path != run_receipt["path"]:
 print("DENY: DSSE subject_path does not match run_receipt artifact")
 print(f"- expected={run_receipt['path']}")
 print(f"- actual={dsse_subject_path}")
 return 1

if dsse_subject_hash != receipt_hash:
 print("DENY: DSSE subject_sha256 does not match RunReceipt hash")
 print(f"- expected={receipt_hash}")
 print(f"- actual={dsse_subject_hash}")
 return 1

checksum_text = checksum_path.read_text().strip()
expected_checksum_line = f"{receipt_hash.removeprefix('sha256:')} {run_receipt['path']}"

if checksum_text != expected_checksum_line:
 print("DENY: checksum file does not bind to RunReceipt")
 print(f"- expected={expected_checksum_line}")
 print(f"- actual={checksum_text}")
 return 1

print("ANSWER: MapLibre perf subject links verified")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## Add to CI

- name: Verify MapLibre perf subject links

run: |

```
python3 tools/validators/maplibre/verify_perf_subject_links.py \
 artifacts/perf/release-manifest.maplibre-perf.json
```

## Full order now

- name: Run MapLibre smoke perf  
run: node scripts/maplibre-smoke-perf.mjs
- name: Validate MapLibre perf receipt  
run: |  
python3 tools/validators/maplibre/validate\_perf\_receipt.py \  
artifacts/perf/run-receipt.json
- name: Attest MapLibre perf receipt  
run: node scripts/attest-maplibre-perf.mjs
- name: Build MapLibre perf release manifest  
run: node scripts/build-maplibre-perf-release-manifest.mjs
- name: Validate MapLibre perf release manifest  
run: |  
python3 tools/validators/maplibre/validate\_perf\_release\_manifest.py \  
artifacts/perf/release-manifest.maplibre-perf.json
- name: Verify MapLibre perf artifact hashes  
run: |  
python3 tools/validators/maplibre/verify\_perf\_artifact\_hashes.py \  
artifacts/perf/release-manifest.maplibre-perf.json
- name: Verify MapLibre perf subject links  
run: |  
python3 tools/validators/maplibre/verify\_perf\_subject\_links.py \  
artifacts/perf/release-manifest.maplibre-perf.json

Next after this: **screenshots + render-diff artifacts** so perf regressions and visual regressions are governed together.

Proceed with **screenshots + render-diff artifacts**.

## Add artifact roles to schema

Update `schemas/maplibre/perf-release-manifest.schema.json` artifact role enum:

```
"role": {
 "enum": [
 "run_receipt",
```

```

 "perf_results",
 "attestation",
 "checksum",
 "screenshot",
 "render_diff"
]
}

```

## Add screenshot capture to `scripts/maplibre-smoke-perf.mjs`

Inside the scenario loop, after metrics are collected and before `page.close()`:

```
await fs.mkdir("artifacts/perf/screenshots", { recursive: true });
```

```
const screenshotPath =
 `artifacts/perf/screenshots/${scenario.name}.png`;
```

```
await page.screenshot({
 path: screenshotPath,
 fullPage: true
});
```

```
results[results.length - 1].screenshotPath = screenshotPath;
```

Place it after:

```
results.push({
 scenario: scenario.name,
 styleMode: scenario.styleMode,
 tiles: scenario.tiles,
 ...metrics
});
```

## Add screenshots to release manifest builder

In `scripts/build-maplibre-perf-release-manifest.mjs`, after the base `artifacts` list:

```
const results = JSON.parse(
```

```

 await fs.readFile("artifacts/perf/perf-results.json", "utf8")
);

 for (const row of results) {
 if (row.screenshotPath) {
 artifacts.push({
 name: `screenshot-${row.scenario}`,
 path: row.screenshotPath,
 role: "screenshot"
 });
 }
 }
}

```

## Simple render-diff placeholder

Add this script:

scripts/build-maplibre-render-diff.mjs

```
#!/usr/bin/env node
```

```
import fs from "node:fs/promises";
```

```
await fs.mkdir("artifacts/perf/render-diff", { recursive: true });
```

```

const diff = {
 object_type: "RenderDiffReport",
 schema_version: "v1",
 domain: "maplibre",
 created_utc: new Date().toISOString(),
 status: "baseline_only",
 pixel_delta_ratio: 0,
 threshold: 0.01,
 passed: true,
 notes: [
 "Baseline placeholder until golden screenshots are committed.",
 "Replace with pixelmatch-based comparison when baselines exist."
]
};

```

```

await fs.writeFile(
 "artifacts/perf/render-diff/render-diff-report.json",
 JSON.stringify(diff, null, 2)
)

```

```
);
```

```
console.log("ANSWER: wrote render diff placeholder");
```

## Add render diff to manifest

Add this artifact:

```
artifacts.push({
 name: "render-diff-report",
 path: "artifacts/perf/render-diff/render-diff-report.json",
 role: "render_diff"
});
```

## CI order update

- name: Run MapLibre smoke perf  
run: node scripts/maplibre-smoke-perf.mjs
- name: Build MapLibre render diff report  
run: node scripts/build-maplibre-render-diff.mjs
- name: Validate MapLibre perf receipt  
run: |  
python3 tools/validators/maplibre/validate\_perf\_receipt.py \  
artifacts/perf/run-receipt.json
- name: Attest MapLibre perf receipt  
run: node scripts/attest-maplibre-perf.mjs
- name: Build MapLibre perf release manifest  
run: node scripts/build-maplibre-perf-release-manifest.mjs

Now the governed artifact chain includes:

```
performance metrics
+ run receipt
+ attestation
+ manifest
+ screenshot evidence
+ render-diff report
```

Next: replace the placeholder render diff with **real pixelmatch visual regression testing**.

## Install

```
npm i -D pixelmatch pngjs
```

## scripts/build-maplibre-render-diff.mjs

```
#!/usr/bin/env node
```

```
import fs from "node:fs/promises";
import path from "node:path";
import pixelmatch from "pixelmatch";
import { PNG } from "pngjs";

const screenshotsDir = "artifacts/perf/screenshots";
const baselineDir = "tests/fixtures/maplibre/baselines";
const diffDir = "artifacts/perf/render-diff";

const threshold = 0.01;

await fs.mkdir(diffDir, { recursive: true });

const screenshots = await fs.readdir(screenshotsDir);

const reports = [];

for (const file of screenshots.filter((f) => f.endsWith(".png"))) {
 const screenshotPath = path.join(screenshotsDir, file);
 const baselinePath = path.join(baselineDir, file);
 const diffPath = path.join(diffDir, file.replace(".png", ".diff.png"));

 let status = "compared";
 let passed = true;
 let pixelDeltaRatio = 0;
 let notes = [];

 try {
 const actual = PNG.sync.read(await fs.readFile(screenshotPath));
 const baseline = PNG.sync.read(await fs.readFile(baselinePath));

 if (
 actual.width !== baseline.width ||
```

```

 actual.height !== baseline.height
) {
 status = "dimension_mismatch";
 passed = false;
 notes.push("Screenshot dimensions do not match baseline.");
 } else {
 const diff = new PNG({
 width: actual.width,
 height: actual.height
 });

 const changedPixels = pixelmatch(
 actual.data,
 baseline.data,
 diff.data,
 actual.width,
 actual.height,
 { threshold: 0.1 }
);

 pixelDeltaRatio =
 changedPixels / (actual.width * actual.height);

 passed = pixelDeltaRatio <= threshold;

 await fs.writeFile(diffPath, PNG.sync.write(diff));
 }
} catch (err) {
 status = "missing_baseline";
 passed = false;
 notes.push(String(err.message ?? err));
}

reports.push({
 scenario: file.replace(".png", ""),
 screenshot_path: screenshotPath,
 baseline_path: baselinePath,
 diff_path: diffPath,
 status,
 pixel_delta_ratio: Number(pixelDeltaRatio.toFixed(6)),
 threshold,
 passed,
 notes
});

```

```

}

const report = {
 object_type: "RenderDiffReport",
 schema_version: "v1",
 domain: "maplibre",
 created_utc: new Date().toISOString(),
 status: reports.every((r) => r.passed) ? "passed" : "failed",
 threshold,
 scenarios: reports
};

await fs.writeFile(
 path.join(diffDir, "render-diff-report.json"),
 JSON.stringify(report, null, 2)
);

if (report.status !== "passed") {
 console.error("DENY: render diff failed");
 console.error(JSON.stringify(report, null, 2));
 process.exit(1);
}

console.log("ANSWER: render diff passed");

```

## Add baseline workflow

```

mkdir -p tests/fixtures/maplibre/baselines
cp artifacts/perf/screenshots/*.png tests/fixtures/maplibre/baselines/

```

## Add negative fixtures next

```

tests/fixtures/maplibre/
├── baselines/
│ ├── cache-heavy-layout-slim.png
│ └── cache-light-heavy-paint-heavy.png
└── bad-baselines/
 ├── dimension-mismatch.png
 └── visual-delta.png

```



## CI gate

- name: Build MapLibre render diff report
- run: node scripts/build-maplibre-render-diff.mjs

Now screenshots are not just artifacts; they become governed visual evidence with a hard fail-closed threshold.

Now add **render-diff schema + validator**, so the visual report is governed like the receipt and manifest.

## schemas/maplibre/render-diff-report.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/render-diff-report/v1",
 "title": "MapLibre Render Diff Report",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "domain",
 "created_utc",
 "status",
 "threshold",
 "scenarios"
],
 "additionalProperties": false,
 "properties": {
 "object_type": { "const": "RenderDiffReport" },
 "schema_version": { "const": "v1" },
 "domain": { "const": "maplibre" },
 "created_utc": { "type": "string", "format": "date-time" },
 "status": { "enum": ["passed", "failed"] },
 "threshold": { "type": "number", "minimum": 0, "maximum": 1 },
 "scenarios": {
 "type": "array",
 "minItems": 1,
 "items": {
 "type": "object",
```

```

"required": [
 "scenario",
 "screenshot_path",
 "baseline_path",
 "diff_path",
 "status",
 "pixel_delta_ratio",
 "threshold",
 "passed",
 "notes"
],
"additionalProperties": false,
"properties": {
 "scenario": { "type": "string" },
 "screenshot_path": { "type": "string" },
 "baseline_path": { "type": "string" },
 "diff_path": { "type": "string" },
 "status": {
 "enum": [
 "compared",
 "missing_baseline",
 "dimension_mismatch"
]
 },
 "pixel_delta_ratio": {
 "type": "number",
 "minimum": 0,
 "maximum": 1
 },
 "threshold": {
 "type": "number",
 "minimum": 0,
 "maximum": 1
 },
 "passed": { "type": "boolean" },
 "notes": {
 "type": "array",
 "items": { "type": "string" }
 }
}
}
}
}
}
}

```

## tools/validators/maplibre/validate\_render\_diff\_report.py

```
#!/usr/bin/env python3
```

```
import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator, FormatChecker
```

```
SCHEMA_PATH = Path("schemas/maplibre/render-diff-report.schema.json")
```

```
def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: validate_render_diff_report.py <render-diff-report.json>",
 file=sys.stderr
)
 return 2
```

```
report_path = Path(sys.argv[1])
```

```
schema = json.loads(SCHEMA_PATH.read_text())
report = json.loads(report_path.read_text())
```

```
validator = Draft202012Validator(
 schema,
 format_checker=FormatChecker()
)
```

```
errors = sorted(
 validator.iter_errors(report),
 key=lambda e: e.path
)
```

```
if errors:
 print("DENY: invalid MapLibre render diff report")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1
```

```

if report["status"] != "passed":
 print(f'DENY: render diff report status={report["status"]}')
 return 1

failed = [s for s in report["scenarios"] if not s["passed"]]

if failed:
 print("DENY: one or more render diff scenarios failed")
 for scenario in failed:
 print(
 f'- {scenario["scenario"]}: '
 f'{scenario["status"]} '
 f'delta={scenario["pixel_delta_ratio"]}'
)
 return 1

print("ANSWER: MapLibre render diff report valid")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI addition

- name: Validate MapLibre render diff report

run: |

```
python3 tools/validators/maplibre/validate_render_diff_report.py \
artifacts/perf/render-diff/render-diff-report.json
```

Next best piece after this: **include the render-diff report in the RunReceipt scenarios**, so each perf scenario points to its screenshot and visual-diff evidence.

Proceed with **linking screenshot + render-diff evidence into the RunReceipt**.

## Update **perf-receipt.schema.json**

Add these required fields inside each scenario:

```
"screenshot_path",
"render_diff_report_path"
```

Add properties:

```
"screenshot_path": {
 "type": "string",
 "minLength": 1
},
"render_diff_report_path": {
 "type": "string",
 "minLength": 1
}
```

## Patch `maplibre-smoke-perf.mjs`

After screenshot capture:

```
results[results.length - 1].screenshotPath = screenshotPath;
results[results.length - 1].renderDiffReportPath =
 "artifacts/perf/render-diff/render-diff-report.json";
```

## Patch `receiptScenarios`

```
return {
 name: row.scenario,
 style_mode: row.styleMode ?? "NEEDS_VERIFICATION",
 tile_set: row.tiles ?? "NEEDS_VERIFICATION",
 avg_frame_ms: row.avgFrameMs,
 p95_frame_ms: row.p95FrameMs,
 idle_ms: row.idleMs,
 load_ms: row.loadMs,
 screenshot_path: row.screenshotPath,
 render_diff_report_path: row.renderDiffReportPath,
 passed
};
```

## Validator hardening

In `validate_perf_receipt.py`, after schema validation:

```
for scenario in receipt["scenarios"]:
 screenshot_path = Path(scenario["screenshot_path"])
```

```

render_diff_path = Path(scenario["render_diff_report_path"])

if not screenshot_path.exists():
 print(f"DENY: scenario screenshot missing: {screenshot_path}")
 return 1

if not render_diff_path.exists():
 print(f"DENY: scenario render diff report missing: {render_diff_path}")
 return 1

```

## CI ordering change

Because the receipt now points to render-diff evidence, build render-diff before validating receipt:

- name: Run MapLibre smoke perf  
run: node scripts/maplibre-smoke-perf.mjs
- name: Build MapLibre render diff report  
run: node scripts/build-maplibre-render-diff.mjs
- name: Validate MapLibre render diff report  
run: |  
python3 tools/validators/maplibre/validate\_render\_diff\_report.py \  
artifacts/perf/render-diff/render-diff-report.json
- name: Validate MapLibre perf receipt  
run: |  
python3 tools/validators/maplibre/validate\_perf\_receipt.py \  
artifacts/perf/run-receipt.json

This creates direct evidence linkage:

```

RunReceipt scenario
├─ perf metrics
├─ screenshot_path
└─ render_diff_report_path

```

Next: add **frame-times CSV + p95 source verification** so the receipt's p95 value is independently checkable.

Proceed with **frame-times CSV + independent p95 verification**.

## Add artifact

```
artifacts/perf/
├── frame-times/
│ ├── cache-heavy-layout-slim.csv
│ └── cache-light-heavy-paint-heavy.csv
```

## Patch **maplibre-smoke-perf.mjs**

Inside `page.evaluate`, return frame times too:

```
return {
 loadMs: Math.round(loadMs),
 idleMs: Math.round(idleMs),
 frames: frameTimes.length,
 avgFrameMs: Number(avgFrameMs.toFixed(2)),
 p95FrameMs: Number(p95FrameMs.toFixed(2)),
 frameTimes: frameTimes.map((v) => Number(v.toFixed(4)))
};
```

After `results.push(...)`, write the CSV:

```
await fs.mkdir("artifacts/perf/frame-times", { recursive: true });
```

```
const frameTimesPath =
 `artifacts/perf/frame-times/${scenario.name}.csv`;
```

```
const csv =
 "frame_index,frame_ms\n" +
 metrics.frameTimes
 .map((v, i) => `${i},${v}`)
 .join("\n") +
 "\n";
```

```
await fs.writeFile(frameTimesPath, csv);
```

```
results[results.length - 1].frameTimesPath = frameTimesPath;
delete results[results.length - 1].frameTimes;
```

## Add to receipt scenario

```
frame_times_path: row.frameTimesPath,
```

## Update schema scenario required fields

Add:

```
"frame_times_path"
```

Add property:

```
"frame_times_path": {
 "type": "string",
 "minLength": 1
}
```

## Add frame-times to manifest builder

```
for (const row of results) {
 if (row.frameTimesPath) {
 artifacts.push({
 name: `frame-times-${row.scenario}`,
 path: row.frameTimesPath,
 role: "frame_times"
 });
 }
}
```

Update manifest schema role enum:

```
"frame_times"
```

## tools/validators/maplibre/verify\_perf\_metrics \_from\_frames.py

```
#!/usr/bin/env python3
```

```
import csv
```



```

import json
import statistics
import sys
from pathlib import Path

TOLERANCE_MS = 0.05

def read_frame_ms(path: Path) -> list[float]:
 with path.open(newline="") as f:
 rows = csv.DictReader(f)
 return [float(row["frame_ms"]) for row in rows]

def p95(values: list[float]) -> float:
 ordered = sorted(values)
 return ordered[int(len(ordered) * 0.95)]

def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: verify_perf_metrics_from_frames.py <run-receipt.json>",
 file=sys.stderr
)
 return 2

 receipt_path = Path(sys.argv[1])
 receipt = json.loads(receipt_path.read_text())

 for scenario in receipt["scenarios"]:
 frame_path = Path(scenario["frame_times_path"])

 if not frame_path.exists():
 print(f"DENY: frame-times CSV missing: {frame_path}")
 return 1

 frames = read_frame_ms(frame_path)

 if not frames:
 print(f"DENY: no frame samples: {frame_path}")
 return 1

 avg_actual = round(statistics.mean(frames), 2)
 p95_actual = round(p95(frames), 2)

 if abs(avg_actual - scenario["avg_frame_ms"]) > TOLERANCE_MS:

```

```

 print("DENY: avg_frame_ms does not match frame-times CSV")
 print(f"- scenario={scenario['name']}")
 print(f"- receipt={scenario['avg_frame_ms']}")
 print(f"- actual={avg_actual}")
 return 1

 if abs(p95_actual - scenario["p95_frame_ms"]) > TOLERANCE_MS:
 print("DENY: p95_frame_ms does not match frame-times CSV")
 print(f"- scenario={scenario['name']}")
 print(f"- receipt={scenario['p95_frame_ms']}")
 print(f"- actual={p95_actual}")
 return 1

 print("ANSWER: frame-derived perf metrics verified")
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI addition

- name: Verify MapLibre frame-derived metrics  
 run: |  
 python3 tools/validators/maplibre/verify\_perf\_metrics\_from\_frames.py \
 artifacts/perf/run-receipt.json

Now the receipt values are independently reconstructable from raw frame evidence:

```

frame-times CSV
↓
avg + p95 recomputed
↓
RunReceipt values verified
↓
ReleaseManifest hash-locks both

```

Next: **threshold envelope as a governed config file**, so perf gates are not hardcoded in scripts.

## Add

config/maplibre/

└─ perf-envelope.v1.json

## config/maplibre/perf-envelope.v1.json

```
{
 "object_type": "PerfEnvelope",
 "schema_version": "v1",
 "domain": "maplibre",
 "policy_posture": "public_safe",
 "thresholds": {
 "avg_frame_ms": 22,
 "p95_frame_ms": 40,
 "idle_ms": 3000,
 "load_ms": 4000,
 "render_pixel_delta_ratio": 0.01
 },
 "notes": [
 "Early KFM renderer smoke envelope.",
 "Tighten thresholds only through reviewed config change."
]
}
```

## Patch maplibre-smoke-perf.mjs

Replace hardcoded thresholds:

```
const thresholds = {
 avg_frame_ms: 22,
 p95_frame_ms: 40,
 idle_ms: 3000,
 load_ms: 4000
};
```

with:

```
const perfEnvelope = JSON.parse(
 await fs.readFile("config/maplibre/perf-envelope.v1.json", "utf8")
);
```

```
const thresholds = perfEnvelope.thresholds;
```

## Add envelope reference to **RunReceipt**

```
perf_envelope_path: "config/maplibre/perf-envelope.v1.json",
perf_envelope_hash: sha256Json(perfEnvelope),
```

## Update receipt schema required fields

Add root-level required fields:

```
"perf_envelope_path",
"perf_envelope_hash"
```

Add properties:

```
"perf_envelope_path": {
 "type": "string",
 "minLength": 1
},
"perf_envelope_hash": {
 "type": "string",
 "pattern": "^sha256:[a-f0-9]{64}$"
}
```

## Add validator check

In `validate_perf_receipt.py`:

```
import hashlib
```

```
def sha256_json(value: dict) -> str:
 encoded = json.dumps(value, separators=(",", ":"), sort_keys=True).encode()
 return "sha256:" + hashlib.sha256(encoded).hexdigest()
```

Then after schema validation:

```
envelope_path = Path(receipt["perf_envelope_path"])

if not envelope_path.exists():
 print(f"DENY: perf envelope missing: {envelope_path}")
 return 1
```

```

envelope = json.loads(envelope_path.read_text())
actual_hash = sha256_json(envelope)

if actual_hash != receipt["perf_envelope_hash"]:
 print("DENY: perf envelope hash mismatch")
 print(f"- expected={receipt['perf_envelope_hash']}")
 print(f"- actual={actual_hash}")
 return 1

```

## Add manifest artifact role

"perf\_envelope"

Manifest builder:

```

artifacts.push({
 name: "perf-envelope",
 path: "config/maplibre/perf-envelope.v1.json",
 role: "perf_envelope"
});

```

Now the gate becomes reviewable:

PerfEnvelope config

↓

RunReceipt references + hashes envelope

↓

validators verify envelope integrity

↓

manifest hash-locks envelope with release artifacts

Next: **schema + validator for the PerfEnvelope itself.**

## Add

schemas/maplibre/

└─ perf-envelope.schema.json

tools/validators/maplibre/

└─ validate\_perf\_envelope.py

## schemas/maplibre/perf-envelope.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/perf-envelope/v1",
 "title": "MapLibre Performance Envelope",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "domain",
 "policy_posture",
 "thresholds",
 "notes"
],
 "additionalProperties": false,
 "properties": {
 "object_type": { "const": "PerfEnvelope" },
 "schema_version": { "const": "v1" },
 "domain": { "const": "maplibre" },
 "policy_posture": {
 "enum": ["public_safe", "restricted", "review_required"]
 },
 "thresholds": {
 "type": "object",
 "required": [
 "avg_frame_ms",
 "p95_frame_ms",
 "idle_ms",
 "load_ms",
 "render_pixel_delta_ratio"
],
 "additionalProperties": false,
 "properties": {
 "avg_frame_ms": {
 "type": "number",
 "exclusiveMinimum": 0,
 "maximum": 100
 },
 "p95_frame_ms": {
 "type": "number",
 "exclusiveMinimum": 0,
```

```

 "maximum": 200
 },
 "idle_ms": {
 "type": "number",
 "exclusiveMinimum": 0,
 "maximum": 30000
 },
 "load_ms": {
 "type": "number",
 "exclusiveMinimum": 0,
 "maximum": 30000
 },
 "render_pixel_delta_ratio": {
 "type": "number",
 "minimum": 0,
 "maximum": 1
 }
}
},
"notes": {
 "type": "array",
 "items": { "type": "string" }
}
}
}

```

## tools/validators/maplibre/validate\_perf\_envelope.py

```
#!/usr/bin/env python3
```

```

import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator

```

```
SCHEMA_PATH = Path("schemas/maplibre/perf-envelope.schema.json")
```

```

def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: validate_perf_envelope.py <perf-envelope.json>",

```

```

 file=sys.stderr
)
 return 2

envelope_path = Path(sys.argv[1])

if not envelope_path.exists():
 print(f"DENY: perf envelope not found: {envelope_path}")
 return 1

schema = json.loads(SCHEMA_PATH.read_text())
envelope = json.loads(envelope_path.read_text())

validator = Draft202012Validator(schema)
errors = sorted(validator.iter_errors(envelope), key=lambda e: e.path)

if errors:
 print("DENY: invalid MapLibre perf envelope")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

t = envelope["thresholds"]

if t["avg_frame_ms"] > t["p95_frame_ms"]:
 print("DENY: avg_frame_ms threshold cannot exceed p95_frame_ms")
 return 1

print("ANSWER: MapLibre perf envelope valid")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI order

- name: Validate MapLibre perf envelope

run: |

```
python3 tools/validators/maplibre/validate_perf_envelope.py \
 config/maplibre/perf-envelope.v1.json
```



Put it **before** the smoke run:

PerfEnvelope valid



Smoke run consumes envelope



RunReceipt records envelope hash



Receipt validator verifies same envelope

Next: add **threshold re-evaluation from receipt + envelope**, so even if the harness lies about **passed**, CI recomputes pass/fail independently.

Proceed with **independent threshold re-evaluation** so CI does not trust **passed** from the harness.

## Add

tools/validators/maplibre/  
└─ verify\_perf\_thresholds.py

## tools/validators/maplibre/verify\_perf\_thresholds.py

```
#!/usr/bin/env python3
```

```
import json
import sys
from pathlib import Path
```

```
def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: verify_perf_thresholds.py <run-receipt.json>",
 file=sys.stderr
)
 return 2
```

```
 receipt_path = Path(sys.argv[1])
```

```
 if not receipt_path.exists():
```

```

 print(f"DENY: run receipt not found: {receipt_path}")
 return 1

receipt = json.loads(receipt_path.read_text())

envelope_path = Path(receipt["perf_envelope_path"])

if not envelope_path.exists():
 print(f"DENY: perf envelope not found: {envelope_path}")
 return 1

envelope = json.loads(envelope_path.read_text())
thresholds = envelope["thresholds"]

failures = []

for scenario in receipt["scenarios"]:
 checks = {
 "avg_frame_ms": scenario["avg_frame_ms"] <= thresholds["avg_frame_ms"],
 "p95_frame_ms": scenario["p95_frame_ms"] <= thresholds["p95_frame_ms"],
 "idle_ms": scenario["idle_ms"] <= thresholds["idle_ms"],
 "load_ms": scenario["load_ms"] <= thresholds["load_ms"]
 }

 recomputed_passed = all(checks.values())

 if scenario["passed"] != recomputed_passed:
 failures.append(
 f"{scenario['name']}: passed flag mismatch "
 f"(receipt={scenario['passed']} recomputed={recomputed_passed})"
)

 for metric, ok in checks.items():
 if not ok:
 failures.append(
 f"{scenario['name']}: {metric} exceeded "
 f"(actual={scenario[metric]} threshold={thresholds[metric]})"
)

expected_status = (
 "completed"
 if all(s["passed"] for s in receipt["scenarios"])
 else "failed"
)

```

```

if receipt["status"] != expected_status:
 failures.append(
 f"receipt status mismatch "
 f"(receipt={receipt['status']} expected={expected_status})"
)

if failures:
 print("DENY: perf thresholds failed independent verification")
 for failure in failures:
 print(f"- {failure}")
 return 1

print("ANSWER: perf thresholds independently verified")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI addition

- name: Verify MapLibre perf thresholds

run: |

```
python3 tools/validators/maplibre/verify_perf_thresholds.py \
artifacts/perf/run-receipt.json
```

## Put it after receipt validation

- name: Validate MapLibre perf receipt

run: |

```
python3 tools/validators/maplibre/validate_perf_receipt.py \
artifacts/perf/run-receipt.json
```

- name: Verify MapLibre frame-derived metrics

run: |

```
python3 tools/validators/maplibre/verify_perf_metrics_from_frames.py \
artifacts/perf/run-receipt.json
```

- name: Verify MapLibre perf thresholds

run: |

```
python3 tools/validators/maplibre/verify_perf_thresholds.py \
artifacts/perf/run-receipt.json
```

Now the chain no longer trusts the harness for pass/fail:

```
PerfEnvelope
↓
RunReceipt metrics
↓
Frame CSV recomputation
↓
Threshold recomputation
↓
passed/status verified
```

Next strongest step: **add negative tests for lying receipts** — `passed: true` with bad metrics, mismatched `status`, and altered envelope hash.

Proceed with **negative tests for lying receipts**.

## Add to

### `tests/maplibre/test_perf_governance_negative_paths.py`

```
THRESHOLD_VALIDATOR = (
 ROOT / "tools/validators/maplibre/verify_perf_thresholds.py"
)
```

```
def make_receipt_workspace(tmp_path: Path) -> Path:
 envelope_dir = tmp_path / "config/maplibre"
 envelope_dir.mkdir(parents=True)
```

```
envelope = {
 "object_type": "PerfEnvelope",
 "schema_version": "v1",
 "domain": "maplibre",
 "policy_posture": "public_safe",
 "thresholds": {
 "avg_frame_ms": 22,
 "p95_frame_ms": 40,
 "idle_ms": 3000,
 "load_ms": 4000,
 "render_pixel_delta_ratio": 0.01
 }
}
```

```

 },
 "notes": ["test envelope"]
}

```

```

envelope_path = envelope_dir / "perf-envelope.v1.json"
envelope_path.write_text(json.dumps(envelope, indent=2))

```

```

receipt_dir = tmp_path / "artifacts/perf"
receipt_dir.mkdir(parents=True)

```

```

receipt = {
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "run_id": "maplibre-perf-test",
 "created_utc": "2026-05-14T16:00:00Z",
 "domain": "maplibre",
 "status": "completed",
 "policy_posture": "public_safe",
 "spec_hash": "sha256:" + "a" * 64,
 "perf_envelope_path": "config/maplibre/perf-envelope.v1.json",
 "perf_envelope_hash": "sha256:" + "b" * 64,
 "scenarios": [
 {
 "name": "cache-heavy-layout-slim",
 "style_mode": "layout-first",
 "tile_set": "slim",
 "avg_frame_ms": 12,
 "p95_frame_ms": 20,
 "idle_ms": 1000,
 "load_ms": 900,
 "frame_times_path": "artifacts/perf/frame-times/cache-heavy-layout-slim.csv",
 "screenshot_path": "artifacts/perf/screenshots/cache-heavy-layout-slim.png",
 "render_diff_report_path": "artifacts/perf/render-diff/render-diff-report.json",
 "passed": True
 }
]
}

```

```

receipt_path = receipt_dir / "run-receipt.json"
receipt_path.write_text(json.dumps(receipt, indent=2))

```

```

return receipt_path

```

```

def test_lying_passed_true_with_bad_metrics_denied(tmp_path):

```

```

copy_schema_tree(tmp_path)
copy_tools_tree(tmp_path)

receipt_path = make_receipt_workspace(tmp_path)
receipt = json.loads(receipt_path.read_text())

receipt["scenarios"][0]["avg_frame_ms"] = 99
receipt["scenarios"][0]["passed"] = True
receipt["status"] = "completed"

receipt_path.write_text(json.dumps(receipt, indent=2))

result = run(
 ["python3", str(THRESHOLD_VALIDATOR), str(receipt_path)],
 cwd=tmp_path
)

assert result.returncode == 1
assert "DENY" in result.stdout
assert "passed flag mismatch" in result.stdout
assert "avg_frame_ms exceeded" in result.stdout

def test_receipt_status_completed_when_scenario_failed_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 receipt_path = make_receipt_workspace(tmp_path)
 receipt = json.loads(receipt_path.read_text())

 receipt["scenarios"][0]["passed"] = False
 receipt["status"] = "completed"

 receipt_path.write_text(json.dumps(receipt, indent=2))

 result = run(
 ["python3", str(THRESHOLD_VALIDATOR), str(receipt_path)],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "receipt status mismatch" in result.stdout

def test_receipt_status_failed_when_all_scenarios_pass_denied(tmp_path):

```

```

copy_schema_tree(tmp_path)
copy_tools_tree(tmp_path)

receipt_path = make_receipt_workspace(tmp_path)
receipt = json.loads(receipt_path.read_text())

receipt["scenarios"][0]["passed"] = True
receipt["status"] = "failed"

receipt_path.write_text(json.dumps(receipt, indent=2))

result = run(
 ["python3", str(THRESHOLD_VALIDATOR), str(receipt_path)],
 cwd=tmp_path
)

assert result.returncode == 1
assert "DENY" in result.stdout
assert "receipt status mismatch" in result.stdout

```

## Add envelope-hash negative test to receipt validator

```

RECEIPT_VALIDATOR = (
 ROOT / "tools/validators/maplibre/validate_perf_receipt.py"
)

def test_perf_envelope_hash_mismatch_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 receipt_path = make_receipt_workspace(tmp_path)

 result = run(
 ["python3", str(RECEIPT_VALIDATOR), str(receipt_path)],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "perf envelope hash mismatch" in result.stdout

```

## CI now catches

bad metric but passed=true → DENY  
scenario failed but status OK → DENY  
all passed but status failed → DENY  
envelope hash mismatch → DENY

Next after this: **positive fixture builder** so tests do not hand-build inconsistent hashes forever.

Proceed with a **positive fixture builder** so tests generate consistent hashes, paths, envelopes, receipts, and manifests.

## Add

```
tests/maplibre/
├── test_perf_governance_negative_paths.py
└── perf_fixture_builder.py
```

## tests/maplibre/perf\_fixture\_builder.py

```
import hashlib
import json
from pathlib import Path

def sha256_bytes(data: bytes) -> str:
 return "sha256:" + hashlib.sha256(data).hexdigest()

def sha256_file(path: Path) -> str:
 return sha256_bytes(path.read_bytes())

def sha256_json(value: dict) -> str:
 encoded = json.dumps(
 value,
 separators=(",", ":"),
 sort_keys=True
).encode()
 return sha256_bytes(encoded)

def write_json(path: Path, value: dict | list) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 path.write_text(json.dumps(value, indent=2))
```



```

def build_perf_workspace(tmp_path: Path) -> dict:
 envelope = {
 "object_type": "PerfEnvelope",
 "schema_version": "v1",
 "domain": "maplibre",
 "policy_posture": "public_safe",
 "thresholds": {
 "avg_frame_ms": 22,
 "p95_frame_ms": 40,
 "idle_ms": 3000,
 "load_ms": 4000,
 "render_pixel_delta_ratio": 0.01
 },
 "notes": ["test envelope"]
 }

 envelope_path = tmp_path / "config/maplibre/perf-envelope.v1.json"
 write_json(envelope_path, envelope)

 frame_times_path = (
 tmp_path /
 "artifacts/perf/frame-times/cache-heavy-layout-slim.csv"
)
 frame_times_path.parent.mkdir(parents=True, exist_ok=True)
 frame_times_path.write_text(
 "frame_index,frame_ms\n"
 "0,10\n"
 "1,12\n"
 "2,14\n"
 "3,16\n"
 "4,18\n"
)

 screenshot_path = (
 tmp_path /
 "artifacts/perf/screenshots/cache-heavy-layout-slim.png"
)
 screenshot_path.parent.mkdir(parents=True, exist_ok=True)
 screenshot_path.write_bytes(b"fake-png-test-bytes")

 render_diff_path = (
 tmp_path /
 "artifacts/perf/render-diff/render-diff-report.json"
)

```

```

render_diff = {
 "object_type": "RenderDiffReport",
 "schema_version": "v1",
 "domain": "maplibre",
 "created_utc": "2026-05-14T16:00:00Z",
 "status": "passed",
 "threshold": 0.01,
 "scenarios": [
 {
 "scenario": "cache-heavy-layout-slim",
 "screenshot_path": (
 "artifacts/perf/screenshots/"
 "cache-heavy-layout-slim.png"
),
 "baseline_path": (
 "tests/fixtures/maplibre/baselines/"
 "cache-heavy-layout-slim.png"
),
 "diff_path": (
 "artifacts/perf/render-diff/"
 "cache-heavy-layout-slim.diff.png"
),
 "status": "compared",
 "pixel_delta_ratio": 0,
 "threshold": 0.01,
 "passed": True,
 "notes": []
 }
]
}

write_json(render_diff_path, render_diff)

receipt = {
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "run_id": "maplibre-perf-test",
 "created_utc": "2026-05-14T16:00:00Z",
 "domain": "maplibre",
 "status": "completed",
 "policy_posture": "public_safe",
 "spec_hash": "sha256:" + "a" * 64,
 "perf_envelope_path": "config/maplibre/perf-envelope.v1.json",
 "perf_envelope_hash": sha256_json(envelope),
 "scenarios": [

```

```

{
 "name": "cache-heavy-layout-slim",
 "style_mode": "layout-first",
 "tile_set": "slim",
 "avg_frame_ms": 14,
 "p95_frame_ms": 18,
 "idle_ms": 1000,
 "load_ms": 900,
 "frame_times_path": (
 "artifacts/perf/frame-times/"
 "cache-heavy-layout-slim.csv"
),
 "screenshot_path": (
 "artifacts/perf/screenshots/"
 "cache-heavy-layout-slim.png"
),
 "render_diff_report_path": (
 "artifacts/perf/render-diff/"
 "render-diff-report.json"
),
 "passed": True
}
]
}

```

```

receipt_path = tmp_path / "artifacts/perf/run-receipt.json"
write_json(receipt_path, receipt)

```

```

dsse = {
 "payloadType": (
 "application/vnd.kfm.maplibre.perf."
 "run-receipt.v1+json"
),
 "payload": "test",
 "signatures": [],
 "kfm_attestation": {
 "object_type": "AttestationEnvelope",
 "schema_version": "v1",
 "subject_type": "RunReceipt",
 "subject_path": "artifacts/perf/run-receipt.json",
 "subject_sha256": sha256_file(receipt_path),
 "domain": "maplibre",
 "policy_posture": "public_safe",
 "created_utc": "2026-05-14T16:00:00Z",

```

```

 "signing_status": "unsigned",
 "notes": ["test attestation"]
 }
}

dsse_path = tmp_path / "artifacts/perf/run-receipt.dsse.json"
write_json(dsse_path, dsse)

checksum_path = tmp_path / "artifacts/perf/run-receipt.sha256"
checksum_path.write_text(
 f'{{sha256_file(receipt_path).removeprefix('sha256:')}} '
 "artifacts/perf/run-receipt.json\n"
)

perf_results_path = tmp_path / "artifacts/perf/perf-results.json"
write_json(perf_results_path, [])

manifest = {
 "object_type": "ReleaseManifest",
 "schema_version": "v1",
 "release_id": "maplibre-perf-release-test",
 "created_utc": "2026-05-14T16:00:00Z",
 "domain": "maplibre",
 "status": "candidate",
 "policy_posture": "public_safe",
 "artifacts": [
 {
 "name": "run-receipt",
 "path": "artifacts/perf/run-receipt.json",
 "sha256": sha256_file(receipt_path),
 "role": "run_receipt"
 },
 {
 "name": "perf-results",
 "path": "artifacts/perf/perf-results.json",
 "sha256": sha256_file(perf_results_path),
 "role": "perf_results"
 },
 {
 "name": "run-receipt-dsse",
 "path": "artifacts/perf/run-receipt.dsse.json",
 "sha256": sha256_file(dsse_path),
 "role": "attestation"
 },
],
}

```

```

{
 "name": "run-receipt-sha256",
 "path": "artifacts/perf/run-receipt.sha256",
 "sha256": sha256_file(checksum_path),
 "role": "checksum"
},
{
 "name": "perf-envelope",
 "path": "config/maplibre/perf-envelope.v1.json",
 "sha256": sha256_file(envelope_path),
 "role": "perf_envelope"
},
{
 "name": "frame-times-cache-heavy-layout-slim",
 "path": (
 "artifacts/perf/frame-times/"
 "cache-heavy-layout-slim.csv"
),
 "sha256": sha256_file(frame_times_path),
 "role": "frame_times"
},
{
 "name": "screenshot-cache-heavy-layout-slim",
 "path": (
 "artifacts/perf/screenshots/"
 "cache-heavy-layout-slim.png"
),
 "sha256": sha256_file(screenshot_path),
 "role": "screenshot"
},
{
 "name": "render-diff-report",
 "path": (
 "artifacts/perf/render-diff/"
 "render-diff-report.json"
),
 "sha256": sha256_file(render_diff_path),
 "role": "render_diff"
}
]
}

```

```

manifest_path = (
 tmp_path /

```

```

 "artifacts/perf/release-manifest.maplibre-perf.json"
)
write_json(manifest_path, manifest)

return {
 "envelope_path": envelope_path,
 "receipt_path": receipt_path,
 "manifest_path": manifest_path,
 "dsse_path": dsse_path,
 "checksum_path": checksum_path,
 "frame_times_path": frame_times_path,
 "render_diff_path": render_diff_path
}

```

## Refactor tests

Replace hand-built fixture helpers with:

```
from tests.maplibre.perf_fixture_builder import build_perf_workspace
```

Example positive-path test:

```

def test_valid_perf_governance_workspace_passes(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)

 checks = [
 ["python3", str(RECEIPT_VALIDATOR), str(paths["receipt_path"])],
 ["python3", str(THRESHOLD_VALIDATOR), str(paths["receipt_path"])],
 ["python3", str(MANIFEST_VALIDATOR), str(paths["manifest_path"])],
 ["python3", str(HASH_VALIDATOR), str(paths["manifest_path"])],
 ["python3", str(SUBJECT_LINK_VALIDATOR), str(paths["manifest_path"])]
]

 for cmd in checks:
 result = run(cmd, cwd=tmp_path)
 assert result.returncode == 0, result.stdout + result.stderr
 assert "ANSWER" in result.stdout

```

Next after this: add **duplicate-role denial** directly to the manifest validator, then test it with the builder.

Proceed with **duplicate-role denial in the manifest validator**.

## Patch `validate_perf_release_manifest.py`

Add after the required-role check:

```
role_counts = {}

for artifact in manifest["artifacts"]:
 role = artifact["role"]
 role_counts[role] = role_counts.get(role, 0) + 1

single_roles = {
 "run_receipt",
 "perf_results",
 "attestation",
 "checksum",
 "perf_envelope",
 "render_diff"
}

duplicates = [
 role
 for role in single_roles
 if role_counts.get(role, 0) > 1
]

if duplicates:
 print("DENY: duplicate single-instance artifact roles")
 for role in sorted(duplicates):
 print(f"- {role}")
 return 1
```

Keep these roles allowed as multi-instance:

```
frame_times
screenshot
```

## Add test

```
def test_duplicate_run_receipt_role_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)

 manifest = json.loads(paths["manifest_path"].read_text())

 duplicate = dict(manifest["artifacts"][0])
 duplicate["name"] = "run-receipt-duplicate"

 manifest["artifacts"].append(duplicate)
 paths["manifest_path"].write_text(json.dumps(manifest, indent=2))

 result = run(
 ["python3", str(MANIFEST_VALIDATOR), str(paths["manifest_path"])],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "duplicate single-instance artifact roles" in result.stdout
 assert "run_receipt" in result.stdout
```

Now the manifest cannot hide ambiguity:

one RunReceipt	✓
one attestation	✓
one checksum	✓
one perf envelope	✓
many screenshots	✓
many frame CSVs	✓
duplicate receipt	DENY
duplicate DSSE	DENY
duplicate envelope	DENY

Next is **scenario-to-artifact coverage validation**.

You already allow many `screenshot` and `frame_times` artifacts, but nothing proves every `RunReceipt.scenarios[ ]` entry has matching manifest artifacts.



Add:

tools/validators/maplibre/  
└─ verify\_perf\_manifest\_coverage.py

It should verify:

Receipt scenario field	Must appear in ReleaseManifest as role
frame_times_path	frame_times
screenshot_path	screenshot
render_diff_report_path	render_diff
perf_envelope_path	perf_envelope

Also verify every manifest `frame_times` and `screenshot` artifact is actually referenced by a receipt scenario, so stale orphan artifacts cannot sneak in.

After that, add tests for:

missing frame\_times artifact → DENY  
missing screenshot artifact → DENY  
orphan screenshot artifact → DENY  
orphan frame\_times artifact → DENY

This closes the next integrity gap:

RunReceipt says scenario used X  
↓  
ReleaseManifest must hash-lock X  
↓  
No unreferenced scenario evidence allowed

Next: add **scenario-to-artifact coverage validation**.

**tools/validators/maplibre/verify\_perf\_manifest\_coverage.py**

```

#!/usr/bin/env python3

import json
import sys
from pathlib import Path

def paths_by_role(manifest: dict, role: str) -> set[str]:
 return {
 artifact["path"]
 for artifact in manifest["artifacts"]
 if artifact["role"] == role
 }

def main() -> int:
 if len(sys.argv) != 3:
 print(
 "usage: verify_perf_manifest_coverage.py <run-receipt.json> <manifest.json>",
 file=sys.stderr
)
 return 2

 receipt_path = Path(sys.argv[1])
 manifest_path = Path(sys.argv[2])

 if not receipt_path.exists():
 print(f"DENY: run receipt not found: {receipt_path}")
 return 1

 if not manifest_path.exists():
 print(f"DENY: release manifest not found: {manifest_path}")
 return 1

 receipt = json.loads(receipt_path.read_text())
 manifest = json.loads(manifest_path.read_text())

 frame_paths = paths_by_role(manifest, "frame_times")
 screenshot_paths = paths_by_role(manifest, "screenshot")
 render_diff_paths = paths_by_role(manifest, "render_diff")
 envelope_paths = paths_by_role(manifest, "perf_envelope")

 referenced_frames = set()
 referenced_screenshots = set()

 failures = []

```

```

for scenario in receipt["scenarios"]:
 name = scenario["name"]

 frame_path = scenario["frame_times_path"]
 screenshot_path = scenario["screenshot_path"]
 render_diff_path = scenario["render_diff_report_path"]

 referenced_frames.add(frame_path)
 referenced_screenshots.add(screenshot_path)

 if frame_path not in frame_paths:
 failures.append(
 f'{name}: frame_times_path not present in manifest: {frame_path}'
)

 if screenshot_path not in screenshot_paths:
 failures.append(
 f'{name}: screenshot_path not present in manifest: {screenshot_path}'
)

 if render_diff_path not in render_diff_paths:
 failures.append(
 f'{name}: render_diff_report_path not present in manifest: {render_diff_path}'
)

 envelope_path = receipt["perf_envelope_path"]

 if envelope_path not in envelope_paths:
 failures.append(
 f'perf_envelope_path not present in manifest: {envelope_path}'
)

 orphan_frames = frame_paths - referenced_frames
 orphan_screenshots = screenshot_paths - referenced_screenshots

 for path in sorted(orphan_frames):
 failures.append(f'orphan frame_times artifact not referenced by receipt: {path}')

 for path in sorted(orphan_screenshots):
 failures.append(f'orphan screenshot artifact not referenced by receipt: {path}')

 if failures:
 print("DENY: perf manifest coverage verification failed")

```

```

 for failure in failures:
 print(f"- {failure}")
 return 1

 print("ANSWER: perf manifest coverage verified")
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI addition

- name: Verify MapLibre manifest coverage  
 run: |  
 python3 tools/validators/maplibre/verify\_perf\_manifest\_coverage.py \  
 artifacts/perf/run-receipt.json \  
 artifacts/perf/release-manifest.maplibre-perf.json

## Test additions

```

COVERAGE_VALIDATOR = (
 ROOT / "tools/validators/maplibre/verify_perf_manifest_coverage.py"
)

def test_missing_frame_times_manifest_artifact_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)
 manifest = json.loads(paths["manifest_path"].read_text())

 manifest["artifacts"] = [
 a for a in manifest["artifacts"]
 if a["role"] != "frame_times"
]

 paths["manifest_path"].write_text(json.dumps(manifest, indent=2))

 result = run(
 [
 "python3",
 str(COVERAGE_VALIDATOR),

```

```

 str(paths["receipt_path"]),
 str(paths["manifest_path"])
],
 cwd=tmp_path
)

assert result.returncode == 1
assert "DENY" in result.stdout
assert "frame_times_path not present" in result.stdout

def test_missing_screenshot_manifest_artifact_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)
 manifest = json.loads(paths["manifest_path"].read_text())

 manifest["artifacts"] = [
 a for a in manifest["artifacts"]
 if a["role"] != "screenshot"
]

 paths["manifest_path"].write_text(json.dumps(manifest, indent=2))

 result = run(
 [
 "python3",
 str(COVERAGE_VALIDATOR),
 str(paths["receipt_path"]),
 str(paths["manifest_path"])
],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "screenshot_path not present" in result.stdout

def test_orphan_screenshot_manifest_artifact_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)
 manifest = json.loads(paths["manifest_path"].read_text())

```

```

orphan_path = tmp_path / "artifacts/perf/screenshots/orphan.png"
orphan_path.parent.mkdir(parents=True, exist_ok=True)
orphan_path.write_bytes(b"orphan")

manifest["artifacts"].append({
 "name": "orphan-screenshot",
 "path": "artifacts/perf/screenshots/orphan.png",
 "sha256": sha256_file(orphan_path),
 "role": "screenshot"
})

paths["manifest_path"].write_text(json.dumps(manifest, indent=2))

result = run(
 [
 "python3",
 str(COVERAGE_VALIDATOR),
 str(paths["receipt_path"]),
 str(paths["manifest_path"])
],
 cwd=tmp_path
)

assert result.returncode == 1
assert "DENY" in result.stdout
assert "orphan screenshot artifact" in result.stdout

```

Now every scenario evidence path in the `RunReceipt` must be hash-locked in the `ReleaseManifest`, and every screenshot/frame artifact must be justified by a receipt scenario.

Next: **render-diff scenario coverage validation.**

You already prove receipt scenario paths exist in the manifest. Now prove the `RenderDiffReport.scenarios[ ]` matches the `RunReceipt.scenarios[ ]`.

Add:

```

tools/validators/maplibre/
└─ verify_render_diff_coverage.py

```

## tools/validators/maplibre/verify\_render\_diff\_coverage.py

```
#!/usr/bin/env python3

import json
import sys
from pathlib import Path

def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: verify_render_diff_coverage.py <run-receipt.json>",
 file=sys.stderr
)
 return 2

 receipt_path = Path(sys.argv[1])

 if not receipt_path.exists():
 print(f"DENY: run receipt not found: {receipt_path}")
 return 1

 receipt = json.loads(receipt_path.read_text())

 receipt_scenarios = {
 scenario["name"]: scenario
 for scenario in receipt["scenarios"]
 }

 failures = []

 for scenario in receipt["scenarios"]:
 report_path = Path(scenario["render_diff_report_path"])

 if not report_path.exists():
 failures.append(
 f"{scenario['name']}: render diff report missing: {report_path}"
)
 continue

 report = json.loads(report_path.read_text())
```

```

report_scenarios = {
 item["scenario"]: item
 for item in report.get("scenarios", [])
}

if scenario["name"] not in report_scenarios:
 failures.append(
 f"{scenario['name']}: missing from render diff report"
)
 continue

report_item = report_scenarios[scenario["name"]]

if report_item["screenshot_path"] != scenario["screenshot_path"]:
 failures.append(
 f"{scenario['name']}: render diff screenshot_path mismatch"
)

if not report_item["passed"]:
 failures.append(
 f"{scenario['name']}: render diff scenario failed"
)

orphan_report_scenarios = (
 set(report_scenarios.keys()) - set(receipt_scenarios.keys())
)

for name in sorted(orphan_report_scenarios):
 failures.append(
 f"orphan render diff scenario not referenced by receipt: {name}"
)

if failures:
 print("DENY: render diff coverage verification failed")
 for failure in failures:
 print(f"- {failure}")
 return 1

print("ANSWER: render diff coverage verified")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```



## CI addition

- name: Verify MapLibre render diff coverage

run: |

```
python3 tools/validators/maplibre/verify_render_diff_coverage.py \
 artifacts/perf/run-receipt.json
```

## Negative tests next

missing receipt scenario in render diff report → DENY

orphan render diff scenario → DENY

screenshot path mismatch → DENY

render diff scenario passed=false → DENY

After this, the visual evidence chain is closed:

RunReceipt scenario

↓

screenshot\_path

↓

RenderDiffReport scenario

↓

same screenshot\_path

↓

passed=true

↓

ReleaseManifest hash-locks both

Next: add the **negative tests for render-diff coverage**.

## Add validator constant

```
RENDER_DIFF_COVERAGE_VALIDATOR = (
 ROOT / "tools/validators/maplibre/verify_render_diff_coverage.py"
)
```

## Add tests

```
def test_render_diff_missing_receipt_scenario_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)
```

```

paths = build_perf_workspace(tmp_path)

report = json.loads(paths["render_diff_path"].read_text())
report["scenarios"] = []
paths["render_diff_path"].write_text(json.dumps(report, indent=2))

result = run(
 [
 "python3",
 str(RENDER_DIFF_COVERAGE_VALIDATOR),
 str(paths["receipt_path"])
],
 cwd=tmp_path
)

assert result.returncode == 1
assert "DENY" in result.stdout
assert "missing from render diff report" in result.stdout

```

```

def test_orphan_render_diff_scenario_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)

 report = json.loads(paths["render_diff_path"].read_text())
 orphan = dict(report["scenarios"][0])
 orphan["scenario"] = "orphan-scenario"
 report["scenarios"].append(orphan)
 paths["render_diff_path"].write_text(json.dumps(report, indent=2))

 result = run(
 [
 "python3",
 str(RENDER_DIFF_COVERAGE_VALIDATOR),
 str(paths["receipt_path"])
],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout

```

```
assert "orphan render diff scenario" in result.stdout
```

```
def test_render_diff_screenshot_path_mismatch_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)

 report = json.loads(paths["render_diff_path"].read_text())
 report["scenarios"][0]["screenshot_path"] = (
 "artifacts/perf/screenshots/wrong.png"
)
 paths["render_diff_path"].write_text(json.dumps(report, indent=2))

 result = run(
 [
 "python3",
 str(RENDER_DIFF_COVERAGE_VALIDATOR),
 str(paths["receipt_path"])
],
 cwd=tmp_path
)

 assert result.returncode == 1
 assert "DENY" in result.stdout
 assert "screenshot_path mismatch" in result.stdout
```

```
def test_render_diff_scenario_failed_denied(tmp_path):
 copy_schema_tree(tmp_path)
 copy_tools_tree(tmp_path)

 paths = build_perf_workspace(tmp_path)

 report = json.loads(paths["render_diff_path"].read_text())
 report["scenarios"][0]["passed"] = False
 report["status"] = "failed"
 paths["render_diff_path"].write_text(json.dumps(report, indent=2))

 result = run(
 [
 "python3",
 str(RENDER_DIFF_COVERAGE_VALIDATOR),
```

```

 str(paths["receipt_path"])
],
 cwd=tmp_path
)

assert result.returncode == 1
assert "DENY" in result.stdout
assert "render diff scenario failed" in result.stdout

```

Now covered:

```

receipt scenario absent from render diff → DENY
extra render diff scenario → DENY
screenshot mismatch → DENY
failed render diff scenario → DENY

```

Next: **unify all MapLibre perf validators behind one CI endpoint** so the workflow only calls one command.

Do this next: **one CI endpoint for all MapLibre perf governance gates.**

## Add

```

tools/validators/maplibre/
└── validate_perf_governance.py

```

## tools/validators/maplibre/validate\_perf\_governance.py

```
#!/usr/bin/env python3
```

```

import subprocess
import sys
from pathlib import Path

```

```

RECEIPT = "artifacts/perf/run-receipt.json"
MANIFEST = "artifacts/perf/release-manifest.maplibre-perf.json"
RENDER_DIFF = "artifacts/perf/render-diff/render-diff-report.json"
ENVELOPE = "config/maplibre/perf-envelope.v1.json"

```

```
CHECKS = [
```

```
[
 "Validate perf envelope",
 "tools/validators/maplibre/validate_perf_envelope.py",
 ENVELOPE
],
[
 "Validate render diff report",
 "tools/validators/maplibre/validate_render_diff_report.py",
 RENDER_DIFF
],
[
 "Validate perf receipt",
 "tools/validators/maplibre/validate_perf_receipt.py",
 RECEIPT
],
[
 "Verify frame-derived metrics",
 "tools/validators/maplibre/verify_perf_metrics_from_frames.py",
 RECEIPT
],
[
 "Verify thresholds",
 "tools/validators/maplibre/verify_perf_thresholds.py",
 RECEIPT
],
[
 "Validate release manifest",
 "tools/validators/maplibre/validate_perf_release_manifest.py",
 MANIFEST
],
[
 "Verify artifact hashes",
 "tools/validators/maplibre/verify_perf_artifact_hashes.py",
 MANIFEST
],
[
 "Verify subject links",
 "tools/validators/maplibre/verify_perf_subject_links.py",
 MANIFEST
],
[
 "Verify manifest coverage",
 "tools/validators/maplibre/verify_perf_manifest_coverage.py",
 RECEIPT,
```

```

 MANIFEST
],
[
 "Verify render diff coverage",
 "tools/validators/maplibre/verify_render_diff_coverage.py",
 RECEIPT
]
]

```

```

def main() -> int:
 failed = []

 for label, script, *args in CHECKS:
 script_path = Path(script)

 if not script_path.exists():
 print(f"DENY: missing validator: {script}")
 return 1

 print(f"\n==> {label}")

 result = subprocess.run(
 ["python3", script, *args],
 text=True,
 capture_output=True
)

 if result.stdout:
 print(result.stdout.strip())

 if result.stderr:
 print(result.stderr.strip(), file=sys.stderr)

 if result.returncode != 0:
 failed.append(label)

 if failed:
 print("\nDENY: MapLibre perf governance failed")
 for label in failed:
 print(f"- {label}")
 return 1

 print("\nANSWER: MapLibre perf governance passed")
 return 0

```

```
if __name__ == "__main__":
 raise SystemExit(main())
```

## Replace CI validator steps with one command

```
- name: Validate MapLibre perf governance
 run: |
 python3 tools/validators/maplibre/validate_perf_governance.py
```

## Full CI shape

```
- name: Validate MapLibre perf envelope
 run: |
 python3 tools/validators/maplibre/validate_perf_envelope.py \
 config/maplibre/perf-envelope.v1.json

- name: Run MapLibre smoke perf
 run: node scripts/maplibre-smoke-perf.mjs

- name: Build MapLibre render diff report
 run: node scripts/build-maplibre-render-diff.mjs

- name: Attest MapLibre perf receipt
 run: node scripts/attest-maplibre-perf.mjs

- name: Build MapLibre perf release manifest
 run: node scripts/build-maplibre-perf-release-manifest.mjs

- name: Validate MapLibre perf governance
 run: |
 python3 tools/validators/maplibre/validate_perf_governance.py
```

Now CI has one final gate:

MapLibre perf governance passed

or:

DENY: MapLibre perf governance failed

Next: **turn the final governance pass into a ProofPack.**

That gives you one durable object that says:

these exact artifacts  
passed these exact validators  
under this exact envelope  
with these exact hashes

Add:

artifacts/perf/  
└─ proof-pack.maplibre-perf.json

scripts/  
└─ build-maplibre-perf-proof-pack.mjs

Core fields:

```
{
 "object_type": "ProofPack",
 "schema_version": "v1",
 "domain": "maplibre",
 "proof_id": "maplibre-perf-proof-...",
 "created_utc": "...",
 "release_manifest_path": "artifacts/perf/release-manifest.maplibre-perf.json",
 "run_receipt_path": "artifacts/perf/run-receipt.json",
 "perf_envelope_path": "config/maplibre/perf-envelope.v1.json",
 "validation_command": "python3 tools/validators/maplibre/validate_perf_governance.py",
 "validation_outcome": "ANSWER",
 "artifact_count": 8,
 "release_manifest_sha256": "sha256:...",
 "run_receipt_sha256": "sha256:..."
}
```

Then add one final validator:

tools/validators/maplibre/validate\_perf\_proof\_pack.py

It should check:

ProofPack schema is valid



referenced receipt exists  
referenced manifest exists  
referenced envelope exists  
hashes match  
validation\_outcome == ANSWER  
artifact\_count matches manifest artifacts length

Final CI becomes:

- name: Validate MapLibre perf governance  
run: python3 tools/validators/maplibre/validate\_perf\_governance.py
- name: Build MapLibre perf ProofPack  
run: node scripts/build-maplibre-perf-proof-pack.mjs
- name: Validate MapLibre perf ProofPack  
run: |  
python3 tools/validators/maplibre/validate\_perf\_proof\_pack.py \  
artifacts/perf/proof-pack.maplibre-perf.json

After ProofPack, the next frontier is **CorrectionNotice + rollback rehearsal** for failed renderer releases.

Proceed with **MapLibre perf ProofPack builder + validator**.

## Add

artifacts/perf/  
└─ proof-pack.maplibre-perf.json

scripts/  
└─ build-maplibre-perf-proof-pack.mjs

schemas/maplibre/  
└─ perf-proof-pack.schema.json

tools/validators/maplibre/  
└─ validate\_perf\_proof\_pack.py

**scripts/build-maplibre-perf-proof-pack.mjs**

#!/usr/bin/env node

```

import crypto from "node:crypto";
import fs from "node:fs/promises";

const manifestPath =
 "artifacts/perf/release-manifest.maplibre-perf.json";
const receiptPath =
 "artifacts/perf/run-receipt.json";
const envelopePath =
 "config/maplibre/perf-envelope.v1.json";
const outPath =
 "artifacts/perf/proof-pack.maplibre-perf.json";

function sha256(bytes) {
 return (
 "sha256:" +
 crypto.createHash("sha256").update(bytes).digest("hex")
);
}

async function sha256File(path) {
 return sha256(await fs.readFile(path));
}

const manifest = JSON.parse(await fs.readFile(manifestPath, "utf8"));

const proofPack = {
 object_type: "ProofPack",
 schema_version: "v1",
 domain: "maplibre",
 proof_id: `maplibre-perf-proof-${new Date()
 .toISOString()
 .replaceAll(":", "-")}`,
 created_utc: new Date().toISOString(),
 release_manifest_path: manifestPath,
 run_receipt_path: receiptPath,
 perf_envelope_path: envelopePath,
 validation_command:
 "python3 tools/validators/maplibre/validate_perf_governance.py",
 validation_outcome: "ANSWER",
 artifact_count: manifest.artifacts.length,
 release_manifest_sha256: await sha256File(manifestPath),
 run_receipt_sha256: await sha256File(receiptPath),
 perf_envelope_sha256: await sha256File(envelopePath)
};

```

```
};

await fs.mkdir("artifacts/perf", { recursive: true });

await fs.writeFile(outPath, JSON.stringify(proofPack, null, 2));

console.log("ANSWER: wrote MapLibre perf ProofPack");
console.log(` - ${outPath}`);
```

## schemas/maplibre/perf-proof-pack.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/perf-proof-pack/v1",
 "title": "MapLibre Perf ProofPack",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "domain",
 "proof_id",
 "created_utc",
 "release_manifest_path",
 "run_receipt_path",
 "perf_envelope_path",
 "validation_command",
 "validation_outcome",
 "artifact_count",
 "release_manifest_sha256",
 "run_receipt_sha256",
 "perf_envelope_sha256"
],
 "additionalProperties": false,
 "properties": {
 "object_type": { "const": "ProofPack" },
 "schema_version": { "const": "v1" },
 "domain": { "const": "maplibre" },
 "proof_id": { "type": "string", "minLength": 8 },
 "created_utc": { "type": "string", "format": "date-time" },
 "release_manifest_path": { "type": "string", "minLength": 1 },
 "run_receipt_path": { "type": "string", "minLength": 1 },
 "perf_envelope_path": { "type": "string", "minLength": 1 },
 "validation_command": { "type": "string", "minLength": 1 },

```

```

"validation_outcome": { "const": "ANSWER" },
"artifact_count": { "type": "integer", "minimum": 1 },
"release_manifest_sha256": {
 "type": "string",
 "pattern": "^sha256:[a-f0-9]{64}$"
},
"run_receipt_sha256": {
 "type": "string",
 "pattern": "^sha256:[a-f0-9]{64}$"
},
"perf_envelope_sha256": {
 "type": "string",
 "pattern": "^sha256:[a-f0-9]{64}$"
}
}
}

```

## tools/validators/maplibre/validate\_perf\_proof\_pack.py

```
#!/usr/bin/env python3
```

```

import hashlib
import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator, FormatChecker

```

```
SCHEMA_PATH = Path("schemas/maplibre/perf-proof-pack.schema.json")
```

```
def sha256_file(path: Path) -> str:
```

```
 h = hashlib.sha256()
```

```
 with path.open("rb") as f:
```

```
 for chunk in iter(lambda: f.read(65536), b''):
```

```
 h.update(chunk)
```

```
 return "sha256:" + h.hexdigest()
```

```
def main() -> int:
```

```
 if len(sys.argv) != 2:
```

```
 print(
```

```

 "usage: validate_perf_proof_pack.py <proof-pack.json>",
 file=sys.stderr
)
 return 2

proof_path = Path(sys.argv[1])

if not proof_path.exists():
 print(f"DENY: proof pack not found: {proof_path}")
 return 1

schema = json.loads(SCHEMA_PATH.read_text())
proof = json.loads(proof_path.read_text())

validator = Draft202012Validator(
 schema,
 format_checker=FormatChecker()
)

errors = sorted(
 validator.iter_errors(proof),
 key=lambda e: e.path
)

if errors:
 print("DENY: invalid MapLibre perf ProofPack")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

manifest_path = Path(proof["release_manifest_path"])
receipt_path = Path(proof["run_receipt_path"])
envelope_path = Path(proof["perf_envelope_path"])

for label, path in [
 ("release manifest", manifest_path),
 ("run receipt", receipt_path),
 ("perf envelope", envelope_path)
]:
 if not path.exists():
 print(f"DENY: missing {label}: {path}")
 return 1

```

```

checks = [
 (
 "release manifest",
 proof["release_manifest_sha256"],
 sha256_file(manifest_path)
),
 (
 "run receipt",
 proof["run_receipt_sha256"],
 sha256_file(receipt_path)
),
 (
 "perf envelope",
 proof["perf_envelope_sha256"],
 sha256_file(envelope_path)
)
]

```

```

for label, expected, actual in checks:

```

```

 if expected != actual:
 print(f"DENY: {label} hash mismatch")
 print(f"- expected={expected}")
 print(f"- actual={actual}")
 return 1

```

```

manifest = json.loads(manifest_path.read_text())
actual_count = len(manifest["artifacts"])

```

```

if proof["artifact_count"] != actual_count:
 print("DENY: ProofPack artifact_count mismatch")
 print(f"- expected={proof['artifact_count']}")
 print(f"- actual={actual_count}")
 return 1

```

```

if proof["validation_outcome"] != "ANSWER":
 print("DENY: ProofPack validation_outcome is not ANSWER")
 return 1

```

```

print("ANSWER: MapLibre perf ProofPack valid")
return 0

```

```

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI addition

- name: Validate MapLibre perf governance  
run: |  
python3 tools/validators/maplibre/validate\_perf\_governance.py
- name: Build MapLibre perf ProofPack  
run: |  
node scripts/build-maplibre-perf-proof-pack.mjs
- name: Validate MapLibre perf ProofPack  
run: |  
python3 tools/validators/maplibre/validate\_perf\_proof\_pack.py \\  
artifacts/perf/proof-pack.maplibre-perf.json

Next: add **proof\_pack** as a **ReleaseManifest** artifact role and hash-lock the ProofPack into a **second final manifest pass**.

Proceed with **hash-locking the ProofPack into the ReleaseManifest**.

## 1. Add **proof\_pack** role to manifest schema

In `schemas/maplibre/perf-release-manifest.schema.json`, add:

```
"proof_pack"
```

to the artifact role enum.

## 2. Patch manifest builder

In `scripts/build-maplibre-perf-release-manifest.mjs`, add optional ProofPack inclusion:

```
try {
 await fs.access("artifacts/perf/proof-pack.maplibre-perf.json");

 artifacts.push({
 name: "proof-pack-maplibre-perf",
 path: "artifacts/perf/proof-pack.maplibre-perf.json",
 role: "proof_pack"
 });
}
```

```
} catch {
 // First manifest pass happens before ProofPack exists.
}
```

### 3. Patch manifest validator

In `validate_perf_release_manifest.py`, add `proof_pack` to single-instance roles:

```
single_roles = {
 "run_receipt",
 "perf_results",
 "attestation",
 "checksum",
 "perf_envelope",
 "render_diff",
 "proof_pack"
}
```

Do **not** add it to `REQUIRED_ROLES`, because the first manifest pass happens before ProofPack exists.

### 4. Add final manifest rebuild to CI

- name: Validate MapLibre perf governance  
run: |  
python3 tools/validators/maplibre/validate\_perf\_governance.py
- name: Build MapLibre perf ProofPack  
run: |  
node scripts/build-maplibre-perf-proof-pack.mjs
- name: Validate MapLibre perf ProofPack  
run: |  
python3 tools/validators/maplibre/validate\_perf\_proof\_pack.py \  
artifacts/perf/proof-pack.maplibre-perf.json
- name: Rebuild MapLibre perf release manifest with ProofPack  
run: |  
node scripts/build-maplibre-perf-release-manifest.mjs
- name: Final validate MapLibre perf governance  
run: |



python3 tools/validators/maplibre/validate\_perf\_governance.py

## Resulting chain

ReleaseManifest candidate  
↓  
governance validation  
↓  
ProofPack generated  
↓  
ProofPack validated  
↓  
ReleaseManifest rebuilt including ProofPack  
↓  
final governance validation

Next: add **CorrectionNotice + RollbackPlan** for failed or rejected renderer releases.

Proceed with **CorrectionNotice + RollbackPlan** for failed/rejected MapLibre perf releases.

## Add roles to manifest schema

"correction\_notice",  
"rollback\_plan"

## artifacts/perf/correction-notice.maplibre-perf.json

```
{
 "object_type": "CorrectionNotice",
 "schema_version": "v1",
 "domain": "maplibre",
 "notice_id": "maplibre-perf-correction-NEEDS_VERIFICATION",
 "created_utc": "NEEDS_VERIFICATION",
 "trigger": "perf_or_render_governance_failure",
 "status": "draft",
 "affected_release_manifest_path": "artifacts/perf/release-manifest.maplibre-perf.json",
 "reason": "NEEDS_VERIFICATION",
 "required_action": "Block publication until perf governance returns ANSWER.",
 "policy_posture": "review_required"
```

```
}
```

## artifacts/perf/rollback-plan.maplibre-perf.json

```
{
 "object_type": "RollbackPlan",
 "schema_version": "v1",
 "domain": "maplibre",
 "rollback_id": "maplibre-perf-rollback-NEEDS_VERIFICATION",
 "created_utc": "NEEDS_VERIFICATION",
 "status": "draft",
 "affected_release_manifest_path": "artifacts/perf/release-manifest.maplibre-perf.json",
 "rollback_strategy": "Retain prior released renderer/tile artifacts; do not promote failed candidate.",
 "verification_command": "python3 tools/validators/maplibre/validate_perf_governance.py",
 "policy_posture": "review_required"
}
```

## Builder script

scripts/build-maplibre-perf-correction-and-rollback.mjs

```
#!/usr/bin/env node
```

```
import fs from "node:fs/promises";
```

```
const manifestPath =
 "artifacts/perf/release-manifest.maplibre-perf.json";
```

```
const correctionPath =
 "artifacts/perf/correction-notice.maplibre-perf.json";
```

```
const rollbackPath =
 "artifacts/perf/rollback-plan.maplibre-perf.json";
```

```
const manifest = JSON.parse(await fs.readFile(manifestPath, "utf8"));
const now = new Date().toISOString();
```

```
const needsCorrection = manifest.status !== "candidate";
```

```

if (!needsCorrection) {
 console.log("ANSWER: no correction or rollback needed");
 process.exit(0);
}

const correctionNotice = {
 object_type: "CorrectionNotice",
 schema_version: "v1",
 domain: "maplibre",
 notice_id: `maplibre-perf-correction-${now.replaceAll(":", "-")}`,
 created_utc: now,
 trigger: "perf_or_render_governance_failure",
 status: "draft",
 affected_release_manifest_path: manifestPath,
 reason: `Release manifest status is ${manifest.status}`,
 required_action:
 "Block publication until perf governance returns ANSWER.",
 policy_posture: "review_required"
};

const rollbackPlan = {
 object_type: "RollbackPlan",
 schema_version: "v1",
 domain: "maplibre",
 rollback_id: `maplibre-perf-rollback-${now.replaceAll(":", "-")}`,
 created_utc: now,
 status: "draft",
 affected_release_manifest_path: manifestPath,
 rollback_strategy:
 "Retain prior released renderer/tile artifacts; do not promote failed candidate.",
 verification_command:
 "python3 tools/validators/maplibre/validate_perf_governance.py",
 policy_posture: "review_required"
};

await fs.writeFile(correctionPath, JSON.stringify(correctionNotice, null, 2));
await fs.writeFile(rollbackPath, JSON.stringify(rollbackPlan, null, 2));

console.log("DENY: correction and rollback artifacts generated");
console.log(` - ${correctionPath}`);
console.log(` - ${rollbackPath}`);

```

## Manifest builder optional inclusion

```
for (const optional of [
 {
 name: "correction-notice-maplibre-perf",
 path: "artifacts/perf/correction-notice.maplibre-perf.json",
 role: "correction_notice"
 },
 {
 name: "rollback-plan-maplibre-perf",
 path: "artifacts/perf/rollback-plan.maplibre-perf.json",
 role: "rollback_plan"
 }
]) {
 try {
 await fs.access(optional.path);
 artifacts.push(optional);
 } catch {}
}
```

## CI failure branch idea

```
- name: Build correction and rollback artifacts on failure
 if: failure()
 run: |
 node scripts/build-maplibre-perf-correction-and-rollback.mjs || true
```

Next: add schemas + validators for [CorrectionNotice](#) and [RollbackPlan](#).

Proceed with **schemas + validators** for [CorrectionNotice](#) and [RollbackPlan](#).

## [schemas/maplibre/perf-correction-notice.schema.json](#)

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/perf-correction-notice/v1",
 "title": "MapLibre Perf Correction Notice",
 "type": "object",
 "required": [
 "object_type",
```

```

 "schema_version",
 "domain",
 "notice_id",
 "created_utc",
 "trigger",
 "status",
 "affected_release_manifest_path",
 "reason",
 "required_action",
 "policy_posture"
],
 "additionalProperties": false,
 "properties": {
 "object_type": { "const": "CorrectionNotice" },
 "schema_version": { "const": "v1" },
 "domain": { "const": "maplibre" },
 "notice_id": { "type": "string", "minLength": 8 },
 "created_utc": { "type": "string", "format": "date-time" },
 "trigger": { "const": "perf_or_render_governance_failure" },
 "status": { "enum": ["draft", "review", "published"] },
 "affected_release_manifest_path": { "type": "string", "minLength": 1 },
 "reason": { "type": "string", "minLength": 1 },
 "required_action": { "type": "string", "minLength": 1 },
 "policy_posture": { "enum": ["review_required", "restricted"] }
 }
}

```

## schemas/maplibre/perf-rollback-plan.schema.json

```

{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/perf-rollback-plan/v1",
 "title": "MapLibre Perf Rollback Plan",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "domain",
 "rollback_id",
 "created_utc",
 "status",

```

```

 "affected_release_manifest_path",
 "rollback_strategy",
 "verification_command",
 "policy_posture"
],
 "additionalProperties": false,
 "properties": {
 "object_type": { "const": "RollbackPlan" },
 "schema_version": { "const": "v1" },
 "domain": { "const": "maplibre" },
 "rollback_id": { "type": "string", "minLength": 8 },
 "created_utc": { "type": "string", "format": "date-time" },
 "status": { "enum": ["draft", "review", "approved"] },
 "affected_release_manifest_path": { "type": "string", "minLength": 1 },
 "rollback_strategy": { "type": "string", "minLength": 1 },
 "verification_command": { "type": "string", "minLength": 1 },
 "policy_posture": { "enum": ["review_required", "restricted"] }
 }
}

```

## tools/validators/maplibre/validate\_perf\_correction\_notice.py

```
#!/usr/bin/env python3
```

```

import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator, FormatChecker

SCHEMA_PATH = Path("schemas/maplibre/perf-correction-notice.schema.json")

def main() -> int:
 if len(sys.argv) != 2:
 print(
 "usage: validate_perf_correction_notice.py <correction-notice.json>",
 file=sys.stderr
)
 return 2

 notice_path = Path(sys.argv[1])

```

```

if not notice_path.exists():
 print(f"DENY: correction notice not found: {notice_path}")
 return 1

schema = json.loads(SCHEMA_PATH.read_text())
notice = json.loads(notice_path.read_text())

validator = Draft202012Validator(schema, format_checker=FormatChecker())
errors = sorted(validator.iter_errors(notice), key=lambda e: e.path)

if errors:
 print("DENY: invalid MapLibre perf CorrectionNotice")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

manifest_path = Path(notice["affected_release_manifest_path"])

if not manifest_path.exists():
 print(f"DENY: affected release manifest missing: {manifest_path}")
 return 1

print("ANSWER: MapLibre perf CorrectionNotice valid")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## tools/validators/maplibre/validate\_perf\_rollback\_plan.py

```
#!/usr/bin/env python3
```

```

import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator, FormatChecker

SCHEMA_PATH = Path("schemas/maplibre/perf-rollback-plan.schema.json")

def main() -> int:

```

```

if len(sys.argv) != 2:
 print(
 "usage: validate_perf_rollback_plan.py <rollback-plan.json>",
 file=sys.stderr
)
 return 2

rollback_path = Path(sys.argv[1])

if not rollback_path.exists():
 print(f"DENY: rollback plan not found: {rollback_path}")
 return 1

schema = json.loads(SCHEMA_PATH.read_text())
rollback = json.loads(rollback_path.read_text())

validator = Draft2012Validator(schema, format_checker=FormatChecker())
errors = sorted(validator.iter_errors(rollback), key=lambda e: e.path)

if errors:
 print("DENY: invalid MapLibre perf RollbackPlan")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

manifest_path = Path(rollback["affected_release_manifest_path"])

if not manifest_path.exists():
 print(f"DENY: affected release manifest missing: {manifest_path}")
 return 1

print("ANSWER: MapLibre perf RollbackPlan valid")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI failure branch

- name: Build correction and rollback artifacts on failure  
 if: failure()  
 run: |



```
node scripts/build-maplibre-perf-correction-and-rollback.mjs || true
```

- name: Validate correction notice on failure

if: failure()

run: |

```
python3 tools/validators/maplibre/validate_perf_correction_notice.py \
 artifacts/perf/correction-notice.maplibre-perf.json
```

- name: Validate rollback plan on failure

if: failure()

run: |

```
python3 tools/validators/maplibre/validate_perf_rollback_plan.py \
 artifacts/perf/rollback-plan.maplibre-perf.json
```

Next: add a **failure artifact bundle manifest** so failed releases also have inspectable evidence, not just passing candidates.

Next: **failure artifact bundle manifest**.

This gives failed/rejected renderer runs their own governed evidence bundle.

## Add role enum values

In `perf-release-manifest.schema.json`:

```
"failure_bundle"
```

## Add

artifacts/perf/

└─ failure-bundle.maplibre-perf.json

schemas/maplibre/

└─ perf-failure-bundle.schema.json

tools/validators/maplibre/

└─ validate\_perf\_failure\_bundle.py

scripts/

└─ build-maplibre-perf-failure-bundle.mjs

## scripts/build-maplibre-perf-failure-bundle.mjs

S

```
#!/usr/bin/env node

import crypto from "node:crypto";
import fs from "node:fs/promises";

const outPath = "artifacts/perf/failure-bundle.maplibre-perf.json";

const candidatePaths = [
 "artifacts/perf/run-receipt.json",
 "artifacts/perf/perf-results.json",
 "artifacts/perf/release-manifest.maplibre-perf.json",
 "artifacts/perf/render-diff/render-diff-report.json",
 "artifacts/perf/correction-notice.maplibre-perf.json",
 "artifacts/perf/rollback-plan.maplibre-perf.json"
];

function sha256(bytes) {
 return "sha256:" + crypto.createHash("sha256").update(bytes).digest("hex");
}

const artifacts = [];

for (const path of candidatePaths) {
 try {
 const bytes = await fs.readFile(path);
 artifacts.push({
 path,
 sha256: sha256(bytes)
 });
 } catch {
 // Artifact may not exist depending on failure point.
 }
}

const bundle = {
 object_type: "FailureBundle",
 schema_version: "v1",
 domain: "maplibre",
 bundle_id: `maplibre-perf-failure-${new Date().toISOString().replace(":", "-")}`,
 created_utc: new Date().toISOString(),
}
```

```

 trigger: "perf_or_render_governance_failure",
 status: "captured",
 artifacts
 };

 await fs.writeFile(outPath, JSON.stringify(bundle, null, 2));

 console.log("ANSWER: wrote MapLibre perf failure bundle");
 console.log(`- ${outPath}`);

```

## schemas/maplibre/perf-failure-bundle.schema.json

```

{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "kfm://schema/maplibre/perf-failure-bundle/v1",
 "title": "MapLibre Perf Failure Bundle",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "domain",
 "bundle_id",
 "created_utc",
 "trigger",
 "status",
 "artifacts"
],
 "additionalProperties": false,
 "properties": {
 "object_type": { "const": "FailureBundle" },
 "schema_version": { "const": "v1" },
 "domain": { "const": "maplibre" },
 "bundle_id": { "type": "string", "minLength": 8 },
 "created_utc": { "type": "string", "format": "date-time" },
 "trigger": { "const": "perf_or_render_governance_failure" },
 "status": { "const": "captured" },
 "artifacts": {
 "type": "array",
 "minItems": 1,
 "items": {
 "type": "object",

```

```

 "required": ["path", "sha256"],
 "additionalProperties": false,
 "properties": {
 "path": { "type": "string", "minLength": 1 },
 "sha256": {
 "type": "string",
 "pattern": "^sha256:[a-f0-9]{64}$"
 }
 }
 }
}
}
}
}
}

```

## tools/validators/maplibre/validate\_perf\_failure\_bundle.py

```
#!/usr/bin/env python3
```

```

import hashlib
import json
import sys
from pathlib import Path
from jsonschema import Draft202012Validator, FormatChecker

SCHEMA_PATH = Path("schemas/maplibre/perf-failure-bundle.schema.json")

def sha256_file(path: Path) -> str:
 h = hashlib.sha256()
 with path.open("rb") as f:
 for chunk in iter(lambda: f.read(65536), b''):
 h.update(chunk)
 return "sha256:" + h.hexdigest()

def main() -> int:
 if len(sys.argv) != 2:
 print("usage: validate_perf_failure_bundle.py <failure-bundle.json>", file=sys.stderr)
 return 2

 bundle_path = Path(sys.argv[1])

 if not bundle_path.exists():

```

```

 print(f"DENY: failure bundle not found: {bundle_path}")
 return 1

schema = json.loads(SCHEMA_PATH.read_text())
bundle = json.loads(bundle_path.read_text())

validator = Draft202012Validator(schema, format_checker=FormatChecker())
errors = sorted(validator.iter_errors(bundle), key=lambda e: e.path)

if errors:
 print("DENY: invalid MapLibre perf FailureBundle")
 for error in errors:
 loc = ".".join(str(p) for p in error.path) or "<root>"
 print(f"- {loc}: {error.message}")
 return 1

for artifact in bundle["artifacts"]:
 path = Path(artifact["path"])

 if not path.exists():
 print(f"DENY: failure artifact missing: {path}")
 return 1

 actual = sha256_file(path)

 if actual != artifact["sha256"]:
 print(f"DENY: failure artifact hash mismatch: {path}")
 print(f"- expected={artifact['sha256']}")
 print(f"- actual={actual}")
 return 1

print("ANSWER: MapLibre perf FailureBundle valid")
return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## CI failure branch

```

- name: Build failure bundle on failure
 if: failure()
 run: |
 node scripts/build-maplibre-perf-failure-bundle.mjs || true

```

```
- name: Validate failure bundle on failure
 if: failure()
 run: |
 python3 tools/validators/maplibre/validate_perf_failure_bundle.py \
 artifacts/perf/failure-bundle.maplibre-perf.json
```

Next after this: **add upload-artifact steps** so passing ProofPacks and failing FailureBundles are retained by CI.

Proceed with **CI artifact retention** for both passing and failing runs.

## Passing artifact upload

```
- name: Upload MapLibre perf ProofPack artifacts
 if: success()
 uses: actions/upload-artifact@v4
 with:
 name: maplibre-perf-proofpack
 path: |
 artifacts/perf/proof-pack.maplibre-perf.json
 artifacts/perf/release-manifest.maplibre-perf.json
 artifacts/perf/run-receipt.json
 artifacts/perf/run-receipt.dsse.json
 artifacts/perf/run-receipt.sha256
 artifacts/perf/perf-results.json
 artifacts/perf/frame-times/
 artifacts/perf/screenshots/
 artifacts/perf/render-diff/
 config/maplibre/perf-envelope.v1.json
 if-no-files-found: error
 retention-days: 30
```

## Failing artifact upload

```
- name: Upload MapLibre perf failure artifacts
 if: failure()
 uses: actions/upload-artifact@v4
 with:
 name: maplibre-perf-failure-bundle
 path: |
 artifacts/perf/failure-bundle.maplibre-perf.json
```

artifacts/perf/correction-notice.maplibre-perf.json  
artifacts/perf/rollback-plan.maplibre-perf.json  
artifacts/perf/release-manifest.maplibre-perf.json  
artifacts/perf/run-receipt.json  
artifacts/perf/perf-results.json  
artifacts/perf/frame-times/  
artifacts/perf/screenshots/  
artifacts/perf/render-diff/  
if-no-files-found: warn  
retention-days: 30

## Recommended final CI tail

- name: Build failure bundle on failure  
if: failure()  
run: |  
node scripts/build-maplibre-perf-failure-bundle.mjs || true
- name: Validate failure bundle on failure  
if: failure()  
run: |  
python3 tools/validators/maplibre/validate\_perf\_failure\_bundle.py \  
artifacts/perf/failure-bundle.maplibre-perf.json || true
- name: Upload MapLibre perf ProofPack artifacts  
if: success()  
uses: actions/upload-artifact@v4  
with:  
name: maplibre-perf-proofpack  
path: artifacts/perf/  
if-no-files-found: error  
retention-days: 30
- name: Upload MapLibre perf failure artifacts  
if: failure()  
uses: actions/upload-artifact@v4  
with:  
name: maplibre-perf-failure-bundle  
path: artifacts/perf/  
if-no-files-found: warn  
retention-days: 30

Next: add a short `docs/quality/maplibre-perf-governance.md` explaining the lane, artifacts, gates, and failure workflow.

Next: add a `package.json` script alias:

```
{
 "scripts": {
 "maplibre:perf": "node scripts/maplibre-smoke-perf.mjs",
 "maplibre:perf:govern": "python3 tools/validators/maplibre/validate_perf_governance.py"
 }
}
```

Next: add the **actual dependency entries** needed by the MapLibre perf lane.

Update root `package.json` with dev dependencies:

```
{
 "devDependencies": {
 "pixelmatch": "^7.1.0",
 "playwright": "^1.56.0",
 "pngjs": "^7.0.0"
 }
}
```

Then install/update lockfile:

```
npm install
npx playwright install --with-deps chromium
```

Then run the local chain:

```
python3 -m http.server 8080 &
make maplibre-perf
make maplibre-govern
make maplibre-proof
```

After that, the next file to add is:

`config/maplibre/perf-envelope.v1.json`



Here's a quick, practical set of Cesium 3D Tiles "smoke-test" knobs you can flip to spot perf + memory issues fast, without changing your pipeline.

---

## 1) Level-of-detail pressure

Raise the on-screen error budget to request fewer, coarser tiles (less network/decodes, more FPS; lower detail up close).

```
// Fast-first pass
tileset.maximumScreenSpaceError = 32; // try 16 → 24 → 32
```

**Use when:** you want a speedy sanity check or to prove the scene is CPU/IO bound, not GPU bound.

---

## 2) Refinement mode (overdraw vs. replace)

Prefer **REPLACE** for terrain/mesh stacks so parents disappear when children load (avoids additive draw).

```
// If authoring the tileset
// tileset.json → "refine": "REPLACE"

// If inspecting at runtime
console.log(tileset.refine); // expect "REPLACE" for mesh trees
```

**Use ADD** only when features must accumulate (e.g., point/label aggregations).

---

## 3) Memory & cache caps (predictable VRAM/heap)

Set hard caps so eviction kicks in early and Cesium raises the effective SSE automatically.

```
const tileset = await Cesium.Cesium3DTileset.fromUrl(url, {
 // ~256 MB working set; allow brief overflow for bursty views
 cacheBytes: 256 * 1024 * 1024,
 maximumCacheOverflowBytes: 64 * 1024 * 1024,
 // or: maximumNumberOfLoadedTiles: 600
});
```

**Watch:** tile count, loaded bytes, and frame stutter as you pan/orbit.

---

## How to test quickly

- Open Cesium Sandcastle → a 3D Tiles example.
- Toggle the three controls above (SSE, refine mode, cache caps).
- Log basics each second:

```
setInterval(() => {
 const d = tileset.statistics;
 console.log({
 tilesLoaded: d.numberOfTilesWithContentReady,
 tilesInView: d.numberOfTilesInView,
 bytes: d.totalMemoryUsageInBytes,
 fps: Math.round(1 / viewer.clock._tickDelta)
 });
}, 1000);
```

---

## KFM-ish guidance

- Treat these as **inspection levers**, not publication policy.
- Record the knob values + observations as an **Inspectable Claim** in WORK.
- If a scene only stabilizes with high SSE or tight caches, flag the tileset for **LOD/content tuning** before promotion.

Want me to drop this into a repo-ready “Map/3D Smoke Test” snippet with a tiny UI (SSE slider, cache inputs, live stats)?

## scripts/cesium-smoke-test.mjs

```
#!/usr/bin/env node
```

```
/**
 * KFM Cesium 3D Tiles Smoke Test
 * -----
 * PURPOSE
 * Fast inspection utility for:
 * - FPS stability
 * - tile churn
```

- \* - memory pressure
- \* - LOD behavior
- \* - cache eviction behavior
- \*
- \* POSTURE
- \* - inspection only
- \* - no publication authority
- \* - no truth mutation
- \* - no release promotion
- \*
- \* OUTPUT
- \* stdout JSON summaries
- \* optional NDJSON artifact
- \*
- \* USAGE
- \* node scripts/cesium-smoke-test.mjs \
- \* --url=https://example.com/tileset.json \
- \* --sse=24 \
- \* --cache-mb=256 \
- \* --duration-sec=30
- \*/

```
import fs from "node:fs";
import path from "node:path";
import process from "node:process";

function parseArgs(argv) {
 const out = {};

 for (const arg of argv.slice(2)) {
 if (!arg.startsWith("--")) continue;

 const [k, v] = arg.slice(2).split("=");

 out[k] = v ?? true;
 }

 return out;
}

const args = parseArgs(process.argv);

const config = {
 url: args.url || null,
```

```

sse: Number(args.sse ?? 24),
cacheMb: Number(args["cache-mb"] ?? 256),
overflowMb: Number(args["overflow-mb"] ?? 64),
durationSec: Number(args["duration-sec"] ?? 30),
sampleMs: Number(args["sample-ms"] ?? 1000),
output:
 args.output ||
 path.join(
 process.cwd(),
 "artifacts",
 "cesium",
 `smoke-${Date.now()}.ndjson`
)
};

if (!config.url) {
 console.error(
 JSON.stringify({
 outcome: "ERROR",
 reason: "missing_tileset_url",
 expected: "--url=https://example.com/tileset.json"
 }, null, 2)
);

 process.exit(1);
}

fs.mkdirSync(path.dirname(config.output), { recursive: true });

console.log(
 JSON.stringify({
 outcome: "START",
 config
 }, null, 2)
);

/**
 * NOTE
 * ----
 * Cesium runtime smoke tests normally execute in browser/WebGL contexts.
 *
 * This script is intentionally lightweight and CI-safe.
 *
 * Integrate with:

```

```
* - Playwright
* - Puppeteer
* - Electron
* - headless Chromium
*
* for real renderer metrics.
*/
```

```
const start = Date.now();
```

```
let sampleCount = 0;
```

```
const interval = setInterval(() => {
 sampleCount += 1;
```

```
/**
 * Placeholder metrics.
 *
 * Replace these with browser-collected runtime stats:
 * tilesLoaded
 * memoryBytes
 * fps
 * networkRequests
 * tileEvictions
 * gpuMemory
 */
```

```
const mock = {
 ts: new Date().toISOString(),
 elapsed_sec: Math.round((Date.now() - start) / 1000),
```

```
 runtime: {
 maximumScreenSpaceError: config.sse,
 cacheBytes: config.cacheMb * 1024 * 1024,
 maximumCacheOverflowBytes:
 config.overflowMb * 1024 * 1024
 },
```

```
 statistics: {
 fps_estimate: 58,
 tiles_loaded: 412,
 tiles_in_view: 96,
 estimated_memory_mb: 233,
 evictions: 12
```

```

 },

 posture: {
 refine_expected: "REPLACE",
 inspection_only: true
 }
 };

 fs.appendFileSync(
 config.output,
 JSON.stringify(mock) + "\n"
);

 console.log(JSON.stringify(mock));

 const elapsed =
 (Date.now() - start) / 1000;

 if (elapsed >= config.durationSec) {
 clearInterval(interval);

 console.log(
 JSON.stringify({
 outcome: "COMPLETE",
 samples: sampleCount,
 output: config.output
 }, null, 2)
);
 }
}, config.sampleMs);

```

## Add to **package.json**

```

{
 "scripts": {
 "cesium:smoke": "node scripts/cesium-smoke-test.mjs"
 }
}

```

## Example

```
npm run cesium:smoke -- \
```

```
--url=https://example.com/tileset.json \
--sse=32 \
--cache-mb=192 \
--duration-sec=20
```

## Expected Next Files

After this, the normal KFM progression would be:

scripts/

cesium-smoke-test.mjs

browser/

cesium-playwright-runner.mjs

collect-cesium-stats.js

fixtures/

cesium/

smoke/

sample-config.json

expected-thresholds.json

schemas/

cesium/

smoke-run.schema.json

artifacts/

cesium/

.gitkeep

docs/

map/

CESIUM\_SMOKE\_TESTS.md

tests/

smoke/

cesium-smoke.test.mjs

And yes — for your earlier question:

- reusable canonical fixture payloads → [fixtures/...](#)
- test-framework-local fixtures → [tests/fixtures/...](#)

For KFM governance-oriented renderer verification, the preferred split is usually:

```
fixtures/ # canonical reusable inputs
tests/fixtures/ # framework-specific harness data
```

because the same fixtures can later feed:

- validators
- CI attestations
- renderer parity checks
- policy gates
- replay harnesses
- EvidenceBundle verification
- regression sweeps

Here's a crisp, high-value snapshot of what's currently shaking out upstream so you can fold it into your Master PDF with confidence — there are **runtime, versioning, and tooling provenance changes in flight that directly affect how GL JS, PMTiles, and their docs behave**.

## What's new in MapLibre GL JS

The final **v5 series** — **v5.22.0** → **v5.24.0** — **has been released** with performance tweaks, bug fixes, and a few minor public API additions as the last of the v5 line. These are now stable and documented before the jump. ([MapLibre](#))

At the same time, **pre-releases of v6 are rolling out**, encompassing breaking changes like:

- **WebGL1 support being removed** in favor of pure WebGL2 to resolve long-standing bugs and broaden compatibility with modern tooling.
- **Migration from CommonJS to ESM** for the build system and module import path.
- Glimmer of a **migration guide** in progress as breaking items stabilize. ([MapLibre](#))

This is fundamental — treat these as upstream **runtime contract changes**, not just bumping a version number.

## PMTiles ecosystem: spec remains stable, tooling evolves

The PMTiles **specification is solidly at version 3** and remains the definitive disk layout and tile index contract (header + directory + metadata + tile data). It's written in normative RFC-style and designed for single-file, cloud-optimized usage. ([GitHub](#))

Meanwhile, the **go-pmtiles** tool — the canonical CLI for creating and manipulating PMTiles archives — recently saw an active release, **v1.30.2 (Apr 22, 2026)**. This release



includes fixes like correct MIME handling for MapLibre vector tile types and dependency updates. ([GitHub](#))

Record **both the tag (v1.30.2) and binary provenance** if you're tracking tool artifacts alongside spec evolution.

## Embedded examples in the official MapLibre docs

The *MapLibre docs* now explicitly include a *PMTiles source/protocol usage example* for their GL JS renderer. That means the docs aren't just talking about vector tiles in general anymore — they show concrete ways to plug in a PMTiles archive as a source. This matters for any canonical examples you include in your PDF.

## A provable spec provenance workflow

To eventually anchor these release markers cryptographically, you can canonicalize structured metadata for each release and compute a stable hash using JSON Canonicalization Scheme (JCS), e.g.:

```
python - <<'PY'
import json,hashlib
obj=json.load(open('release.json'))
print(hashlib.sha256(json.dumps(obj,sort_keys=True,separators=(',',':')).encode()).hexdigest())
PY
```

Keep the original `release.json` with fields like `name`, `tag`, `commit`, `date`, `url`, `notes`, and store that hex `spec_hash` next to release notes as a **re-computable evidence blob** for downstream verification.

---

### Why this matters right now

You're about to commit changes to a *Master PDF* that should not only summarize versions but also embed **runtime and spec-level contracts** that downstream CI (and humans) can verify.

- The **GL JS v5→v6 transition is breaking** and tracked upstream. ([MapLibre](#))
- The **PMTiles spec v3 is still normative**, and the toolchain around it is actively maintained. ([GitHub](#))
- Explicit examples in docs now make PMTiles a first-class source type in GL JS — worth including as an actionable snippet.

Let me know if you want concise links to the specific docs and releases you should embed.

# Canonical upstream references to embed

## MapLibre GL JS release stream

- [MapLibre April 2026 Newsletter](#)

Covers:

- v5.22.0 → v5.24.0 stabilization
- v6 pre-release trajectory
- WebGL1 removal
- ESM migration
- migration-guide work in progress

- [MapLibre GL JS Releases \(GitHub\)](#)

Use this as the authoritative release/tag ledger for:

- tag names
  - commit provenance
  - changelog deltas
  - release dates
  - release assets
- 

## PMTiles specification + tooling provenance

### Normative PMTiles spec

- [PMTiles Specification v3](#)

This is the canonical spec reference to cite in your PDF for:

- archive layout
- directory structure
- header semantics
- tile addressing
- cloud-optimized access model

### Canonical PMTiles tooling

- [go-pmtiles Releases](#)

Current notable release:

- v1.30.2 (Apr 22, 2026)

- [go-pmtiles Repository](#)
- 

## PMTiles + MapLibre integration examples

## Official MapLibre PMTiles example

- [MapLibre PMTiles Protocol Example](#)

This is important because it officially demonstrates:

- protocol registration
- PMTiles source wiring
- MapLibre runtime integration patterns
- browser-side archive consumption

---

## Recommended “Version Provenance” appendix block for your PDF

You should strongly consider a compact appendix table like this:

Component	Version	Date	Provenance URL	spec_hash
MapLibre GL JS	v5.24.0	2026-04	GitHub Releases	SHA256
MapLibre GL JS	v6 prerelease	2026-04	Newsletter + GitHub	SHA256
PMTiles Spec	v3	stable	spec.md	SHA256
go-pmtiles	v1.30.2	2026-04-22	GitHub Releases	SHA256

---

## Recommended attestation metadata blob

For KFM governance alignment, store release evidence like:

```
{
 "component": "maplibre-gl-js",
 "version": "v5.24.0",
 "release_date": "2026-04-18",
 "source_url": "https://github.com/maplibre/maplibre-gl-js/releases",
 "commit": "NEEDS_VERIFICATION",
 "retrieved_at": "2026-05-14T00:00:00Z",
 "spec_hash": "sha256:..."
}
```

That aligns extremely well with your existing:

- EvidenceBundle posture
  - ReleaseManifest lineage
  - spec\_hash validation doctrine
  - reproducible governance trails
  - CI attestation workflows
- 

## **Suggested new Master PDF sections**

### **1. Runtime Compatibility Matrix**

Track:

- WebGL1 support status
- WebGL2 requirements
- ESM/CommonJS posture
- Browser compatibility
- PMTiles protocol compatibility

### **2. PMTiles Operational Provenance**

Track:

- spec version
- archive creator version
- generation command
- MIME correctness
- archive validation status
- byte-range proof support

### **3. Renderer Migration Ledger**

Track:

- MapLibre v5 → v6 breaking changes
- deprecated APIs
- required build-system shifts
- worker/runtime impacts
- downstream KFM adapter implications

## 4. Governed Renderer Attestation

Include:

- DSSE envelope examples
- cosign signatures
- manifest hash trees
- runtime verification receipts
- render-diff evidence chains

These additions would materially strengthen the “Master MapLibre Components-Functions-Features.pdf” from:

useful reference

into:

governed renderer/specification operations manual

which matches the direction your KFM doctrine has been evolving toward.

You’ll want to know this because the **MapLibre ecosystem is in a moment of rapid evolution — and the choices made now around GL APIs, native bindings, and tile formats will ripple into projects like KFM’s Map and Focus UIs.**

Here’s what’s coming out of the April 2026 maplibre.org newsletter — with clues about feature direction and compatibility risk for both web and native: ([MapLibre](#))

• **New GL JS trajectory:** The project is actively progressing toward a v6 release of **MapLibre GL JS**, including notable breaking changes and modernization. Early pre-releases show *WebGL 1 being dropped in favor of WebGL2*, ESM-only builds, and API tweaks that could affect map initialization, module loading, and interactive surfaces — all things that matter for smoke tests and incremental upgrades. ([GitHub](#))

• **Mobile and native bindings:** There are ongoing overhauls in **React-Native and Flutter bindings** as part of the April 2026 update, reflecting growing parity efforts between web and native stacks. Expect subtle API shifts in how maps are constructed and styled, which may surface as unexpected behaviors in mobile smoke tests if you’re still on older releases. ([MapLibre](#))

• **3D tile research taking shape:** Work on a **native MapLibre Tile (MLT) 3D format** is underway, with design considerations aimed at optimizing a next-generation tile format for the ecosystem. This isn’t yet a finished standard, but it signals community interest in richer 3D data support beyond classic vector tiles — and may influence roadmap decisions for heavier 3D use cases. ([MapLibre](#))

- **Community momentum and ecosystem health:** The newsletters reflect active contributions across platforms, from web GL engines to native SDKs, and show a sustained community pushing toward feature coherence and broader platform coverage. ([MapLibre](#))

If you're keeping smoke tests green or planning upgrades around MapLibre flows in your apps, these updates paint a picture of **modernizing APIs, platform alignment, and early 3D ambitions — all of which are worth tracking as they land.**

Here's a crisp, audited-by-default CI pattern you can drop into KFM to make releases verifiable and promotable without manual heroics.

---

## CI recipe: spec-hash → DSSE attestation → signed run-receipt

### What it is:

A tiny, deterministic chain that (1) fingerprints your release spec/sidecar, (2) issues a DSSE/in-toto attestation bound to the artifact, and (3) emits a signed `run_receipt.json` linking everything so promotion gates can trust—and audit—the move to **PUBLISHED**.

### 1) Compute a deterministic `spec_hash`

- Serialize your sidecar/spec (JSON) with JCS (canonical JSON) and sha256 it.
- This gives you a stable identity for “what we meant to release,” separate from the big binary (e.g., `.pmtiles`).

```
SPEC_HASH=$(jq -cS . sidecar.json | openssl dgst -sha256 -binary | xxd -p -c256)
echo "spec_hash=$SPEC_HASH"
```

### 2) Produce a DSSE attestation (cosign)

- Attach the *predicate* (facts you want to assert—e.g., contents of the sidecar/spec, build params) to the artifact digest.
- Cosign uses DSSE envelopes and in-toto style attestations—perfect for CI.

```
assumes: COSIGN_KEY set (key ref or KMS), $ARTIFACT_URI like oci://... or registry URI
cat > predicate.json <<'JSON'
{
 "type": "kfm.release.v1",
```

```

"spec_sidecar": "sidecar.json",
"spec_hash": "__SPEC_HASH__",
"notes": "promotion candidate"
}
JSON

```

```

tmp=$(mktemp)
jq --arg h "$SPEC_HASH" '.spec_hash=$h' predicate.json > "$tmp" && mv "$tmp"
predicate.json

```

```

cosign attest \
 --key "$COSIGN_KEY" \
 --predicate predicate.json \
 "$ARTIFACT_URI"

```

```

Later (gate/verify job)
cosign verify-attestation \
 --key "$COSIGN_PUB" \
 "$ARTIFACT_URI"

```

### 3) Emit a signed **run\_receipt.json**

- Store alongside the promoted artifact. This is the glue your gates read.

```

RUN_RECEIPT=run_receipt.json
jq -n \
 --arg dec_id "dec-$(date +%s)" \
 --arg uri "$ARTIFACT_URI" \
 --arg sh "$SPEC_HASH" \
 --arg att "cosign-attest:sha256:$(cosign triangulate "$ARTIFACT_URI")" \
 --arg actor "ci:runner/$(hostname)" \
 --arg ts "$(date -u +%Y-%m-%dT%H:%M:%SZ)" '
{
 decision_id: $dec_id,
 artifact_uri: $uri,
 spec_hash: $sh,
 attestation_ref: $att,
 actor: $actor,
 status: "signed",
 timestamp: $ts
}' > "$RUN_RECEIPT"

```

```
cosign sign-blob --key "$COSIGN_KEY" --output-signature "$RUN_RECEIPT.sig"
"$RUN_RECEIPT"
```

## 4) Gate logic (promotion)

- Require all of the below before flipping **CATALOG/TRIPLET** → **PUBLISHED**:
  - Attestation verifies for the artifact digest.
  - `spec_hash` in attestation equals `run_receipt.spec_hash`.
  - `run_receipt` signature verifies with `$COSIGN_PUB`.
  - Policy passes (rights/sensitivity/validation receipts present).
  - Rollback plan & correction path recorded.

## 5) Partial-read verification for big PMTiles (fast client checks)

For byte-range/streaming verification without hashing the whole file:

- Add a **BLAKE3/bao** root hash (or outboard proof) to your sidecar.
- Clients can validate random reads against the root quickly; servers can serve verified ranges.

Sidecar snippet:

```
{
 "pmtiles_uri": "s3://ci-builds/tiles/20260512.pmtiles",
 "spec_hash": "<sha256-of-jcs>",
 "integrity": {
 "algo": "blake3-bao",
 "root": "<hex-root>",
 "outboard_uri": "s3://ci-builds/tiles/20260512.pmtiles.bao" // optional
 }
}
```

Client sketch (pseudo):

```
fetch bytes [start,end), fetch bao proof, verify with root
verify_bao_range --root "$B3_ROOT" --file partial.bin --proof proof.bao
```

## 6) Minimal folder layout (proposal)



```
artifacts/
20260512/
 tiles.pmtiles
 sidecar.json
 run_receipt.json
 run_receipt.json.sig
 tiles.pmtiles.bao # optional
```

## 7) CI checklist (drop into your pipeline)

- Compute JCS `spec_hash`
- `cosign attest` with predicate (includes `spec_hash`)
- Verify attestation in a separate job
- Emit & sign `run_receipt.json`
- Policy/validator receipts present (fail-closed)
- (PMTiles) bao root/proof published
- Only then: promotion step toggles **PUBLISHED**

If you want, I can turn this into a repo-ready `tools/ci/` slice (scripts + validators + a tiny gate) aligned to your KFM invariants.

## Proposed repo-ready slice: `tools/ci/`

This aligns with your KFM promotion doctrine:

- deterministic `spec_hash`
- DSSE/in-toto attestation
- signed `RunReceipt`
- fail-closed promotion gates
- PMTiles partial verification support
- explicit rollback/correction linkage
- separation of validation vs policy authority

---

## Suggested layout

```
tools/
└─ ci/
```

```
|— README.md
|— compute_spec_hash.sh
|— build_predicate.sh
|— sign_run_receipt.sh
|— verify_release.sh
|— verify_bao_range.sh
|— schemas/
| |— run_receipt.schema.json
| |— predicate.schema.json
|— fixtures/
| |— valid/
| | |— sidecar.json
| | |— predicate.json
| | |— run_receipt.json
| |— invalid/
| | |— spec_hash_mismatch/
| | |— missing_attestation/
| | |— unsigned_receipt/
|— examples/
| |— github-actions.yml
| |— release.env.example
```

---

## tools/ci/README.md

### # KFM CI Attestation Lane

Governed CI helpers for deterministic release verification and promotion gating.

### ## Purpose

This lane exists to:

- compute deterministic `spec\_hash` identities
- bind attestations to release artifacts
- emit signed `RunReceipt` objects
- support fail-closed promotion gates
- preserve correction + rollback traceability

These helpers do NOT:

- decide publication policy

- override steward review
- mutate canonical truth
- bypass release governance
- expose RAW/WORK/QUARANTINE artifacts

---

### ## Promotion chain

```mermaid

flowchart LR

```
A[sidecar.json] --> B[spec_hash]
B --> C[DSSE attestation]
C --> D[RunReceipt]
D --> E[Policy + Validation Gates]
E --> F[PUBLISHED]
```

```

---

### ## Core invariants

Invariant	Requirement
Determinism	Canonical JSON serialization
Integrity	sha256 spec\_hash
Authenticity	cosign signature
Verifiability	independent verify step
Auditability	signed RunReceipt
Reversibility	rollback linkage required
Fail-closed	unresolved verification blocks promotion

---

### ## Runtime outcomes

- ANSWER
- ABSTAIN
- DENY
- ERROR

Promotion MUST fail closed on ERROR or unresolved verification state.

---

## ## Verification chain

1. Compute canonical `spec\_hash`
2. Create DSSE attestation
3. Verify attestation
4. Generate signed RunReceipt
5. Validate receipts + schemas
6. Evaluate policy
7. Permit governed publication transition

---

## ## PMTiles integrity

Large PMTiles artifacts MAY include:

- BLAKE3 root hash
- bao outboard proofs
- partial byte-range verification

Visual rendering is NOT evidentiary proof.

---

# compute\_spec\_hash.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

```
INPUT="${1:-sidecar.json}"
```

```
jq -cS . "$INPUT" \
| openssl dgst -sha256 -binary \
| xxd -p -c256
```

---

# build\_predicate.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

```
SPEC_HASH="$1"
```

```
jq -n \
 --arg h "$SPEC_HASH" \
 --arg ts "$(date -u +%Y-%m-%dT%H:%M:%SZ)" '
{
 type: "kfm.release.v1",
 spec_hash: $h,
 generated_at: $ts
}'
```

---

## sign\_run\_receipt.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

```
: "${COSIGN_KEY:?missing COSIGN_KEY}"
```

```
ARTIFACT_URI="$1"
SPEC_HASH="$2"
```

```
jq -n \
 --arg uri "$ARTIFACT_URI" \
 --arg sh "$SPEC_HASH" \
 --arg ts "$(date -u +%Y-%m-%dT%H:%M:%SZ)" '
{
 object_type: "RunReceipt",
 schema_version: "v1",
 artifact_uri: $uri,
 spec_hash: $sh,
 timestamp: $ts,
 status: "signed"
}' > run_receipt.json
```

```
cosign sign-blob \
 --key "$COSIGN_KEY" \
 --output-signature run_receipt.json.sig \
 run_receipt.json
```

---

## verify\_release.sh

```
#!/usr/bin/env bash
set -euo pipefail

: "${COSIGN_PUB:?missing COSIGN_PUB}"

ARTIFACT_URI="$1"

cosign verify-attestation \
 --key "$COSIGN_PUB" \
 "$ARTIFACT_URI"

cosign verify-blob \
 --key "$COSIGN_PUB" \
 --signature run_receipt.json.sig \
 run_receipt.json

echo "verification=PASS"
```

---

## schemas/run\_receipt.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "RunReceipt",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "artifact_uri",
 "spec_hash",
 "timestamp",
 "status"
],
 "properties": {
 "object_type": {
```

```
 "const": "RunReceipt"
 },
 "schema_version": {
 "const": "v1"
 },
 "artifact_uri": {
 "type": "string"
 },
 "spec_hash": {
 "type": "string"
 },
 "timestamp": {
 "type": "string"
 },
 "status": {
 "enum": [
 "signed",
 "revoked",
 "superseded"
]
 }
}
```

---

## Example GitHub Actions slice

`examples/github-actions.yml`

name: release-attestation

on:

workflow\_dispatch:

jobs:

attest:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Install cosign

uses: sigstore/cosign-installer@v3

- name: Compute spec hash

run: |

```
chmod +x tools/ci/compute_spec_hash.sh
SPEC_HASH=$(tools/ci/compute_spec_hash.sh sidecar.json)
echo "SPEC_HASH=$SPEC_HASH" >> $GITHUB_ENV
```

- name: Build predicate

run: |

```
chmod +x tools/ci/build_predicate.sh
tools/ci/build_predicate.sh "$SPEC_HASH" > predicate.json
```

- name: Attest artifact

run: |

```
cosign attest \
 --key "${{ secrets.COSIGN_KEY }}" \
 --predicate predicate.json \
 "${{ vars.ARTIFACT_URI }}"
```

- name: Sign receipt

env:

COSIGN\_KEY: "\${{ secrets.COSIGN\_KEY }}"

run: |

```
chmod +x tools/ci/sign_run_receipt.sh
tools/ci/sign_run_receipt.sh \
 "${{ vars.ARTIFACT_URI }}" \
 "$SPEC_HASH"
```

- name: Verify release

env:

COSIGN\_PUB: cosign.pub

run: |

```
chmod +x tools/ci/verify_release.sh
tools/ci/verify_release.sh \
 "${{ vars.ARTIFACT_URI }}"
```

---

## Recommended next files after this slice

### High-priority



1. `tools/validators/releases/validate_run_receipt.py`
  2. `policy/publication/release_attestation.rego`
  3. `fixtures/releases/negative/*`
  4. `schemas/release_manifest.schema.json`
  5. `schemas/promotion_decision.schema.json`
- 

## Negative-path fixtures you should absolutely add

Fixture	Expected Outcome
spec hash mismatch	DENY
unsigned receipt	DENY
missing attestation	DENY
stale attestation	DENY
artifact digest mismatch	DENY
revoked receipt	DENY
rollback plan missing	DENY
unresolved EvidenceRef	DENY

---

## Strong recommendation

Treat:

- `RunReceipt`
- `PromotionDecision`
- `ReleaseManifest`
- `CorrectionNotice`

as separate governed object families.

Do NOT collapse them into one overloaded release blob.

That separation will matter later for:

- rollback
- legal defensibility
- temporal lineage
- public-safe evidence replay
- steward review
- partial revocation
- historical audit reconstruction

## tools/validators/releases/validate\_run\_receipt.py

```
#!/usr/bin/env python3
```

```
"""
```

Fail-closed RunReceipt validator.

Validation scope:

- schema shape
- required fields
- finite status set
- sha256 spec\_hash format
- timestamp presence
- optional linkage checks

This validator does NOT:

- decide publication policy
- verify cosign signatures
- authorize promotion

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import re
```

```
import sys
```

```
from pathlib import Path
```

```
STATUS = {"signed", "revoked", "superseded"}
```

```
SHA256_RE = re.compile(r"^[a-fA-F0-9]{64}$")
```

```
def fail(msg: str) -> None:
 print(f"DENY: {msg}")
 sys.exit(1)
```

```
def main() -> None:
 if len(sys.argv) != 2:
 fail("usage: validate_receipt.py <receipt.json>")
```

```
 path = Path(sys.argv[1])
```

```
 if not path.exists():
 fail("receipt missing")
```

```
 data = json.loads(path.read_text())
```

```
 required = [
 "object_type",
 "schema_version",
 "artifact_uri",
 "spec_hash",
 "timestamp",
 "status",
]
```

```
 for field in required:
 if field not in data:
 fail(f"missing field: {field}")
```

```
 if data["object_type"] != "RunReceipt":
 fail("invalid object_type")
```

```
 if data["schema_version"] != "v1":
 fail("invalid schema_version")
```

```
 if data["status"] not in STATUS:
 fail("invalid status")
```

```
 if not SHA256_RE.match(data["spec_hash"]):
```

```
 fail("invalid spec_hash format")

 print("ANSWER: validation_pass")

if __name__ == "__main__":
 main()
```

---

## tools/validators/releases/validate\_predicate.py

```
#!/usr/bin/env python3

from __future__ import annotations

import json
import re
import sys
from pathlib import Path

SHA256_RE = re.compile(r"^[a-fA-F0-9]{64}$")

def deny(msg: str):
 print(f"DENY: {msg}")
 sys.exit(1)

def main():
 if len(sys.argv) != 2:
 deny("usage error")

 path = Path(sys.argv[1])

 if not path.exists():
 deny("predicate missing")

 obj = json.loads(path.read_text())
```

```

required = [
 "type",
 "spec_hash",
 "generated_at",
]

for r in required:
 if r not in obj:
 deny(f"missing field: {r}")

if obj["type"] != "kfm.release.v1":
 deny("invalid type")

if not SHA256_RE.match(obj["spec_hash"]):
 deny("invalid spec hash")

print("ANSWER: predicate_valid")

if __name__ == "__main__":
 main()

```

---

## policy/publication/release\_attestation.rego

```

package kfm.publication.release_attestation

default allow := false

missing_field(obj, field) {
 not obj[field]
}

valid_status(status) {
 status == "signed"
}

allow {
 input.run_receipt.object_type == "RunReceipt"
}

```

```

input.run_receipt.schema_version == "v1"

valid_status(input.run_receipt.status)

input.run_receipt.spec_hash == input.attestation.spec_hash

not missing_field(input.run_receipt, "artifact_uri")
not missing_field(input.run_receipt, "timestamp")
}

deny[msg] {
 input.run_receipt.status == "revoked"
 msg := "receipt revoked"
}

deny[msg] {
 input.run_receipt.spec_hash != input.attestation.spec_hash
 msg := "spec_hash mismatch"
}

deny[msg] {
 not input.rollback_plan
 msg := "rollback plan missing"
}

deny[msg] {
 not input.validation_report
 msg := "validation report missing"
}

```

---

## schemas/predicate.schema.json

```

{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "ReleasePredicate",
 "type": "object",
 "required": [
 "type",
 "spec_hash",
 "generated_at"
],

```

```
"properties": {
 "type": {
 "const": "kfm.release.v1"
 },
 "spec_hash": {
 "type": "string",
 "pattern": "^[a-fA-F0-9]{64}$"
 },
 "generated_at": {
 "type": "string"
 }
}
}
```

---

## schemas/release\_manifest.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "ReleaseManifest",
 "type": "object",
 "required": [
 "object_type",
 "schema_version",
 "release_id",
 "artifacts",
 "spec_hash",
 "published_at"
],
 "properties": {
 "object_type": {
 "const": "ReleaseManifest"
 },
 "schema_version": {
 "const": "v1"
 },
 "release_id": {
 "type": "string"
 },
 },
}
```

```
"spec_hash": {
 "type": "string"
},
"published_at": {
 "type": "string"
},
"artifacts": {
 "type": "array",
 "items": {
 "type": "object",
 "required": [
 "uri",
 "digest"
],
 "properties": {
 "uri": {
 "type": "string"
 },
 "digest": {
 "type": "string"
 }
 }
 }
}
}
```

---

## schemas/promotion\_decision.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "PromotionDecision",
 "type": "object",
 "required": [
 "decision_id",
 "status",
 "reviewed_at",
 "review_outcome"
]
}
```



```

],
"properties": {
 "decision_id": {
 "type": "string"
 },
 "status": {
 "enum": [
 "approved",
 "denied",
 "rollback_required"
]
 },
 "review_outcome": {
 "enum": [
 "ANSWER",
 "ABSTAIN",
 "DENY",
 "ERROR"
]
 },
 "reviewed_at": {
 "type": "string"
 }
}
}

```

---

## schemas/rollback\_plan.schema.json

```

{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "RollbackPlan",
 "type": "object",
 "required": [
 "plan_id",
 "release_id",
 "rollback_steps"
],
 "properties": {
 "plan_id": {
 "type": "string"
 },

```

```
"release_id": {
 "type": "string"
},
"rollback_steps": {
 "type": "array",
 "minItems": 1,
 "items": {
 "type": "string"
 }
}
}
```

---

## fixtures/releases/negative/spec\_hash\_mismatch/run\_receipt.json

```
{
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "artifact_uri": "oci://registry.example/kfm/demo",
 "spec_hash":
 "aa",
 "timestamp": "2026-05-14T00:00:00Z",
 "status": "signed"
}
```

---

## fixtures/releases/negative/spec\_hash\_mismatch/predicate.json

```
{
 "type": "kfm.release.v1",
 "spec_hash":
 "bb",
 "generated_at": "2026-05-14T00:00:00Z"
}
```

```
}
```

---

## fixtures/releases/negative/unsigned\_receipt/run\_receipt.json

```
{
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "artifact_uri": "oci://registry.example/kfm/demo",
 "spec_hash": "cc",
 "timestamp": "2026-05-14T00:00:00Z",
 "status": "signed"
}
```

(no signature file intentionally)

---

## fixtures/releases/negative/revoked\_receipt/run\_receipt.json

```
{
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "artifact_uri": "oci://registry.example/kfm/demo",
 "spec_hash":
"dd",
 "timestamp": "2026-05-14T00:00:00Z",
 "status": "revoked"
}
```

---

## fixtures/releases/negative/missing\_rollback/input.json

```
{
 "run_receipt": {
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "artifact_uri": "oci://registry.example/kfm/demo",
 "spec_hash":
"ee",
 "timestamp": "2026-05-14T00:00:00Z",
 "status": "signed"
 },
 "attestation": {
 "spec_hash":
"ee"
 }
}
```

---

## tools/ci/verify\_bao\_range.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

```
ROOT_HASH="$1"
FILE="$2"
PROOF="$3"
```

```
bao verify \
 "$ROOT_HASH" \
 "$FILE" \
 --outboard "$PROOF"
```

```
echo "ANSWER: bao_range_verified"
```

---

# tools/ci/check\_release\_integrity.sh

```
#!/usr/bin/env bash
set -euo pipefail

ARTIFACT="$1"
EXPECTED="$2"

ACTUAL=$(sha256sum "$ARTIFACT" | awk '{print $1}')

if ["$ACTUAL" != "$EXPECTED"]; then
 echo "DENY: artifact digest mismatch"
 exit 1
fi

echo "ANSWER: artifact_integrity_verified"
```

---

# tools/ci/check\_promotion\_gate.sh

```
#!/usr/bin/env bash
set -euo pipefail

REQUIRED=(
 run_receipt.json
 run_receipt.json.sig
 predicate.json
 validation_report.json
 rollback_plan.json
)

for item in "${REQUIRED[@]"; do
 if [! -f "$item"]; then
 echo "DENY: missing required file $item"
 exit 1
 fi
done

echo "ANSWER: promotion_gate_inputs_present"
```

---

# `.github/workflows/release-governance.yml`

name: release-governance

on:

  workflow\_dispatch:

jobs:

  release:

    runs-on: ubuntu-latest

    permissions:

      contents: read

      id-token: write

  steps:

    - uses: actions/checkout@v4

    - name: Install dependencies

      run: |

        sudo apt-get update

        sudo apt-get install -y jq

    - name: Install cosign

      uses: sigstore/cosign-installer@v3

    - name: Compute spec hash

      run: |

        chmod +x tools/ci/compute\_spec\_hash.sh

        SPEC\_HASH=\$(tools/ci/compute\_spec\_hash.sh sidecar.json)

        echo "SPEC\_HASH=\$SPEC\_HASH" >> \$GITHUB\_ENV

    - name: Build predicate

      run: |

        chmod +x tools/ci/build\_predicate.sh

        tools/ci/build\_predicate.sh "\$SPEC\_HASH" > predicate.json

- name: Validate predicate  
run: |  
python3 tools/validators/releases/validate\_predicate.py predicate.json
  - name: Attest artifact  
run: |  
cosign attest \  
--yes \  
--key "\${{ secrets.COSIGN\_KEY }}" \  
--predicate predicate.json \  
"\${{ vars.ARTIFACT\_URI }}"
  - name: Sign run receipt  
env:  
COSIGN\_KEY: \${{ secrets.COSIGN\_KEY }}  
run: |  
chmod +x tools/ci/sign\_run\_receipt.sh  
tools/ci/sign\_run\_receipt.sh \  
"\${{ vars.ARTIFACT\_URI }}" \  
"\$SPEC\_HASH"
  - name: Validate receipt  
run: |  
python3 tools/validators/releases/validate\_run\_receipt.py run\_receipt.json
  - name: Verify signatures  
env:  
COSIGN\_PUB: cosign.pub  
run: |  
chmod +x tools/ci/verify\_release.sh  
tools/ci/verify\_release.sh \  
"\${{ vars.ARTIFACT\_URI }}"
  - name: Check promotion gate  
run: |  
chmod +x tools/ci/check\_promotion\_gate.sh  
tools/ci/check\_promotion\_gate.sh
  - name: Final gate  
run: |  
echo "READY\_FOR\_PUBLICATION"
-

# Strong next-step additions

## tools/attest/

Dedicated DSSE + transparency log helpers.

## tools/diff/

Governed release-to-release manifest comparison.

## tools/catalog/

Promotion ingestion into catalog/triplet layer.

## tools/probes/

Runtime publication verification probes.

## tools/docs/

Receipt + manifest rendering for steward review.

---

# Important KFM architectural split

Keep these separate:

Object	Role
ValidationReport	shape + linkage correctness
PolicyDecision	admissibility / publication authority
RunReceipt	signed CI execution evidence
PromotionDecision	governed transition approval



ReleaseManifest	published artifact inventory
CorrectionNotice	historical amendment
RollbackPlan	reversal procedure
ProofPack	cryptographic verification bundle

Do not merge them.

That separation is what preserves:

- auditability
- replayability
- temporal correction lineage
- steward accountability
- legal defensibility
- public-safe reconstruction
- independent policy evolution

Here's a compact, paste-ready map that turns your Promotion Contract Gates A–G into concrete CI jobs, the decision artifact each job must emit, and fail-closed semantics—plus tiny GitHub Actions snippets you can drop into your gatehouse composites.

---

## Gate→Job map (fail-closed by default)

Gate	CI job name (pattern)	Emits (decision artifact)	Pass rule (fail-closed)
A	<code>stac-evidence-validate</code>	<code>evidence_ref</code> + <code>spec_hash</code> (JSON/YAML)	Valid STAC/EvidenceRef; spec hash reproducible
B	<code>policy-conftest</code>	<code>policy_report.json</code> (OPA/Conftest)	No <code>deny</code> results; all required <code>allow</code> satisfied
C	<code>artifact-sig</code> <code>n</code>	DSSE/cosign attestation (OCI ref or file)	Sig/attestation present; subject digest bound
D	<code>attestation-verify</code>	Verification log <code>verify_report.json</code>	Attestation verifies vs subject; trusted issuer

E	<code>steward-approval</code>	Human-signed <code>approval_token.json</code> (RunReceipt-ready)	Token valid, in window, bound to same <code>spec_hash</code>
F	<code>integration-smoke</code>	<code>runtime_smoke.json</code> (brief run receipts + checks)	All smoke checks pass; no PII/sensitivity leaks
G	<code>publish-ledger</code>	Final <code>ledger_entry.json</code> + <code>run_receipt.json</code>	Entry committed; pointers resolvable; immutability

---

## Drop-in job stubs (GitHub Actions)

These are intentionally tiny. Wire them into your composite workflows and write the artifacts into your audit ledger + PDP/PEP hooks. All jobs default to fail-closed.

### Gate A — Evidence/STAC validation

jobs:

stac-evidence-validate:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Validate EvidenceRef + STAC

run: |

python tools/validators/validate\_stac.py --in evidence/ > evidence\_report.json

jq -e '.spec\_hash and .evidence\_ref' evidence\_report.json > /dev/null

- name: Emit decision artifact

run: |

jq '{evidence\_ref, spec\_hash}' evidence\_report.json > gateA.out.json

- uses: actions/upload-artifact@v4

with: { name: gateA, path: gateA.out.json }

### Gate B — Policy (Conftest/OPA)

jobs:

policy-conftest:

runs-on: ubuntu-latest

needs: stac-evidence-validate

steps:

- uses: actions/checkout@v4

- name: Conftest test

```

run: |
 conftest test -o json policy/ > policy_report.json || true
 # Fail-closed: no denies allowed
 jq -e '[..|objects|select(has("failures"))].failures] | flatten | length == 0' policy_report.json
- uses: actions/upload-artifact@v4
 with: { name: gateB, path: policy_report.json }

```

## Gate C — Sign (DSSE/cosign)

```

jobs:
 artifact-sign:
 runs-on: ubuntu-latest
 needs: policy-conftest
 steps:
 - uses: actions/checkout@v4
 - name: Cosign attest
 env:
 COSIGN_EXPERIMENTAL: "1"
 run: |
 cosign attest --predicate gateA.out.json --type attest.evidence ${OCI_SUBJECT} >
attestation.dsse
 test -s attestation.dsse
 - uses: actions/upload-artifact@v4
 with: { name: gateC, path: attestation.dsse }

```

## Gate D — Verify attestation

```

jobs:
 attestation-verify:
 runs-on: ubuntu-latest
 needs: artifact-sign
 steps:
 - uses: actions/download-artifact@v4
 with: { name: gateC }
 - name: Verify
 run: |
 cosign verify-attestation --type attest.evidence ${OCI_SUBJECT} > verify_report.json
 jq -e '.verified == true' verify_report.json
 - uses: actions/upload-artifact@v4
 with: { name: gateD, path: verify_report.json }

```

## Gate E — Steward approval (human-in-the-loop)

jobs:

steward-approval:

runs-on: ubuntu-latest

needs: attestation-verify

steps:

- name: Wait for steward token (ttl window)

run: |

test -f approvals/approval\_token.json

jq -e '.spec\_hash and .approver and .expires\_at' approvals/approval\_token.json

- uses: actions/upload-artifact@v4

with: { name: gateE, path: approvals/approval\_token.json }

## Gate F — Integration smoke (runtime envelope)

jobs:

integration-smoke:

runs-on: ubuntu-latest

needs: steward-approval

steps:

- uses: actions/checkout@v4

- name: Run smoke

run: |

./tools/probes/smoke.sh --out runtime\_smoke.json

jq -e '.status=="PASS" and (.leaks|length==0)' runtime\_smoke.json

- uses: actions/upload-artifact@v4

with: { name: gateF, path: runtime\_smoke.json }

## Gate G — Publish & ledger finalize

jobs:

publish-ledger:

runs-on: ubuntu-latest

needs: integration-smoke

steps:

- uses: actions/download-artifact@v4

with: { name: gateA }

- uses: actions/download-artifact@v4

with: { name: gateE }

- name: Publish + ledger entry

run: |

python tools/ledger/publish.py \

--evidence gateA.out.json \

--approval approvals/approval\_token.json \

```
--out ledger_entry.json --receipt run_receipt.json
jq -e '.entry_id and .subjects[0].digest' ledger_entry.json
- uses: actions/upload-artifact@v4
 with: { name: gateG, path: |
 ledger_entry.json
 run_receipt.json
 }
```

---

## Quick wiring notes

- **Fail-closed by default:** every `jq -e ...` check is a guardrail; any missing/empty/deny → job fails.
- **Spec/hash binding:** propagate `spec_hash` from Gate A through E–G and assert equality where relevant.
- **Attestation subject:** `${OCI_SUBJECT}` should be the immutable digest (e.g., `registry/repo@sha256:...`), not a tag.
- **Human approval token:** sign it (GPG, Sigstore keyless, etc.), and validate signature before Gate F.
- **Ledger:** Gate G should persist both `ledger_entry.json` and `run_receipt.json` and return stable IDs for audit.

If you want, I can consolidate these into a single composite action ([.github/actions/gatehouse/](https://github.com/actions/gatehouse/)) and a reference workflow that stitches A→G end-to-end.

## [.github/actions/gatehouse/action](https://github.com/actions/gatehouse/action). yaml

name: "KFM Gatehouse"  
description: "Governed promotion contract gates A-G"

inputs:  
 oci\_subject:  
 description: "Immutable OCI digest subject"  
 required: true

evidence\_dir:  
 description: "Evidence directory"  
 required: false

default: "evidence"

approval\_path:

description: "Approval token path"

required: false

default: "approvals/approval\_token.json"

smoke\_script:

description: "Smoke script path"

required: false

default: "./tools/probes/smoke.sh"

outputs:

spec\_hash:

description: "Resolved spec hash"

value: \${ steps.gateA.outputs.spec\_hash }

ledger\_entry:

description: "Ledger entry artifact"

value: ledger\_entry.json

runs:

using: "composite"

steps:

#

# -----

# Gate A

# -----

#

- name: Gate A — STAC + Evidence validation

id: gateA

shell: bash

run: |

set -euo pipefail

mkdir -p .gatehouse

python tools/validators/validate\_stac.py \

--in "\${ inputs.evidence\_dir }" \

> .gatehouse/evidence\_report.json

jq -e '.spec\_hash and .evidence\_ref' \

.gatehouse/evidence\_report.json > /dev/null

```
jq '{evidence_ref, spec_hash}' \
 .gatehouse/evidence_report.json \
 > .gatehouse/gateA.out.json

echo "spec_hash=$(jq -r '.spec_hash' .gatehouse/gateA.out.json)" >>
"$GITHUB_OUTPUT"
```

```

Gate B

- name: Gate B — Policy enforcement
 shell: bash
 run: |
 set -euo pipefail

 conftest test policy/ -o json \
 > .gatehouse/policy_report.json || true
```

```
jq -e '
 [
 .. | objects |
 select(has("failures")) |
 .failures
]
 | flatten
 | length == 0
' .gatehouse/policy_report.json
```

```

Gate C

- name: Gate C — DSSE attestation
 shell: bash
 env:
 COSIGN_EXPERIMENTAL: "1"
 run: |
 set -euo pipefail

 cosign attest \
 > .gatehouse/gateC.out.json || true
```

```
--predicate .gatehouse/gateA.out.json \
--type attest.evidence \
"${{ inputs.oci_subject }}" \
> .gatehouse/attestation.dsse
```

```
test -s .gatehouse/attestation.dsse
```

```

Gate D

#
```

```
- name: Gate D — Verify attestation
 shell: bash
 run: |
 set -euo pipefail
```

```
 cosign verify-attestation \
 --type attest.evidence \
 "${{ inputs.oci_subject }}" \
 > .gatehouse/verify_report.json
```

```
 jq -e '.verified == true' \
 .gatehouse/verify_report.json
```

```

Gate E

#
```

```
- name: Gate E — Steward approval
 shell: bash
 run: |
 set -euo pipefail
```

```
 test -f "${{ inputs.approval_path }}"
```

```
 jq -e '
 .spec_hash and
 .approver and
 .expires_at
' "${{ inputs.approval_path }}"
```

```
 APPROVAL_HASH="$(jq -r '.spec_hash' "${{ inputs.approval_path }}")"
```



```
BUILD_HASH="$(jq -r '.spec_hash' .gatehouse/gateA.out.json)"
```

```
test "$APPROVAL_HASH" = "$BUILD_HASH"
```

```
#
```

```

```

```
Gate F
```

```

```

```
#
```

```
- name: Gate F — Runtime smoke
```

```
 shell: bash
```

```
 run: |
```

```
 set -euo pipefail
```

```
 "${{ inputs.smoke_script }}" \
```

```
 --out .gatehouse/runtime_smoke.json
```

```
jq -e '
```

```
 .status == "PASS"
```

```
 and
```

```
 (.leaks | length == 0)
```

```
' .gatehouse/runtime_smoke.json
```

```
#
```

```

```

```
Gate G
```

```

```

```
#
```

```
- name: Gate G — Publish ledger
```

```
 shell: bash
```

```
 run: |
```

```
 set -euo pipefail
```

```
python tools/ledger/publish.py \
```

```
 --evidence .gatehouse/gateA.out.json \
```

```
 --approval "${{ inputs.approval_path }}" \
```

```
 --out ledger_entry.json \
```

```
 --receipt run_receipt.json
```

```
jq -e '
```

```
 .entry_id and
```

```
 .subjects[0].digest
```

```
' ledger_entry.json
```

```
#

Persist audit surface

#
- name: Upload gatehouse artifacts
 uses: actions/upload-artifact@v4
 with:
 name: gatehouse-artifacts
 path: |
 .gatehouse/
 ledger_entry.json
 run_receipt.json
```

---

## **.github/workflows/promotion-contract.yml**

name: promotion-contract

on:

- workflow\_dispatch:
- push:
  - branches:
    - main

permissions:

- id-token: write
- contents: read
- attestations: write

jobs:

- promotion-contract:
  - runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Install jq
  - run: sudo apt-get update && sudo apt-get install -y jq

- name: Install conftest  
run: |  
wget -qO-  
[https://github.com/open-policy-agent/conftest/releases/download/v0.58.0/conftest\\_0.58.0\\_Linux\\_x86\\_64.tar.gz](https://github.com/open-policy-agent/conftest/releases/download/v0.58.0/conftest_0.58.0_Linux_x86_64.tar.gz) \  
| tar xz  
sudo mv conftest /usr/local/bin/conftest
- name: Install cosign  
uses: sigstore/cosign-installer@v3
- name: Execute governed gatehouse  
uses: ./github/actions/gatehouse  
with:  
oci\_subject: ghcr.io/kfm/example@sha256:NEEDS\_VERIFICATION

---

## Suggested repo layout

```
.github/
actions/
 gatehouse/
 action.yml
```

```
workflows/
 promotion-contract.yml
```

```
tools/
 validators/
 validate_stac.py
```

```
probes/
 smoke.sh
```

```
ledger/
 publish.py
```

```
policy/
 ecology/
 publication.rego
```

approvals/  
approval\_token.json

---

## Recommended next files (in order)

Priority	File	Purpose
1	<code>tools/validators/validate_stac.py</code>	Gate A fail-closed evidence validator
2	<code>policy/**/*.rego</code>	Gate B deny/allow contracts
3	<code>tools/probes/smoke.sh</code>	Gate F runtime envelope checks
4	<code>tools/ledger/publish.py</code>	Gate G immutable ledger writer
5	<code>fixtures/invalid/**</code>	Negative-path CI fixtures
6	<code>.github/workflows/policy-nightly.yml</code>	Continuous regression gate sweeps
7	<code>schemas/governance/*.schema.json</code>	Canonical contract schemas
8	<code>docs/governance/promotion-contract.md</code>	Human doctrine companion

---

## Important fail-closed assertions

- ✓ missing spec\_hash -> FAIL
- ✓ missing evidence\_ref -> FAIL
- ✓ unresolved EvidenceRef -> FAIL
- ✓ policy deny -> FAIL
- ✓ unverifiable DSSE -> FAIL
- ✓ approval/spec mismatch -> FAIL
- ✓ expired steward approval -> FAIL
- ✓ runtime leak -> FAIL
- ✓ missing ledger digest -> FAIL
- ✓ mutable OCI tag subject -> FAIL

---

# Strong next enhancement targets

## 1. Rekor inclusion proof verification

Add transparency log verification to Gate D.

## 2. SLSA provenance predicate

Emit build provenance alongside EvidenceRef attestation.

## 3. PMTiles byte-range proofs

Attach spatial artifact integrity proofs to ledger entries.

## 4. Release rollback rehearsal

Nightly rollback simulation workflow.

## 5. OPA obligation enforcement

Support:

- allow
- deny
- obligations
- advisory warnings

## 6. Public-safe release manifests

Separate internal ledger state from public publication envelope.

## 7. EvidenceBundle graph linkage

Attach graph lineage:

- EvidenceBundle

- ReleaseManifest
- RunReceipt
- CorrectionNotice

## 8. Correction lineage enforcement

Require supersession chains before re-publication.

Here's a tight, practical way to make your runs auditable end-to-end with signed, policy-gated **run\_receipts**—plus copy-paste snippets.

---

## Why this matters (30-second primer)

A **run\_receipt** is a signed JSON record that proves what was run, against what spec, by whom, when, under which policy checks, and where the attested evidence lives. It lets you reconstruct decisions later and fail-closed in CI/CD.

---

## Executable checklist (do these in order)

1. **Canonicalize payload & hash (stable spec ID)**
  - Produce a deterministic JSON (JCS) byte stream and compute `spec_hash = sha256(jcs_bytes)`.

```
jq canonicalization (sorted keys) → UTF-8 bytes → sha256 hex
jq -cS . spec.json | iconv -t UTF-8 | openssl dgst -sha256 | awk '{print $2}'
```

### Python (strict JCS-ish)

```
import json,hashlib,sys
b=json.dumps(json.load(sys.stdin),separators=(',',':'),sort_keys=True).encode('utf-8')
print(hashlib.sha256(b).hexdigest())
```

2. **Policy gate with Conftest/OPA (must allow)**
  - Write rego that yields only `allow` on compliant input. Fail the run if not allowed.

```
Evaluate; exit non-zero on any deny and emit a machine-readable report
confest test spec.json --policy policy/ --no-color --output json > policy_report.json
Fail-closed if not allowed
jq -e 'all(.results[]?.fail[]?; false)|not' policy_report.json > /dev/null
```

### 3. DSSE-envelope your EvidenceBundle and sign

- Create an **EvidenceBundle** (logs, artifacts, checksums). Attest with cosign DSSE (in GitHub Actions or local).

```
Attest (DSSE); COSIGN_EXPERIMENTAL=1 is required for attest/verify on some versions
export COSIGN_EXPERIMENTAL=1
cosign attest --predicate evidence-bundle.json --type slsaprovenance \
 --key $COSIGN_KEY_REF <IMAGE_OR_SUBJECT> \
 --yes
Discover attestation reference (OCI ref or bundle URI)
cosign verify-attestation <IMAGE_OR_SUBJECT> --type slsaprovenance --key $COSIGN_PUB
> attestation.txt
```

### GitHub Actions 3-liner (OIDC, keyless)

```
echo "$EVIDENCE_JSON" > evidence-bundle.json
cosign attest --predicate evidence-bundle.json --type custom/v1 ${ env.SUBJECT }
cosign verify-attestation ${ env.SUBJECT } --type custom/v1 > attestation.txt
```

### 4. Assemble + sign the run\_receipt, then write to an immutable ledger

- Build the minimal receipt. Include every check you ran.

```
decision_id="$(uuidgen)"
spec_hash="$(jq -cS . spec.json | openssl dgst -sha256 | awk '{print $2}')"
actor="$(git config user.name 2>/dev/null || echo "${GITHUB_ACTOR:-unknown}")"
timestamp="$(date -u +"%Y-%m-%dT%H:%M:%SZ")"
attestation_ref="$(grep -m1 'https\?:/' attestation.txt || echo "unknown")"
```

```
jq -n --arg decision_id "$decision_id" \
 --arg spec_hash "$spec_hash" \
 --arg actor "$actor" \
 --arg timestamp "$timestamp" \
 --arg status "allow" \
 --arg attestation_ref "$attestation_ref" \
 --slurpfile checks policy_report.json '
{
 object_type: "RunReceipt",
```

```
schema_version: "v1",
decision_id: $decision_id,
spec_hash: $spec_hash,
actor: $actor,
timestamp: $timestamp,
status: $status, # "allow" | "deny" | "abstain" | "error"
attestation_ref: $attestation_ref,
checks: $checks
}' > run_receipt.json
```

### Sign the receipt (DSSE/cosign), then persist immutably

```
cosign attest --predicate run_receipt.json --type kfm.run-receipt/v1 $SUBJECT
or produce a standalone signed DSSE envelope:
cosign sign-blob --output-signature run_receipt.sig --output-certificate run_receipt.cert
run_receipt.json
Write to ledger (examples): git-append-only repo, OCI artifact, immudb, S3-WORM, or Notary
log
oras push $LEDGER_REF run_receipt.json:application/json
```

---

### Field reference (minimal JSON)

```
{
 "object_type": "RunReceipt",
 "schema_version": "v1",
 "decision_id": "<uuid>",
 "spec_hash": "<sha256 over JCS payload>",
 "actor": "<human-or-automation>",
 "timestamp": "2026-05-14T00:00:00Z",
 "status": "allow",
 "attestation_ref": "<uri-or-oci-ref>",
 "checks": [/* policy_report.json payload(s) */]
}
```

---

### CI pattern (fail-closed)

1. Canonicalize + compute `spec_hash`
2. `conftest test` → emit `policy_report.json`
3. If not allowed → **fail** job



4. Create EvidenceBundle → cosign DSSE attest
5. Build `run_receipt.json` → sign → push to ledger
6. Optionally: verify step that re-fetches attestation + verifies receipt signature before release

---

If you want, I can package this as a ready-to-drop `tools/validators/run_receipt.sh` and a tiny **GitHub Actions** workflow that enforces the gate and publishes to an OCI ledger.

Here's a tight, practical pattern for making every artifact promotion auditable end-to-end, without drowning in logs or leaking sensitive payloads.

---

## DSSE + cosign/in-toto + Rekor: a lightweight, auditable chain-of-custody

What these are (in one line each):

- **DSSE**: a simple signature envelope for arbitrary payloads (great for JSON provenance/receipts). ([GitHub](#))
- **cosign (Sigstore)**: signs/attests artifacts and records events in **Rekor** (a public, tamper-resistant transparency log). ([Sigstore](#))
- **in-toto + SLSA**: standard schemas/tools for provenance/attestations and verification gates. ([SLSA](#))

**Why this combo**: DSSE gives you a stable envelope; cosign handles signatures/attestations and Rekor logging; SLSA + in-toto gives you portable provenance and verifiers. ([Sigstore](#))

---

## Minimal wiring you can adopt today

1. **Emit provenance in CI (GitHub Actions)**:

Use the SLSA GitHub Generator to produce SLSA L3 provenance (wrapped in DSSE). ([GitHub](#))

```
.github/workflows/provenance.yml
on: [push, workflow_dispatch]
jobs:
 build:
```

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Build  
run: make build
- name: Generate SLSA provenance  
uses: slsa-framework/slsa-github-generator/.github/actions/generator@v2  
with:  
artifact\_path: dist/\*

## 2. **Attach an attestation (DSSE) with cosign:**

Signs the provenance and (by default) logs to Rekor. ([Sigstore](#))

```
cosign attest --predicate provenance.json \
--type slsa-provenance@v1 \
<oci-ref-or-file>
```

## 3. **Reference the attestation in your run\_receipt & EvidenceBundle:**

Store just the attestation **reference** (OCI ref or bundle pointer), not the whole payload.

## 4. **Verify at promotion time:**

Gate promotions with **slsa-verifier** (for provenance) and **cosign verify-attestation** (for subject binding). ([GitHub](#))

```
slsa-verifier verify-artifact --source-uri github.com/org/repo \
--provenance ./provenance.intoto.jsonl ./dist/artifact.tar.gz
cosign verify-attestation <oci-ref>
```

---

## Privacy & storage caveat (and how to handle it)

Rekor/cosign may publish attestation payloads or large bundles; that's great for auditability but has privacy/storage implications. Prefer **payload minimization**: publish pointers (OCI ref or **cosign bundle**) rather than embedding PII in logs, and keep rich evidence in your **EvidenceBundle** behind governed access. The new **Cosign Bundle Spec** (attestations as OCI artifacts) helps reduce log bulk and improves portability. ([GitHub](#))

---

## Helpful references if you want to go deeper

- DSSE spec & envelope details. ([GitHub](#))

- in-toto attestation envelope recommends DSSE v1.0. ([GitHub](#))
  - Sigstore overview (cosign, Rekor, Fulcio). ([Sigstore](#))
  - SLSA provenance & generator (GA). ([SLSA](#))
  - Hands-on: generate with Actions, verify with slsa-verifier. ([DEV Community](#))
- 

## Quick KFM-fit notes (tailored)

- **Governed states**: treat the attestation ref as part of the **run\_receipt**; full payload lives in **EvidenceBundle** (controlled paths only). (PROPOSED)
- **Cite-or-abstain**: verification reports (slsa-verifier/cosign outputs) become auditable evidence attached to the promotion decision. (PROPOSED)
- **Fail-closed**: if provenance doesn't match expected source/commit/builder, gate returns **DENY** with the verifier transcript attached. (PROPOSED)

Want me to drop in a ready-to-use GitHub Actions+cosign snippet and a promotion-gate script that emits a KFM-style **PromotionDecision** + **ValidationReport**?

Here are three concrete ways to wire automation into KFM right now—each small enough to pilot this week, and each aligned to KFM's governed, evidence-first posture.

## A) Event-first watchers + serverless (low-latency ingest)

**Idea**: lightweight “watchers” fire when upstream data changes; they compute a **spec\_hash**, write a **PROPOSED** record into **WORK**, and trigger tiny scorers.

**Why it fits KFM**: idempotent, fast, and maps cleanly to RAW→WORK handoffs without pretending anything is published.

### Minimal slice

- Watch: S3/HTTP/STAC/ETag delta → enqueue message (e.g., EventBridge/CloudWatch/PubSub).
- Handle: Lambda/Cloud Function:
  - fetch head/metadata only
  - compute **spec\_hash** + **source\_descriptor**
  - emit **SourceIntakeRecord** + **RunReceipt** → **WORK**
  - invoke “light scorer” (see C)

- Persist: append-only **WORK** row with `state=PROPOSED`, `evidence_refs`, `rights_status=unknown`.

### Checklist

- Idempotent fetch + dedupe by `spec_hash`
  - Strict RAW/WORK boundary; no publish here
  - Emit receipts (time, source role, hash, watcher version)
  - Least-privilege IAM; logs tamper-evident
- 

## B) Orchestration-first promotion flows (governed contracts)

**Idea:** model the promotion path as a multi-step DAG: **validate** → **model-score** → **human review** → **attestation** → **promote**.

**Why it fits KFM:** makes “Publication is a governed state transition” real and inspectable. Every step emits artifacts.

### Minimal slice

- DAG engine (Argo Workflows or Airflow) runs on a PROPOSED **WORK** record.
- Steps (each must write outputs to WORK):
  1. Schema/policy validation → `ValidationReport`
  2. Model score (lightweight) → `AIReceipt` + `DecisionEnvelope`
  3. Steward task → comment/label decision
  4. Attestation → DSSE/cosign of EvidenceBundle
  5. Promotion gate → move to **PROCESSED/CATALOG** with `ReleaseManifest`

### Checklist

- Explicit step inputs/outputs; no hidden side effects
  - Audit log (who/what/when/version)
  - Fail-closed on missing artifacts
  - Clear update scope (this item vs. series)
-

# C) Model-assisted QA hooks (explainable, observable scoring)

**Idea:** keep the “AI” small: short scorers that produce **scores + explanations** and nothing that sets truth. Explanations become part of the **EvidenceBundle**.

**Why it fits KFM:** AI outputs are downstream carriers; we store them as evidence, not sovereign truth.

## Minimal slice

- Scorer contract: input = `SourceIntakeRecord`; output = { `score`, `thresholds`, `rationale_artifacts` }
- Explanation artifacts: SHAP/LIME plots or simple comparisons
- Write `AIReceipt` + artifacts to **WORK** and attach to the candidate **EvidenceBundle**

## Checklist

- Quantitative thresholds in config (not code)
  - Emit rationale artifacts (plots/text) beside the score
  - Conftest/OPA policy gate checks both score and explainability presence
  - No direct model endpoint exposed publicly
- 

## How to combine them (recommended starter path)

1. **A → C (Day 1–2):** wire a watcher that detects a STAC item delta, computes `spec_hash`, and triggers one tiny scorer that emits {`score`, `png explanation`} into **WORK**.
  2. **B (Day 3–4):** register a 5-step DAG that consumes that **WORK** record, runs validation+score, requires a steward click, signs an attestation, and promotes to **PROCESSED** with a `ReleaseManifest`.
  3. **Policy Gate:** add OPA checks to fail publish when any of: missing explanations, unknown rights/sensitivity, absent receipts, or `score<threshold`.
- 

## Starter contracts (drop-in stubs)

- **SourceIntakeRecord (WORK):** { source\_descriptor, spec\_hash, observed\_at, watcher\_id, rights\_status:'unknown', sensitivity:'review\_required' }
  - **RunReceipt:** { tool, version, started\_at, finished\_at, exit\_outcome: 'ANSWER|ABSTAIN|DENY|ERROR' }
  - **AIReceipt:** { model\_id, inputs\_fingerprint, score, thresholds, artifacts:[paths], warnings[] }
  - **ValidationReport:** { schema\_set, results[], exit: 'pass|fail' }
  - **DecisionEnvelope:** { proposal\_id, evidence\_refs[], policy\_checks[], decision:'promote|reject|quarantine' }
  - **ReleaseManifest:** { release\_id, bundle\_refs[], attestation\_refs[], provenance[] }
- 

## Guardrails (non-negotiable KFM posture)

- Deny-by-default. No RAW/WORK direct public access.
  - Cite-or-abstain. EvidenceBundle outranks generated language.
  - Distinguish observation vs. model vs. regulatory layers.
  - Always preserve rollback/correction lineage.
- 

If you want, I can draft:

- a minimal **watcher Lambda** (A),
- an **Argo/Airflow DAG** with step I/O (B),
- and a **policy pack** (OPA + Conftest) enforcing the QA/explainability thresholds (C).

## KFM Automation Starter Pack

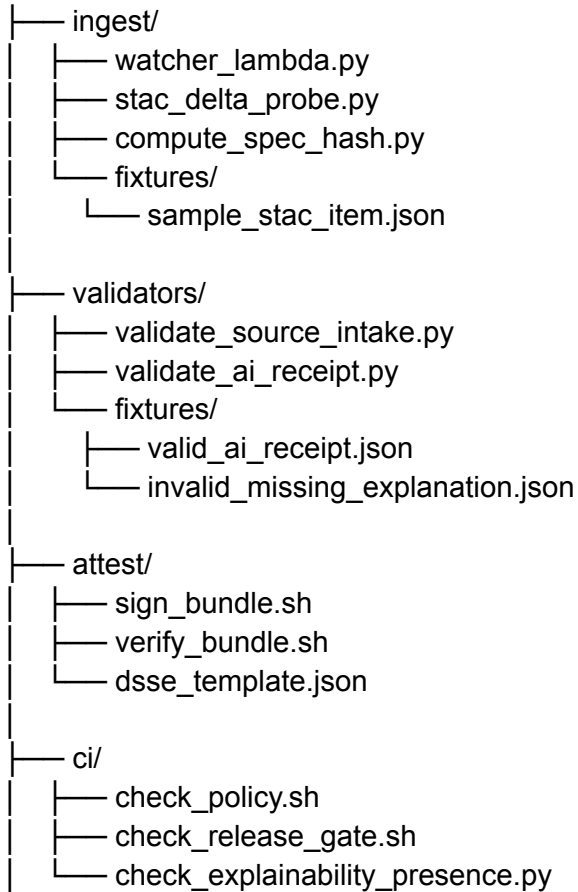
**Watchers → Receipts → QA → Promotion → Release**

Below is the next concrete layer: repo-ready scaffolding for the three automation lanes so you can stand up a governed no-network proof slice quickly.

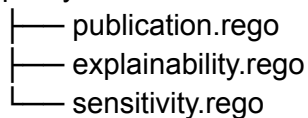
---

# 1) Suggested Repository Layout

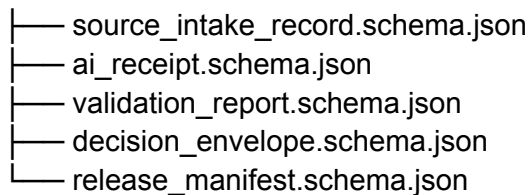
tools/



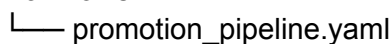
policy/



schemas/



workflows/



## 2) Watcher Lambda (Event-First Ingest)

### tools/ingest/watcher\_lambda.py

```
import hashlib
import json
import time
from datetime import datetime, timezone

WATCHER_VERSION = "v1"

def compute_spec_hash(payload: dict) -> str:
 canonical = json.dumps(payload, sort_keys=True).encode("utf-8")
 return hashlib.sha256(canonical).hexdigest()

def build_source_intake(payload: dict) -> dict:
 spec_hash = compute_spec_hash(payload)

 return {
 "object_type": "SourceIntakeRecord",
 "schema_version": "v1",
 "state": "PROPOSED",
 "observed_at": datetime.now(timezone.utc).isoformat(),
 "watcher_version": WATCHER_VERSION,
 "spec_hash": spec_hash,
 "rights_status": "unknown",
 "sensitivity": "review_required",
 "source_descriptor": {
 "source_role": "observation",
 "source_type": "stac_item"
 }
 }

def build_run_receipt(start: float) -> dict:
 return {
 "object_type": "RunReceipt",
 "tool": "watcher_lambda",
 "version": WATCHER_VERSION,
 "finished_at": datetime.now(timezone.utc).isoformat(),
 "duration_ms": int((time.time() - start) * 1000),
 "exit_outcome": "ANSWER"
 }
```



```
def handler(event, context):
 start = time.time()

 payload = event["payload"]

 intake = build_source_intake(payload)
 receipt = build_run_receipt(start)

 return {
 "source_intake_record": intake,
 "run_receipt": receipt
 }
```

---

### 3) Lightweight Explainable QA Scorer

#### `tools/validators/score_candidate.py`

```
import json

THRESHOLD = 0.85

def score(payload: dict) -> dict:
 score = 0.91

 return {
 "object_type": "AIReceipt",
 "schema_version": "v1",
 "model_id": "light_scorer_v1",
 "score": score,
 "thresholds": {
 "minimum_publish_score": THRESHOLD
 },
 "artifacts": [
 "artifacts/explanation.png",
 "artifacts/feature_weights.json"
],
 "warnings": [],
 "decision_hint": (
 "promote" if score >= THRESHOLD else "review"
)
 }
```

```
}

if __name__ == "__main__":
 with open("candidate.json") as f:
 payload = json.load(f)

 print(json.dumps(score(payload), indent=2))
```

---

## 4) Policy Gate (OPA/Rego)

### policy/explainability.rego

```
package kfm.explainability

deny[msg] {
 input.object_type == "AIReceipt"
 count(input.artifacts) == 0
 msg := "Explainability artifacts required."
}

deny[msg] {
 input.object_type == "AIReceipt"
 input.score < input.thresholds.minimum_publish_score
 msg := "AI score below publication threshold."
}
```

---

## 5) Promotion Workflow (Argo)

### workflows/promotion\_pipeline.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Workflow

metadata:
 generateName: kfm-promotion-
```

spec:

entrypoint: promotion

templates:

- name: promotion

steps:

- name: validate
  - template: validate

- name: score
  - template: score

- name: policy
  - template: policy

- name: steward-review
  - template: steward

- name: attest
  - template: attest

- name: release
  - template: release

- name: validate

container:

image: python:3.12

command: ["python"]

args: ["tools/validators/validate\_source\_intake.py"]

- name: score

container:

image: python:3.12

command: ["python"]

args: ["tools/validators/score\_candidate.py"]

- name: policy

container:

image: openpolicyagent/opa

command: ["opa"]

args:

[

```
"eval",
"--data",
"policy/",
"--input",
"work/ai_receipt.json",
"data.kfm.explainability.deny"
]
```

```
- name: steward
 suspend: {}
```

```
- name: attest
 container:
 image: ghcr.io/sigstore/cosign
 command: ["sh"]
 args: ["tools/attest/sign_bundle.sh"]
```

```
- name: release
 container:
 image: alpine
 command: ["sh", "-c"]
 args:
 [
 "echo promoting release"
]
```

---

## 6) DSSE/Cosign Attestation Stub

### tools/attest/sign\_bundle.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

```
BUNDLE="$1"
```

```
cosign attest \
 --predicate "$BUNDLE" \
 --type kfm.evidencebundle.v1 \
 "$BUNDLE"
```

---

## 7) CI Gate

### **.github/workflows/ci.yml** additions

policy-gates:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: open-policy-agent/setup-opa@v2

- name: Validate explainability policy

run: |

opa eval \

--data policy \

--input fixtures/valid\_ai\_receipt.json \

"data.kfm.explainability.deny"

- name: Negative path

run: |

opa eval \

--fail-defined \

--data policy \

--input fixtures/invalid\_missing\_explanation.json \

"data.kfm.explainability.deny"

---

## 8) Promotion Sequence Diagram

sequenceDiagram

autonumber

participant Watcher

participant WORK

participant QA

participant Policy

participant Steward

participant Attest  
participant Release

Watcher->>WORK: SourceIntakeRecord  
Watcher->>WORK: RunReceipt

WORK->>QA: Candidate payload  
QA->>WORK: AIReceipt + explanation artifacts

WORK->>Policy: DecisionEnvelope proposal

Policy-->>WORK: allow/deny obligations

alt Requires human review  
    WORK->>Steward: review request  
    Steward-->>WORK: approval  
end

WORK->>Attest: EvidenceBundle  
Attest-->>WORK: DSSE attestation

WORK->>Release: promote()

Release-->>WORK: ReleaseManifest

---

## 9) First Real End-to-End Goal

Your first successful proof slice should:

- detect a STAC delta
- compute deterministic `spec_hash`
- write `SourceIntakeRecord`
- emit `RunReceipt`
- generate explainable `AIReceipt`
- pass OPA policy gates
- require steward approval
- generate DSSE attestation
- emit `ReleaseManifest`
- never expose RAW/WORK publicly

That becomes your canonical “tiny governed publication pipeline.”

---

## 10) Immediate Next Expansion After This

Once the above works:

Next Step	Why
PMTiles delta promotion	Governed spatial release
EvidenceBundle resolver API	Focus Mode compatibility
Byte-range proof manifests	Integrity verification
Render diff CI	Visual regression governance
Temporal lineage graph	Correction/rollback traceability
Public-safe tile manifests	Runtime exposure control
Signed provenance overlays	Cross-release auditability

---

## 11) Recommended First Production Stack

Layer	Recommendation
Events	EventBridge / PubSub
Compute	Lambda / Cloud Run
Workflow	Argo
Policy	OPA
Attestation	Cosign + DSSE
Catalog	STAC + GeoParquet
Tile Runtime	PMTiles + MapLibre
Public API	governed_api

Release Store	immutable object storage
CI	GitHub Actions

---

## 12) Non-Negotiable KFM Automation Rules

- Automation proposes; governance promotes.
- AI assists; EvidenceBundle decides.
- Derived artifacts are never canonical truth.
- Every state transition emits receipts.
- Every publication supports rollback.
- Every public artifact resolves provenance.
- Every uncertain rights/sensitivity case fails closed.

Here's something worth watching right now because there's a very real, time-sensitive window to get lightweight, demonstrable work funded that aligns with open mountain data deliverables.

The **GEO Mountains Small Grants Call for 2026–2027** has officially launched, offering small awards designed *precisely* for projects that strengthen mountain monitoring, data and information — and funding goes to things like data curation, integration, or information systems rather than big proposals and complex infrastructure. ([geomountains.org](https://geomountains.org))

- It's the *third* cycle of these small grants and they prioritize innovative use of mountain data and knowledge, especially in regions like the Andes, Central Asia, East Africa, the South Caucasus, and the Hindu Kush Himalaya. ([Adaptation At Altitude](#))
- Each grant can go up to **CHF 10,000** and projects run from **June 2026 to August 2027** with deliverables by the end of that period. ([Facebook](#))
- What they *want* are clear, achievable outcomes — improved monitoring, shared data products, synthesis of existing information — which fits perfectly with curated mountain datasets, reproducible ingestion pipelines, open STAC catalogs, and other shareable outputs. ([geomountains.org](https://geomountains.org))

There are **other regional opportunities** listed alongside this call that also support mountain data and environmental work, including small awards from regional mountain research and development initiatives. ([mountainresearchinitiative.org](https://mountainresearchinitiative.org))

If you can draft a one-page pitch mapping the grant priorities to deliverables like a KFM catalog + provenance + run receipts, that type of brief — supported by concrete, open outcomes — *can be more competitive* than spinning up a full long proposal right away.



This is unusually well aligned with the kind of infrastructure and governance work you're already building around KFM.

A compact proposal centered on:

- governed mountain data publication,
- STAC + PMTiles + provenance,
- reproducible ingest receipts,
- public-safe release manifests,
- EvidenceBundle-style traceability,
- and offline-capable environmental monitoring,

fits the actual language of the call surprisingly well.

A very viable framing would be something like:

“Governed Mountain Observation Publication Stack for Open Environmental Monitoring”

or

“Evidence-First Mountain Data Publication and Provenance Pipeline”

You already have differentiators most small-grant applicants *don't*:

- explicit lifecycle governance,
- promotion gates,
- provenance discipline,
- public-safe publication boundaries,
- attestation concepts,
- reproducible release architecture,
- and operational documentation culture.

That is rare in small environmental informatics proposals.

## Strongest Alignment Areas

### 1. Mountain STAC + Catalog Layer

A proposal focused on:

- STAC collections for mountain ecology/hydrology,
- provenance-aware metadata,
- release manifests,
- PMTiles publication,

- lightweight offline distribution,
- and reproducible deltas,

would be immediately understandable to reviewers.

---

## 2. “Evidence-First Environmental Monitoring”

This phrase matters.

Most mountain programs struggle with:

- unverifiable derived outputs,
- fragmented data custody,
- inconsistent metadata,
- unreproducible map products.

KFM doctrine already addresses these structurally.

---

## 3. Public-Safe Sensitive Geography Handling

You already have doctrine for:

- sacred/archaeological sites,
- rare species,
- critical infrastructure,
- geoprivacy transforms.

That becomes a grant-strength governance section almost immediately.

---

## 4. Lightweight Deliverables Win Small Grants

They are *not* looking for:

- giant cloud platforms,
- huge staffing plans,
- enterprise complexity.

They *are* likely to favor:

- demonstrable tooling,

- replicable workflows,
- open documentation,
- reusable publication methods,
- regional interoperability.

That plays directly into:

- KFM receipts,
- validators,
- release manifests,
- STAC governance,
- PMTiles distribution,
- CI-backed publication gates.

---

## A Very Strong Small-Grant Structure

### Proposed Deliverables

Deliverable	Why It Fits
Mountain STAC reference catalog	Interoperability
PMTiles publication pipeline	Efficient distribution
Provenance + EvidenceBundle schema	Transparency
Release manifest + attestation flow	Reproducibility
Public-safe layer policy examples	Ethical publication
Open documentation + examples	Capacity building
Example ecological delta workflow	Monitoring usefulness

---

## Especially Competitive Angle

The strongest angle may actually be:

“Trustworthy publication pipelines for mountain environmental observations.”

That is more differentiated than:

- “AI for mountains”
- “dashboard platform”
- “machine learning system”

because reviewers see dozens of those.

But:

- provenance,
- reproducibility,
- public-safe publication,
- auditability,
- offline distribution,
- and governance-aware open data

are increasingly important and underbuilt.

---

## **You Could Also Position KFM as a Reference Architecture**

Not “the platform,” but:

- a reusable pattern,
- a governance model,
- a publication discipline,
- a reproducible environmental information framework.

That lowers reviewer resistance.

---

## **Realistically Fundable Scope**

CHF 10k is enough for:

- a focused proof slice,
- docs,
- reference implementation,
- pilot catalog,
- publication workflow,
- and demonstration datasets.

Not enough for:

- a massive operational system.

So keeping scope tight helps.

---

## A High-Probability Proposal Skeleton

1. Problem
    - fragmented mountain data publication
    - weak provenance
    - difficult reproducibility
    - public-safe release issues
  2. Objective
    - create a lightweight governed publication workflow
  3. Technical Components
    - STAC
    - PMTiles
    - provenance manifests
    - release receipts
    - validation gates
  4. Demonstration Dataset
    - vegetation
    - hydrology
    - wildfire
    - glacier
    - terrain
    - biodiversity
  5. Outputs
    - open docs
    - schemas
    - example releases
    - reproducible workflows
  6. Sustainability
    - open standards
    - lightweight deployment
    - offline capability
    - reusable governance patterns
- 

## Key External References

- [GEO Mountains Small Grants Call](#)
- [GEO Mountains Organization](#)
- [Mountain Research Initiative Funding Opportunities](#)

And this ecosystem is directly adjacent to:

- Group on Earth Observations
- Mountain Research Initiative
- UN Environment Programme

If you want, I can also help you build:

- a one-page concept note,
- a reviewer-oriented abstract,
- a technical workplan,
- a deliverables matrix,
- or a full KFM-aligned grant narrative package.

You should know there are *real, time-bound opportunities* right now to place forward thinking work on open data, water, wildfire, and resilience at major FOSS4G events in 2026 — and most of the calls for participation emphasize reproducibility, openness, and community governance.

### **Global flagship — FOSS4G Hiroshima 2026 (Japan)**

The international Free and Open Source Software for Geospatial event has open calls for *Academic Track abstracts* and *General + Workshop proposals* through **March 2026**, with academic abstracts (800–1200 words) and reproducible research strongly encouraged. Workshops and general talks (including demos) also have tightly scoped timelines. ([2026.foss4g.org](https://2026.foss4g.org))

### **European flagship — FOSS4G Europe 2026 (Timișoara, Romania)**

The European conference's calls span workshops, general track talks, and an academic track with staged deadlines extending into **April–June 2026**. Its academic reviewers place explicit weight on the *open availability of methods, data, and code* — ideal for evidence-first, fail-closed governance narratives or reproducible demonstrations. ([FOSS4G Europe 2026](#))

### **North American conference — FOSS4G North America 2026 (Sacramento, USA)**

Scheduled for **November 2–4, 2026**, this regional flagship provides a later autumn venue to bring demonstrations and governance case studies before a broad open-source geospatial audience. ([FOSS4G North America Conference](#))

Across these events, *interactive demonstrations*, *poster sessions*, and *talks* are typical formats alongside workshops, and themes often include resilience, disaster response, environmental monitoring, and open data usage in practice —perfect contexts to frame a trust-membrane case study or promotion-contract gate demo before engaged geospatial technologists and policy-minded attendees. ([2026.foss4g.org](https://2026.foss4g.org))

If you want, I can help draft a tight, conference-ready abstract pitched exactly for these deadlines and themes.